ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TÀI LIỆU ÔN TẬP TRỌNG TÂM

KIẾN TRÚC MÁY TÍNH (PHIÊN BẢN RISC-V)

Mã học phần: IT3030, IT3034, IT3283

Nội dung tài liệu:

- Hệ thống hóa lý thuyết từng bước từ cơ bản, giải thích rõ bản chất và cách đổi đơn vị.
- Chuẩn hóa toàn bộ theo kiến trúc tập lệnh RISC-V mới nhất.
- Tổng hợp toàn bộ câu hỏi và bài tập từ các đề thi Giữa kỳ, Cuối kỳ và bài giảng chính thức.
- Hướng dẫn giải chi tiết ngay dưới từng câu hỏi để rèn luyện kỹ năng làm bài.

Hà Nội – 2026

Mục lục

1 GIỚI THIỆU CHUNG VÀ HIỆU NĂNG MÁY TÍNH **4**

1.1 Phần 1: Lý thuyết trọng tâm 4

1.1.1 Khái niệm cơ bản và các thành phần của máy tính 4

1.1.2 Phân loại máy tính và cấu hình thực tế 5

1.1.3 Ba khía cạnh của Kiến trúc máy tính và các mức ngôn ngữ 6

1.1.4 Đánh giá hiệu năng máy tính (Trọng tâm tính toán) 6

1.2 Phần 2: Toàn bộ bài tập Chương 1 và Hướng dẫn giải 9

1.2.1 Nhóm 1: Các thành phần máy tính và Phân cấp bộ nhớ 9

1.2.2 Nhóm 2: Tính chu kỳ xung nhịp (T_0), CPI và thời gian thực thi (t_{CPU}) 12

1.2.3 Nhóm 3: Bài tập về chỉ số tốc độ MIPS 15

1.2.4 Nhóm 4: Tính CPI trung bình và thời gian với nhiều lớp lệnh 18

1.2.5 Nhóm 5: So sánh hiệu năng và thời gian giữa các bộ xử lý 21

1.2.6 Nhóm 6: CPI hiệu dụng và Phân loại kiến trúc máy tính 28

2 HỆ THỐNG MÁY TÍNH **31**

2.1 Phần 1: Lý thuyết trọng tâm 31

2.1.1 Chi tiết các thành phần cơ bản của máy tính 31

2.1.2 Ba hoạt động cơ bản của máy tính 32

2.1.3 Liên kết hệ thống (Cấu trúc Bus và Không gian địa chỉ) 34

2.2 Phần 2: Toàn bộ bài tập Chương 2 và Hướng dẫn giải 36

2.2.1 Nhóm 1: Chu trình lệnh, các thanh ghi PC/IR/SP và Ngăn nhớ 36

2.2.2 Nhóm 2: Hệ thống Bus và Tính toán độ rộng Bus địa chỉ 38

2.2.3 Nhóm 3: Bài toán xác định Băng nhớ vật lý (Memory Banks) 43

2.2.4 Nhóm 4: Các phương pháp Vào-Ra (Polling, Interrupt, DMA) và Thiết bị lưu trữ 46

3 KIẾN TRÚC TẬP LỆNH RISC-V **51**

3.1 Phần 1: Lý thuyết trọng tâm 51
3.1.1 Tập 32 thanh ghi của RISC-V 32-bit 51
3.1.2 Tổ chức dữ liệu trong bộ nhớ và Quy tắc Little-Endian 52
3.1.3 Các nhóm lệnh hợp ngữ RISC-V cốt lõi 52

4 KIẾN TRÚC TẬP LỆNH RISC-V 56

4.1 Phần 1: Lý thuyết trọng tâm 56
4.1.1 Tập 32 thanh ghi của RISC-V 32-bit 56
4.1.2 Tổ chức dữ liệu trong bộ nhớ và Quy tắc Little-Endian 57
4.1.3 Các nhóm lệnh hợp ngữ RISC-V cốt lõi 57
4.1.4 Mã máy 32-bit của RISC-V và 4 Phương thức định địa chỉ 59
4.2 Phần 2: Toàn bộ bài tập Chương 3 và Hướng dẫn giải 62
4.2.1 Nhóm 1: Thanh ghi, Phương thức định địa chỉ và Kiểm tra hợp lệ . . . 62
4.2.2 Nhóm 2: Thực thi lệnh Số học, Logic, Dịch bit và Khởi tạo thanh ghi . 66
4.2.3 Nhóm 3: Lệnh so sánh (`slt, sltu, blt, bltu`) và Rẽ nhánh cơ bản . 74
4.2.4 Nhóm 4: Truy cập bộ nhớ (`lw, lh, lb, sw`) theo thứ tự Little-Endian 76
4.2.5 Nhóm 5: Chuyển đổi câu lệnh C sang Hợp ngữ RISC-V (Mảng, Con trỏ, Rẽ nhánh) 81
4.2.6 Nhóm 6: Phân tích Vòng lặp và Đếm tổng số lệnh thực thi 88
4.2.7 Nhóm 7: Chương trình con, Ngăn xếp (Stack) và Lệnh `jal / jalr` . . 92
4.2.8 Nhóm 8: Dịch Mã máy ↔ Hợp ngữ, Tính địa chỉ đích và Lập trình Vào-Ra 95

5 SỐ HỌC MÁY TÍNH 104

5.1 Phần 1: Lý thuyết trọng tâm 104
5.1.1 Hệ đếm và Chuyển đổi giữa các hệ đếm 104
5.1.2 Đại số Boolean, Cổng logic và Mạch số cơ bản 105
5.1.3 Biểu diễn Số nguyên không dấu và Số nguyên có dấu (Mã bù 2) 105
5.1.4 Cộng/Trừ số nguyên và Hiện tượng Tràn số học (Overflow) 106
5.1.5 Nhân/Chia số nguyên và Mở rộng RVM trong RISC-V 107
5.1.6 Số thực dấu phẩy động chuẩn IEEE 754 108

5.2 Phần 2: Toàn bộ bài tập Chương 4 và Hướng dẫn giải 110

5.2.1 Nhóm 1: Nhận diện Cổng logic và Biểu thức Đại số Boolean 110

5.2.2 Nhóm 2: Chuyển đổi và So sánh Số nguyên có dấu (Mã bù 2) 111

5.2.3 Nhóm 3: Phép cộng/trừ số nguyên và Hiện tượng Tràn số học (Overflow) 114

5.2.4 Nhóm 4: Lệnh Chia và Lấy phần dư số nguyên (`div, rem`) 117

5.2.5 Nhóm 5: Số thực dấu phẩy động chuẩn IEEE 754 32-bit 118

CHƯƠNG 1

GIỚI THIỆU CHUNG VÀ HIỆU NĂNG MÁY TÍNH

1.1 Phần 1: Lý thuyết trọng tâm

1.1.1 Khái niệm cơ bản và các thành phần của máy tính

1. Máy tính và Chương trình là gì?

- **Máy tính (Computer):** Là thiết bị điện tử hoạt động tự động theo chương trình. Máy tính nhận dữ liệu đầu vào (Input data), lưu trữ, xử lý dữ liệu đó và xuất ra thông tin kết quả (Output information).
- **Chương trình (Program):** Là một dãy các lệnh được viết sẵn và lưu trong bộ nhớ để điều khiển máy tính thực hiện công việc.

2. Bốn thành phần cơ bản của mọi máy tính

Dù là điện thoại thông minh, máy tính cá nhân hay siêu máy tính, mọi máy tính đều cấu tạo từ 4 thành phần cốt lõi:

1. **Bộ xử lý trung tâm (CPU - Central Processing Unit):** Thành phần quan trọng nhất và nhanh nhất, có nhiệm vụ điều khiển hoạt động của toàn bộ máy tính và trực tiếp xử lý dữ liệu.
2. **Bộ nhớ chính (Main Memory - thường gọi là RAM):** Chứa các lệnh và dữ liệu của chương trình **đang được thực hiện**. (Lưu ý: Các đề thi trắc nghiệm rất hay hỏi câu này. Ổ cứng SSD/HDD là bộ nhớ ngoài dùng để lưu trữ lâu dài, còn chương trình đang chạy bắt buộc phải nằm ở Bộ nhớ chính).
3. **Hệ thống vào-ra (Input/Output - I/O):** Trao đổi thông tin giữa máy tính với thế giới bên ngoài (bàn phím, chuột, màn hình, mạng, ổ cứng...).
4. **Liên kết hệ thống (System Interconnection):** Hệ thống các đường truyền (Bus) kết nối và vận chuyển thông tin giữa CPU, Bộ nhớ chính và Hệ thống vào-ra.

1.1.2 Phân loại máy tính và cấu hình thực tế

1. Năm nhóm máy tính phổ biến

- **Máy tính cá nhân (PC - Personal Computer):** Máy để bàn (Desktop), máy xách tay (Laptop). Dùng cho mục đích đa dụng cá nhân.
- **Thiết bị di động cá nhân (PMD - Personal Mobile Devices):** Điện thoại thông minh (Smartphone), máy tính bảng (Tablet).
- **Máy tính nhúng / IoT (Embedded Computers / Internet of Things):** Máy tính được đặt ẩn bên trong một thiết bị khác (trong ô tô, máy giặt, tivi) để điều khiển chuyên dụng. Nếu máy tính nhúng có kết nối Internet thì được gọi là thiết bị IoT.
- **Máy chủ (Servers):** Máy tính có hiệu năng và độ tin cậy cao, hoạt động liên tục trên mạng để cung cấp dịch vụ cho nhiều người dùng.
- **Máy tính quy mô kho / Siêu máy tính (WSC / Supercomputers):** Kết nối hàng nghìn máy chủ với nhau (Clusters), phục vụ tính toán khoa học kỹ thuật phức tạp, trị giá hàng triệu đến hàng trăm triệu USD.

2. Thứ tự dung lượng bộ nhớ và cách chọn cấu hình PC hợp lý

Trong đề thi thường có câu hỏi yêu cầu sắp xếp dung lượng bộ nhớ hoặc chọn một cấu hình máy tính PC hợp lý. Bạn chỉ cần nhớ nguyên tắc: **Bộ nhớ càng gần CPU thì tốc độ càng nhanh nhưng dung lượng càng nhỏ**.

Thứ tự dung lượng tăng dần (từ nhỏ nhất đến lớn nhất):

Tập thanh ghi < Cache L1 < Cache L2 < Cache L3 < Bộ nhớ chính (RAM) < Bộ nhớ ngoài (SSD/HDD)

Cách nhận biết nhanh cấu hình PC đúng trong đề thi trắc nghiệm:

- **Cache L1:** Rất nhỏ, chỉ tính bằng **KB** (thường là 32KB hoặc 64KB). Nếu đáp án ghi Cache L1 là MB hay GB → Loại ngay.
- **Cache L2:** Lớn hơn L1, thường vài trăm **KB** (ví dụ: 256KB hoặc 512KB). Nếu ghi 256GB → Loại ngay.
- **Cache L3:** Tính bằng **MB** (thường là 8MB, 16MB). Nếu ghi 16GB → Loại ngay.
- **RAM (Bộ nhớ chính):** Tính bằng **GB** (thường từ 4GB, 8GB, 16GB, 32GB). Nếu ghi 2TB cho PC thông thường → Loại ngay.
- **SSD (Ổ cứng):** Dung lượng lớn nhất, thường là **512GB** hoặc **1TB**.

1.1.3 Ba khía cạnh của Kiến trúc máy tính và các mức ngôn ngữ

1. Ba khía cạnh nghiên cứu máy tính

- **Kiến trúc tập lệnh (ISA - Instruction Set Architecture):** Là cách nhìn máy tính của người lập trình hợp ngữ. Nó đóng vai trò là **giao diện giữa phần cứng và phần mềm**. Các ISA phổ biến: Intel x86 (dùng cho PC, Server), ARM (dùng cho điện thoại), và **RISC-V** (kiến trúc tập lệnh mở, miễn phí bản quyền, ra đời năm 2010 tại ĐH UC Berkeley).
- **Tổ chức máy tính (Computer Organization) hay Vi kiến trúc (Microarchitecture):** Cách thiết kế phần cứng ở mức cao để thực thi tập lệnh đó (thiết kế bên trong CPU, đường ống lệnh, bộ nhớ Cache, cấu trúc Bus).
- **Phần cứng (Hardware):** Công nghệ chế tạo mạch điện tử và đóng gói chip vật lý.

2. Quá trình dịch một chương trình

Máy tính không hiểu trực tiếp ngôn ngữ bậc cao như C/C++. Chương trình phải được dịch qua 3 mức:

- **Bước 1 – Biên dịch (Compiler):** Dịch từ **Ngôn ngữ bậc cao (C)** sang **Hợp ngữ (Assembly)**.
- **Bước 2 – Hợp dịch (Assembler):** Dịch từ **Hợp ngữ (Assembly)** sang **Ngôn ngữ máy (Mã nhị phân 0 và 1)**.

1.1.4 Đánh giá hiệu năng máy tính (Trọng tâm tính toán)

Đây là phần chắc chắn xuất hiện từ 2 đến 4 câu trong cả bài thi Giữa kỳ và Cuối kỳ. Để không bao giờ làm sai, bạn cần nắm vững bảng đổi đơn vị và 5 công thức dưới đây.

1. Bảng quy đổi đơn vị bắt buộc phải thuộc

Trong các bài toán hiệu năng, thời gian thường cho bằng mili-giây (ms), nano-giây (ns), pico-giây (ps); còn tần số cho bằng MHz, GHz. Khi thay vào công thức, **cách an toàn nhất là đổi tất cả về đơn vị gốc: giây (s), Hertz (Hz), và số lệnh**.

Đại lượng	**Đơn vị trong đề bài**	**Cách đổi về đơn vị chuẩn**
Tần số (f_0)	1 MHz (Mega-Hertz)	$= 10^6$ Hz (1 triệu xung/giây)
	1 GHz (Giga-Hertz)	$= 10^9$ Hz (1 tỷ xung/giây)
Thời gian (T_0, t)	1 ms (mili-giây)	$= 10^{-3}$ s
	1 μs (micro-giây)	$= 10^{-6}$ s
	1 ns (nano-giây)	$= 10^{-9}$ s
	1 ps (pico-giây)	$= 10^{-12}$ s
Số lượng lệnh (IC)	1 triệu lệnh	$= 10^6$ lệnh
	1 tỷ lệnh (hoặc 1.0E9)	$= 10^9$ lệnh

Mẹo tính nhanh giữa GHz và ns: Nếu tần số tính bằng GHz (10^9 Hz) và chu kỳ tính bằng ns (10^{-9} s), hai số mũ 10^9 và 10^{-9} triệt tiêu nhau:

$$T_0 \text{ (ns)} = \frac{1}{f_0 \text{ (GHz)}}$$

Ví dụ: $f_0 = 4$ GHz $\Rightarrow T_0 = \frac{1}{4} = 0.25$ ns.

2. Năm công thức cốt lõi về Hiệu năng

Công thức 1: Chu kỳ xung nhịp (T_0) và Tần số xung nhịp (f_0)

$$T_0 = \frac{1}{f_0} \quad \text{và} \quad f_0 = \frac{1}{T_0}$$

- T_0 (Clock period): Thời gian của 1 chu kỳ xung nhịp (đơn vị: giây, ns, ps).
- f_0 (Clock rate / Clock speed): Tốc độ xung nhịp, tức số chu kỳ trong 1 giây (đơn vị: Hz, MHz, GHz).

Công thức 2: Thời gian thực thi của CPU (t_{CPU})

Tổng số chu kỳ xung nhịp (n) của chương trình bằng Số lệnh (IC) nhân với Số chu kỳ trung bình để thực hiện 1 lệnh (CPI):

$$n = IC \times CPI$$

Thời gian thực thi của CPU bằng tổng số chu kỳ nhân với thời gian 1 chu kỳ:

$$t_{\text{CPU}} = n \times T_0 = \frac{n}{f_0} = IC \times CPI \times T_0 = \frac{IC \times CPI}{f_0}$$

- t_{CPU}: Thời gian CPU hoàn thành chương trình (đơn vị: giây).
- n: Tổng số chu kỳ xung nhịp (Clock cycles).
- IC (Instruction Count): Tổng số lệnh được thực thi của chương trình.

- CPI (Cycles Per Instruction): Số chu kỳ xung nhịp trung bình trên một lệnh.

Công thức 3: Tính CPI trung bình khi chương trình có nhiều loại lệnh

- **Trường hợp 1:** Đề bài cho số lượng lệnh cụ thể $(IC_1, IC_2, \ldots, IC_k)$ của từng loại lệnh:
$$CPI_{\text{TB}} = \frac{n}{IC} = \frac{CPI_1 \times IC_1 + CPI_2 \times IC_2 + \cdots + CPI_k \times IC_k}{IC_1 + IC_2 + \cdots + IC_k}$$
- **Trường hợp 2:** Đề bài cho tỷ lệ phần trăm $(p_1\%, p_2\%, \ldots, p_k\%)$ của từng loại lệnh:
$$CPI_{\text{TB}} = (p_1\% \times CPI_1) + (p_2\% \times CPI_2) + \cdots + (p_k\% \times CPI_k)$$

Công thức 4: So sánh hiệu năng giữa hai máy tính

Hiệu năng (P) tỷ lệ nghịch với thời gian thực thi (t): máy nào chạy hết **ít thời gian hơn** thì máy đó **nhanh hơn** (hiệu năng cao hơn):

$$P = \frac{1}{t_{\text{CPU}}}$$

Khi đề bài hỏi: "Máy A nhanh hơn máy B bao nhiêu lần?" (hoặc "P2 nhanh hơn P1 bao nhiêu lần?"), ta lấy **thời gian của máy B chia cho thời gian của máy A**:

$$\frac{P_A}{P_B} = \frac{t_B}{t_A}$$

(Lưu ý: Hỏi máy A nhanh hơn máy B thì trên tử số phải là thời gian t_B, dưới mẫu số là t_A).

Công thức 5: Tốc độ thực hiện lệnh MIPS (Millions of Instructions Per Second)

Từ **MIPS** ở đây là đơn vị đo "Số triệu lệnh thực hiện được trong 1 giây" (không phải tên kiến trúc bộ xử lý MIPS):

$$\text{MIPS} = \frac{IC}{t_{\text{CPU}} \times 10^6} = \frac{f_0}{CPI \times 10^6}$$

Cách tính nhanh không sợ nhầm số mũ:

- Nếu f_0 cho theo đơn vị **MHz**: $\text{MIPS} = \dfrac{f_0 \text{ (MHz)}}{CPI}$.
- Nếu f_0 cho theo đơn vị **GHz**: $\text{MIPS} = \dfrac{f_0 \text{ (GHz)} \times 1000}{CPI}$.
- Công thức suy ngược ra CPI: $CPI = \dfrac{f_0 \text{ (GHz)} \times 1000}{\text{MIPS}}$.

1.2 Phần 2: Toàn bộ bài tập Chương 1 và Hướng dẫn giải

1.2.1 Nhóm 1: Các thành phần máy tính và Phân cấp bộ nhớ

Bài tập 1.1 (Đề Cuối kỳ 2020.2 & Ôn tập Cuối kỳ P2 - Câu 19)

Hãy chỉ ra tên của thành phần máy tính thực hiện chức năng: "Chứa các lệnh và dữ liệu của chương trình đang được thực hiện"?

A. Bộ nhớ chính

B. Bộ nhớ đệm nhanh cache

C. Tập thanh ghi

D. Bộ nhớ ngoài

Hướng dẫn giải:

- Theo nguyên lý hoạt động cơ bản của máy tính (mô hình von Neumann), mọi chương trình khi đang chạy (đang thực hiện) đều phải được nạp vào **Bộ nhớ chính** (Main Memory / RAM).
- Bộ nhớ ngoài (ổ cứng) chỉ lưu trữ phần mềm và dữ liệu khi chưa chạy. Cache và Tập thanh ghi chỉ lưu tạm một phần rất nhỏ dữ liệu/lệnh mà CPU đang dùng tức thời.
- **Đáp án đúng: A (Bộ nhớ chính).**

Bài tập 1.2 (Đề Cuối kỳ 2020.2)

Sắp xếp các thành phần của bộ nhớ máy tính theo thứ tự dung lượng tăng dần:

A. Cache L3, Cache L2, Cache L1, Bộ nhớ chính, Bộ nhớ ngoài

B. Cache L1, Cache L2, Cache L3, Bộ nhớ chính, Bộ nhớ ngoài

C. Bộ nhớ chính, Cache L3, Cache L2, Cache L1, Bộ nhớ ngoài

D. Cache L1, Cache L2, Bộ nhớ chính, Cache L3, Bộ nhớ ngoài

Hướng dẫn giải:

- Trong phân cấp bộ nhớ, Cache L1 nằm sát nhân CPU nhất nên có tốc độ nhanh nhất nhưng dung lượng nhỏ nhất (vài chục KB), tiếp theo là Cache L2 (vài trăm KB), rồi đến Cache L3 (vài MB đến vài chục MB).
- Lớn hơn Cache là Bộ nhớ chính (RAM - vài GB đến vài chục GB), và lớn nhất là Bộ nhớ ngoài (SSD/HDD - hàng trăm GB đến hàng TB).
- **Đáp án đúng: B (Cache L1, Cache L2, Cache L3, Bộ nhớ chính, Bộ nhớ ngoài).**

Bài tập 1.3 (Đề Cuối kỳ 2022.2 - #ef5037 & Đề Cuối kỳ 2024.1 - #018513)

Hãy chỉ ra đâu là cấu hình bộ nhớ phù hợp cho máy tính PC?

A. Cache L1: 64KB, Cache L2: 256KB, Cache L3: 8MB, RAM: 8GB, SSD: 512GB

B. Cache L1: 64KB, Cache L2: 256KB, Cache L3: 16GB, RAM: 2TB, SSD: 1TB

C. Cache L1: 64KB, Cache L2: 256GB, Cache L3: 8MB, RAM: 8GB, SSD: 512GB

D. Cache L1: 16MB, Cache L2: 256KB, Cache L3: 16MB, RAM: 8GB, SSD: 1TB

Hướng dẫn giải:

Ta kiểm tra từng phương án dựa trên nguyên tắc dung lượng L1 < L2 < L3 < RAM < SSD:

- Phương án B sai vì Cache L3 lên tới 16GB (quá lớn, ngang RAM) và RAM 2TB lớn hơn cả ổ cứng SSD 1TB.
- Phương án C sai vì Cache L2 là 256GB (lớn hơn cả Cache L3 8MB và RAM 8GB).
- Phương án D sai vì Cache L1 là 16MB (lớn hơn cả Cache L2 256KB).
- Phương án A hoàn toàn hợp lý ở mọi thông số: 64KB < 256KB < 8MB < 8GB < 512GB.
- **Đáp án đúng: A.**

Bài tập 1.4 (Ôn tập Giữa kỳ - Câu 6)

Hãy chỉ ra đâu là cấu hình phù hợp cho máy tính PC?

A. RAM: 2TB, Cache L1: 64KB, Cache L2: 256KB, Cache L3: 16GB, SSD: 1TB

B. RAM: 4GB, Cache L1: 64KB, Cache L2: 256KB, Cache L3: 8MB, SSD: 512GB

C. RAM: 8GB, Cache L1: 16MB, Cache L2: 256KB, Cache L3: 16MB, SSD: 1TB

D. RAM: 8GB, Cache L1: 64KB, Cache L2: 256GB, Cache L3: 8MB, SSD: 512GB

Hướng dẫn giải:

- Cẩn thận bẫy thứ tự liệt kê: ở câu này đề bài đưa thông số RAM lên đầu dòng để đánh lạc hướng.
- Loại A vì Cache L3 = 16GB và RAM = 2TB. Loại C vì Cache L1 = 16MB > Cache L2 = 256KB. Loại D vì Cache L2 = 256GB.
- **Đáp án đúng: B (RAM: 4GB, Cache L1: 64KB, Cache L2: 256KB, Cache L3: 8MB, SSD: 512GB).**

Bài tập 1.5 (Đề Giữa kỳ 2022.2 - Đề 0008, Câu 11 & Ôn tập Giữa kỳ - Câu 69)

Tác dụng của bộ nhớ đệm (Cache) là gì? (Hoặc: Loại bộ nhớ giúp CPU tăng tốc truy cập bộ nhớ chính có tên là gì?)

A. Tăng tốc độ truy cập đến ổ đĩa cứng

B. Tăng dung lượng bộ nhớ chính

C. Tăng tốc độ truy cập đến bộ nhớ chính

D. Giảm chi phí cho hệ thống máy tính

Hướng dẫn giải:

- Bộ nhớ Cache được đặt đệm giữa CPU và Bộ nhớ chính (RAM) nhằm mục đích **tăng tốc độ CPU truy cập bộ nhớ chính**.
- Nếu điền khuyết tự luận, từ cần điền là: **Bộ nhớ đệm** hoặc **Cache**.
- **Đáp án đúng: C.**

1.2.2 Nhóm 2: Tính chu kỳ xung nhịp (T_0), CPI và thời gian thực thi (t_{CPU})

Bài tập 1.6 (Slide Chương 1 - Trang 24)

Một bộ xử lý có tần số xung nhịp $f_0 = 4$ GHz. Hãy tính chu kỳ xung nhịp T_0 của bộ xử lý đó theo đơn vị nano-giây (ns).

Hướng dẫn giải:

- Đổi đơn vị: $f_0 = 4$ GHz $= 4 \times 10^9$ Hz.
- Áp dụng công thức chu kỳ xung nhịp:

$$T_0 = \frac{1}{f_0} = \frac{1}{4 \times 10^9} = 0.25 \times 10^{-9} \text{ s} = 0.25 \text{ ns}$$

- **Đáp số:** 0.25 **ns.**

Bài tập 1.7 (Đề Cuối kỳ 2020.2 & Ôn tập Cuối kỳ P2 - Câu 59)

Bộ xử lý có tốc độ xung nhịp là 7.9 GHz. Cho biết chu kỳ xung nhịp tương ứng là bao nhiêu?

A. 0.13 ms

B. 7.9 ns

C. 0.13 ps

D. 0.13 ns

Hướng dẫn giải:

- Ta có tần số $f_0 = 7.9$ GHz $= 7.9 \times 10^9$ Hz.
- Chu kỳ xung nhịp:

$$T_0 = \frac{1}{f_0} = \frac{1}{7.9 \times 10^9} \text{ s} \approx 0.12658 \times 10^{-9} \text{ s} \approx 0.13 \times 10^{-9} \text{ s}$$

- Vì 10^{-9} s $= 1$ ns, nên $T_0 \approx 0.13$ ns.
- **Đáp án đúng: D** (0.13 **ns**).

Bài tập 1.8 (Đề Giữa kỳ 2023.2 - Câu 1)

Máy tính có bộ xử lý với tốc độ xung nhịp là $f = 4$ GHz. Giả sử có chương trình được thực hiện trên máy tính đó với số lệnh $n = 5$ triệu lệnh và CPI trung bình bằng 2. Hãy cho biết thời gian của CPU (t_{CPU}) để hoàn thành chương trình đó (tính theo ms)?

Hướng dẫn giải:

- Bước 1 – Đổi tất cả dữ kiện về đơn vị chuẩn:
  - Số lệnh: $IC = 5$ triệu lệnh $= 5 \times 10^6$ lệnh.
  - $CPI = 2$.
  - Tần số xung nhịp: $f_0 = 4$ GHz $= 4 \times 10^9$ Hz.
- Bước 2 – Áp dụng công thức tính thời gian thực thi của CPU:

$$t_{\text{CPU}} = \frac{IC \times CPI}{f_0} = \frac{5 \times 10^6 \times 2}{4 \times 10^9} = \frac{10 \times 10^6}{4 \times 10^9} = 2.5 \times 10^{-3} \text{ s}$$

- Bước 3 – Đổi sang mili-giây (ms): Vì 10^{-3} s $= 1$ ms, nên $t_{\text{CPU}} = 2.5$ ms.
- **Đáp số:** 2.5 **ms.**

Bài tập 1.9 (Đề Cuối kỳ 2024.1 - #54583d & Ôn tập Giữa kỳ - Câu 51)

Máy tính có bộ xử lý với tần số xung nhịp 5 GHz và CPI trung bình bằng 2. Giả sử có chương trình được thực hiện trên máy tính đó với 6 triệu lệnh. Hãy cho biết thời gian của bộ xử lý để hoàn thành chương trình đó?

A. 24 ms

B. 60 ms

C. 2.4 ms

D. 240 ms

Hướng dẫn giải:

- Đổi đơn vị: $IC = 6 \times 10^6$ lệnh; $CPI = 2$; $f_0 = 5$ GHz $= 5 \times 10^9$ Hz.
- Thời gian thực thi:

$$t_{\text{CPU}} = \frac{IC \times CPI}{f_0} = \frac{6 \times 10^6 \times 2}{5 \times 10^9} = \frac{12}{5} \times 10^{-3} \text{ s} = 2.4 \times 10^{-3} \text{ s} = 2.4 \text{ ms}$$

- **Đáp án đúng: C** (2.4 **ms**).

Bài tập 1.10 (Đề Cuối kỳ 2020.2)

Máy tính có bộ xử lý với tốc độ xung nhịp là 1.8 GHz và số chu kỳ lệnh trung bình (CPI) là 2.1. Giả sử có chương trình với 3 triệu lệnh được thực hiện trên máy tính này. Cho biết thời gian của bộ xử lý để hoàn thành chương trình đó?

A. 3.5 ms

B. 11.3 ms

C. 35.0 ms

D. 3500.0 ms

Hướng dẫn giải:

- Đổi đơn vị: $IC = 3 \times 10^6$ lệnh; $CPI = 2.1$; $f_0 = 1.8$ GHz $= 1.8 \times 10^9$ Hz.
- Thời gian thực thi:

$$t_{\text{CPU}} = \frac{IC \times CPI}{f_0} = \frac{3 \times 10^6 \times 2.1}{1.8 \times 10^9} = \frac{6.3}{1.8} \times 10^{-3}\ \text{s} = 3.5 \times 10^{-3}\ \text{s} = 3.5\ \text{ms}$$

- **Đáp án đúng: A** (3.5 **ms**).

Bài tập 1.11 (Đề Cuối kỳ 2022.2 - Trang 12)

Một chương trình được thực hiện trên máy tính với 6 triệu lệnh. Biết bộ xử lý có tần số xung nhịp $f_0 = 2$ GHz và $CPI = 1.5$ (hoặc $f_0 = 4$ GHz, $CPI = 3$). Hãy cho biết thời gian của bộ xử lý để hoàn thành chương trình đó?

A. 72 ms

B. 450 ms

C. 4.5 ms

D. 45 ms

Hướng dẫn giải:

- Với $IC = 6 \times 10^6$ lệnh, $f_0 = 2$ GHz $= 2 \times 10^9$ Hz và $CPI = 1.5$:

$$t_{\text{CPU}} = \frac{6 \times 10^6 \times 1.5}{2 \times 10^9} = \frac{9}{2} \times 10^{-3} \text{ s} = 4.5 \times 10^{-3} \text{ s} = 4.5 \text{ ms}$$

- **Đáp án đúng: C** (4.5 **ms**).

Bài tập 1.12 (Đề Giữa kỳ 2022.2 - Đề 0008, Câu 1)

Bộ xử lý có tần số xung nhịp 50 MHz thực hiện phần mềm gồm 800 000 lệnh máy trong 0.04 giây. Hỏi phần mềm đó có CPI trung bình bằng bao nhiêu?

Hướng dẫn giải:

- Bước 1 – Đổi đơn vị về chuẩn:
  - $f_0 = 50$ MHz $= 50 \times 10^6$ Hz.
  - $IC = 800\,000 = 8 \times 10^5$ lệnh.
  - $t_{\text{CPU}} = 0.04$ s.
- Bước 2 – Từ công thức $t_{\text{CPU}} = \frac{IC \times CPI}{f_0}$, ta rút ra công thức tính CPI:

$$CPI = \frac{t_{\text{CPU}} \times f_0}{IC} = \frac{0.04 \times 50 \times 10^6}{800\,000} = \frac{2\,000\,000}{800\,000} = 2.5$$

- **Đáp số:** $CPI = 2.5$**.**

1.2.3 Nhóm 3: Bài tập về chỉ số tốc độ MIPS

Bài tập 1.13 (Slide Chương 1 - Trang 35)

Tính chỉ số MIPS của bộ xử lý với tần số xung nhịp $f_0 = 2$ GHz và $CPI = 4$.

Hướng dẫn giải:

- Áp dụng công thức tính nhanh khi f_0 tính bằng GHz:

$$\text{MIPS} = \frac{f_0 \text{ (GHz)} \times 1000}{CPI} = \frac{2 \times 1000}{4} = 500 \text{ MIPS}$$

- **Đáp số:** 500 **MIPS.**

Bài tập 1.14 (Slide Chương 1 - Trang 36)

Tính CPI của bộ xử lý biết tần số xung nhịp $f_0 = 1$ GHz và tốc độ thực hiện đạt 400 MIPS.

Hướng dẫn giải:

- Áp dụng công thức suy ra CPI từ MIPS:

$$CPI = \frac{f_0 \text{ (GHz)} \times 1000}{\text{MIPS}} = \frac{1 \times 1000}{400} = 2.5$$

- **Đáp số:** $CPI = 2.5$**.**

Bài tập 1.15 (Đề Cuối kỳ 2022.2 - #1fd28d & Ôn tập Giữa kỳ - Câu 17)

Bộ xử lý có tốc độ xung nhịp 2.5 GHz và CPI trung bình là 2. Hãy cho biết số triệu lệnh được thực hiện trên một giây?

A. 1250 MIPS

B. 1.25 MIPS

C. 5 MIPS

D. 500 MIPS

Hướng dẫn giải:

- "Số triệu lệnh được thực hiện trên một giây" chính là định nghĩa của chỉ số MIPS.
- Thay số vào công thức:

$$\text{MIPS} = \frac{f_0}{CPI \times 10^6} = \frac{2.5 \times 10^9}{2 \times 10^6} = \frac{2500}{2} = 1250 \text{ MIPS}$$

- **Đáp án đúng: A** (1250 **MIPS).**

Bài tập 1.16 (Đề Cuối kỳ 2024.1 - #3e5442 & Ôn tập Giữa kỳ - Câu 36)

Bộ xử lý có tần số xung nhịp 3 GHz và CPI trung bình là 1.5. Hãy cho biết số triệu lệnh được thực hiện trên một giây?

A. 4.5 MIPS

B. 500 MIPS

C. 2 MIPS

D. 2000 MIPS

Hướng dẫn giải:

- Áp dụng công thức:

$$\text{MIPS} = \frac{3 \times 10^9}{1.5 \times 10^6} = \frac{3000}{1.5} = 2000 \text{ MIPS}$$

- **Đáp án đúng: D** (2000 **MIPS**).

Bài tập 1.17 (Đề Cuối kỳ 2020.2)

Máy tính có CPU với tốc độ xung nhịp 1.2 GHz và CPI bằng 1.5. Xác định số lệnh thực hiện trên 1 giây?

A. 1 MIPS

B. 1800 MIPS

C. 800 MIPS

D. 1250 MIPS

Hướng dẫn giải:

- Áp dụng công thức:

$$\text{MIPS} = \frac{1.2 \times 10^9}{1.5 \times 10^6} = \frac{1200}{1.5} = 800 \text{ MIPS}$$

- **Đáp án đúng: C** (800 **MIPS**).

Bài tập 1.18 (Đề Cuối kỳ 2021.2 - Mã đề B1, Câu 1.1)

Máy tính A có bộ xử lý với tần số xung nhịp 1.5 GHz, có khả năng thực hiện 600 triệu lệnh trên một giây. Tính CPI của nó?

Hướng dẫn giải:

- Đề bài cho thực hiện "600 triệu lệnh trên một giây", nghĩa là MIPS = 600.
- Ta có tần số $f_0 = 1.5$ GHz $= 1500$ MHz.
- Áp dụng công thức:

$$CPI = \frac{f_0}{\text{MIPS} \times 10^6} = \frac{1.5 \times 10^9}{600 \times 10^6} = \frac{1500}{600} = 2.5$$

- **Đáp số:** $CPI = 2.5$**.**

Bài tập 1.19 (Đề Cuối kỳ 2021.2 - Mã đề B5, Câu 1.2)

Cho máy tính A với bộ xử lý có MIPS = 900. Hãy tính CPI của máy tính đó nếu biết chu kỳ xung nhịp của bộ xử lý là $T_A = 300$ ps.

Hướng dẫn giải:

- Bước 1 – Đổi chu kỳ T_A ra giây và tìm tần số f_A:

$$T_A = 300 \text{ ps} = 300 \times 10^{-12} \text{ s} \implies f_A = \frac{1}{T_A} = \frac{1}{300 \times 10^{-12}} = \frac{10^{12}}{300} \text{ Hz}$$

- Bước 2 – Thay vào công thức tính CPI:

$$CPI = \frac{f_A}{\text{MIPS} \times 10^6} = \frac{10^{12}}{300 \times 900 \times 10^6} = \frac{10^6}{270\,000} = \frac{100}{27} \approx 3.704$$

- **Đáp số:** $CPI = \dfrac{100}{27} \approx 3.7$**.**

1.2.4 Nhóm 4: Tính CPI trung bình và thời gian với nhiều lớp lệnh

Bài tập 1.20 (Slide Chương 1 - Trang 32)

Cho bảng chỉ ra các dãy lệnh sử dụng các lệnh thuộc các loại A, B, C. Hãy tính CPI trung bình của Dãy lệnh 1 và Dãy lệnh 2:

Loại lệnh	**A**	**B**	**C**
CPI theo loại lệnh	1	2	3
Số lệnh (IC) trong dãy lệnh 1	20	10	20
Số lệnh (IC) trong dãy lệnh 2	40	10	10

Hướng dẫn giải:

- **Xét Dãy lệnh 1:**
  - Tổng số lệnh: $IC_1 = 20 + 10 + 20 = 50$ lệnh.
  - Tổng số chu kỳ: $n_1 = (1 \times 20) + (2 \times 10) + (3 \times 20) = 20 + 20 + 60 = 100$ chu kỳ.
  - $CPI_{\text{TB1}} = \dfrac{n_1}{IC_1} = \dfrac{100}{50} = 2.0$.
- **Xét Dãy lệnh 2:**
  - Tổng số lệnh: $IC_2 = 40 + 10 + 10 = 60$ lệnh.
  - Tổng số chu kỳ: $n_2 = (1 \times 40) + (2 \times 10) + (3 \times 10) = 40 + 20 + 30 = 90$ chu kỳ.
  - $CPI_{\text{TB2}} = \dfrac{n_2}{IC_2} = \dfrac{90}{60} = 1.5$.

Bài tập 1.21 (Đề Cuối kỳ 2021.2 - Mã đề B4, Câu 1)

Trong tập lệnh của máy tính có 3 lớp lệnh A, B, C. CPI và số lệnh (IC) theo mỗi lớp lệnh được cho trong bảng sau:

Loại lệnh	**A**	**B**	**C**
CPI theo loại lệnh	1	2	4
IC trong dãy lệnh 1	1	2	2
IC trong dãy lệnh 2	5	2	1
IC trong dãy lệnh 3	4	1	1

Tính CPI trung bình ứng với mỗi dãy lệnh?

Hướng dẫn giải:

Áp dụng công thức $CPI_{\text{TB}} = \dfrac{\sum(CPI_i \times IC_i)}{\sum IC_i}$:

- **Dãy lệnh 1:**

$$CPI_{\text{TB1}} = \frac{(1 \times 1) + (2 \times 2) + (4 \times 2)}{1 + 2 + 2} = \frac{1 + 4 + 8}{5} = \frac{13}{5} = 2.6$$

- **Dãy lệnh 2:**

$$CPI_{\text{TB2}} = \frac{(1 \times 5) + (2 \times 2) + (4 \times 1)}{5 + 2 + 1} = \frac{5 + 4 + 4}{8} = \frac{13}{8} = 1.625$$

- **Dãy lệnh 3:**

$$CPI_{\text{TB3}} = \frac{(1 \times 4) + (2 \times 1) + (4 \times 1)}{4 + 1 + 1} = \frac{4 + 2 + 4}{6} = \frac{10}{6} \approx 1.67$$

Bài tập 1.22 (Đề Giữa kỳ 2024.1 - #a6b720 & Ôn tập Giữa kỳ - Câu 4)

Một bộ xử lý chạy một chương trình với tần số xung nhịp 4 GHz. Chương trình gồm ba loại lệnh với các đặc điểm sau:

- Loại A: chiếm 40% tổng số lệnh với CPI là 1
- Loại B: chiếm 35% tổng số lệnh với CPI là 3
- Loại C: chiếm 25% tổng số lệnh với CPI là 5

Tổng số lệnh cho chương trình là 2 tỷ lệnh. Tổng thời gian thực thi là bao nhiêu?

A. 1.1 giây

B. 1.25 giây

C. 1.35 giây

D. 1.4 giây

Hướng dẫn giải:

- **Bước 1 – Tính CPI trung bình của chương trình:**

$$CPI_{\text{TB}} = (0.40 \times 1) + (0.35 \times 3) + (0.25 \times 5) = 0.4 + 1.05 + 1.25 = 2.7$$

- **Bước 2 – Tính tổng thời gian thực thi t_{CPU}:** Biết $IC = 2$ tỷ lệnh $= 2 \times 10^9$ lệnh và $f_0 = 4$ GHz $= 4 \times 10^9$ Hz:

$$t_{\text{CPU}} = \frac{IC \times CPI_{\text{TB}}}{f_0} = \frac{2 \times 10^9 \times 2.7}{4 \times 10^9} = \frac{5.4}{4} = 1.35 \text{ giây}$$

- **Đáp án đúng: C (1.35 giây).**

Bài tập 1.23 (Ôn tập Giữa kỳ - Câu 68 & Câu 37)

Cho một bộ xử lý 2 GHz thực thi tập lệnh gồm năm lớp lệnh A, B, C, D, E với CPI lần lượt là $2, 3, 4, 6$ và 10. Máy tính đang thực hiện một chương trình trong đó các lớp lệnh A, B, C, D, E chiếm lần lượt $30\%, 30\%, 15\%, 15\%$ và 10% tổng số lệnh.

a) (Câu 68) Tìm CPI trung bình của bộ xử lý này.

b) (Câu 37) Nếu thời gian thực thi của chương trình này là 20 giây, hãy tìm tổng số lệnh (IC). Đáp án viết không chứa dấu phân tách chữ số.

Hướng dẫn giải:

- **Ý a) Tính CPI trung bình:**

$$CPI_{\text{TB}} = (0.30 \times 2) + (0.30 \times 3) + (0.15 \times 4) + (0.15 \times 6) + (0.10 \times 10)$$

$$CPI_{\text{TB}} = 0.6 + 0.9 + 0.6 + 0.9 + 1.0 = 4.0 \text{ (hoặc điền 4)}$$

- **Ý b) Tìm tổng số lệnh IC khi $t_{\text{CPU}} = 20$ giây:** Ta có $f_0 = 2$ GHz $= 2 \times 10^9$ Hz. Từ công thức $t_{\text{CPU}} = \dfrac{IC \times CPI_{\text{TB}}}{f_0}$, ta rút ra:

$$IC = \frac{t_{\text{CPU}} \times f_0}{CPI_{\text{TB}}} = \frac{20 \times 2 \times 10^9}{4} = 10 \times 10^9 = 10\,000\,000\,000$$

- **Đáp số:** Ý a điền **4**; Ý b điền **10000000000** (số 1 và 10 chữ số 0).

1.2.5 Nhóm 5: So sánh hiệu năng và thời gian giữa các bộ xử lý

Bài tập 1.24 (Slide Chương 1 - Trang 29 & Đề Cuối kỳ 2021.2 - Mã đề B1, Câu 1.2)

Hai máy tính A và B có cùng kiến trúc tập lệnh (ISA).

- Máy tính A có chu kỳ xung nhịp $T_A = 250$ ps, CPI trung bình là 2.0.
- Máy tính B có chu kỳ xung nhịp $T_B = 750$ ps (trong Slide là 500 ps), CPI trung bình là 1.2.

Hỏi máy tính nào nhanh hơn và nhanh hơn bao nhiêu lần?

Hướng dẫn giải:

- Vì hai máy có cùng kiến trúc tập lệnh (ISA) và chạy cùng một chương trình nên tổng số lệnh thực thi là bằng nhau: $IC_A = IC_B = IC$.
- **Trường hợp 1 (Theo Đề thi 2021.2 - Mã đề B1 với $T_B = 750$ ps):**
  - Thời gian chạy trên máy A: $t_A = IC \times CPI_A \times T_A = IC \times 2.0 \times 250$ ps $= 500 \times IC$ (ps).
  - Thời gian chạy trên máy B: $t_B = IC \times CPI_B \times T_B = IC \times 1.2 \times 750$ ps $= 900 \times IC$ (ps).
  - Vì $t_A < t_B$ ($500 < 900$) nên **máy A nhanh hơn máy B**.
  - Số lần nhanh hơn: $\dfrac{P_A}{P_B} = \dfrac{t_B}{t_A} = \dfrac{900 \times IC}{500 \times IC} = 1.8$ lần.
- **Trường hợp 2 (Theo Slide bài giảng với $T_B = 500$ ps):**
  - $t_A = 500 \times IC$ (ps); $t_B = IC \times 1.2 \times 500$ ps $= 600 \times IC$ (ps).
  - Máy A nhanh hơn máy B số lần là: $\dfrac{t_B}{t_A} = \dfrac{600}{500} = 1.2$ lần.

Bài tập 1.25 (Đề Cuối kỳ 2023.1 - #2262f & Ôn tập Giữa kỳ - Câu 3)

Xét hai bộ xử lý sau đây làm cùng một công việc:

- P1 có tần số xung nhịp 4 GHz, CPI trung bình 0.9 và yêu cầu thực hiện 5.0E9 lệnh cho công việc đó.
- P2 có tần số xung nhịp 3 GHz, CPI trung bình 0.75 và yêu cầu thực hiện 1.0E9 lệnh cho công việc đó.

Hỏi P2 nhanh hơn P1 bao nhiêu lần khi làm công việc này? (Lưu ý: $x\text{E}y = x \times 10^y$).

Hướng dẫn giải:

- **Bước 1 – Tính thời gian thực thi của P1 (t_1):**

$$t_1 = \frac{IC_1 \times CPI_1}{f_1} = \frac{5.0 \times 10^9 \times 0.9}{4 \times 10^9} = \frac{4.5}{4} = 1.125 \text{ giây}$$

- **Bước 2 – Tính thời gian thực thi của P2 (t_2):**

$$t_2 = \frac{IC_2 \times CPI_2}{f_2} = \frac{1.0 \times 10^9 \times 0.75}{3 \times 10^9} = \frac{0.75}{3} = 0.25 \text{ giây}$$

- **Bước 3 – Tính độ tăng tốc của P2 so với P1:** Đề hỏi "P2 nhanh hơn P1 bao nhiêu lần", ta lấy thời gian của P1 chia cho thời gian của P2:

$$\frac{P_2}{P_1} = \frac{t_1}{t_2} = \frac{1.125}{0.25} = 4.5 \text{ lần}$$

- **Đáp số: 4.5**

Bài tập 1.26 (Đề Giữa kỳ 2024.1 - #8cf496)

Xét hai bộ xử lý sau đây làm cùng một công việc:

- P1 có tần số xung nhịp 1.5 GHz, CPI trung bình là 1.8, và cần thực hiện 3.0E9 lệnh.
- P2 có tần số xung nhịp 2 GHz, CPI trung bình là 1.2, và cần thực hiện 4.0E9 lệnh.

Hỏi P2 nhanh hơn P1 bao nhiêu lần khi thực hiện công việc này?

Hướng dẫn giải:

- **Bước 1 – Tính thời gian thực thi của P1 (t_1):**

$$t_1 = \frac{IC_1 \times CPI_1}{f_1} = \frac{3.0 \times 10^9 \times 1.8}{1.5 \times 10^9} = \frac{5.4}{1.5} = 3.6 \text{ giây}$$

- **Bước 2 – Tính thời gian thực thi của P2 (t_2):**

$$t_2 = \frac{IC_2 \times CPI_2}{f_2} = \frac{4.0 \times 10^9 \times 1.2}{2 \times 10^9} = \frac{4.8}{2} = 2.4 \text{ giây}$$

- **Bước 3 – So sánh P2 với P1:**

$$\frac{P_2}{P_1} = \frac{t_1}{t_2} = \frac{3.6}{2.4} = 1.5 \text{ lần}$$

- **Đáp số: 1.5**

Bài tập 1.27 (Slide Chương 1 - Trang 26)

Hai máy tính A và B cùng chạy một chương trình:

- Máy tính A có tần số xung nhịp $f_A = 2$ GHz, thời gian thực hiện chương trình $t_A = 10$ s.

- Máy tính B cần đạt thời gian thực hiện chương trình $t_B = 6$ s. Biết số chu kỳ xung nhịp khi chạy chương trình trên máy B (n_B) bằng 1.2 lần số chu kỳ xung nhịp khi chạy trên máy A (n_A).

Hãy xác định tần số xung nhịp cần thiết cho máy B (f_B)?

Hướng dẫn giải:

- Bước 1 – Tính tổng số chu kỳ xung nhịp của máy A:

$$n_A = t_A \times f_A = 10 \text{ s} \times 2 \times 10^9 \text{ Hz} = 20 \times 10^9 \text{ chu kỳ}$$

- Bước 2 – Tính tổng số chu kỳ xung nhịp của máy B:

$$n_B = 1.2 \times n_A = 1.2 \times 20 \times 10^9 = 24 \times 10^9 \text{ chu kỳ}$$

- Bước 3 – Tính tần số xung nhịp f_B để máy B chạy xong trong $t_B = 6$ s:

$$f_B = \frac{n_B}{t_B} = \frac{24 \times 10^9}{6} = 4 \times 10^9 \text{ Hz} = 4 \text{ GHz}$$

- **Đáp số:** $f_B = 4$ **GHz.**

Bài tập 1.28 (Đề Cuối kỳ 2021.2 - Mã đề B2, Câu 1)

Máy tính A có CPI trung bình là 1.3 và tần số xung nhịp của bộ xử lý là 600 MHz. Máy tính B có CPI trung bình là 2.5 và tần số xung nhịp của bộ xử lý là 750 MHz. Giả sử có một chương trình chạy được trên cả hai máy tính đó. Khi dịch trên máy A chương trình này được dịch thành 100 000 lệnh. Hãy xác định khi được dịch trên máy B thì chương trình đó được dịch thành bao nhiêu lệnh để cả hai máy có cùng thời gian thực hiện của CPU ($t_A = t_B$)?

Hướng dẫn giải:

- Vì cả hai máy đều cho tần số theo đơn vị MHz (10^6 Hz), nên khi lập phương trình $t_A = t_B$, số mũ 10^6 ở hai vế sẽ tự triệt tiêu:

$$t_A = t_B \iff \frac{IC_A \times CPI_A}{f_A} = \frac{IC_B \times CPI_B}{f_B}$$

- Thay số vào phương trình:

$$\frac{100\,000 \times 1.3}{600} = \frac{IC_B \times 2.5}{750}$$

- Rút ra IC_B:

$$IC_B = \frac{100\,000 \times 1.3 \times 750}{600 \times 2.5} = \frac{97\,500\,000}{1500} = 65\,000 \text{ lệnh}$$

- **Đáp số:** $65\,000$ **lệnh.**

Bài tập 1.29 (Đề Cuối kỳ 2021.2 - Mã đề B3, Câu 1)

Máy tính A có CPI trung bình là 1.2 và tần số xung nhịp là 500 MHz. Máy tính B có CPI trung bình là 2.4 và tần số xung nhịp là 600 MHz. Khi dịch trên máy A chương trình được dịch thành 90 000 lệnh. Hãy xác định khi dịch trên máy B thì chương trình đó được dịch thành bao nhiêu lệnh để cả hai máy có cùng thời gian thực hiện của CPU?

Hướng dẫn giải:

- Lập phương trình cân bằng thời gian $t_A = t_B$:

$$\frac{IC_A \times CPI_A}{f_A} = \frac{IC_B \times CPI_B}{f_B} \iff \frac{90\,000 \times 1.2}{500} = \frac{IC_B \times 2.4}{600}$$

- Giải tìm IC_B:

$$IC_B = \frac{90\,000 \times 1.2 \times 600}{500 \times 2.4} = \frac{64\,800\,000}{1200} = 54\,000 \text{ lệnh}$$

- **Đáp số:** $54\,000$ **lệnh.**

Bài tập 1.30 (Đề Cuối kỳ 2021.2 - Mã đề B6, Câu 1)

Máy tính A có CPI trung bình là 1.4 và tần số xung nhịp là 750 MHz. Máy tính B có CPI trung bình là 2.8 và tần số xung nhịp là 1.5 GHz. Khi dịch trên máy A chương trình được dịch thành 80 000 lệnh. Hãy xác định khi dịch trên máy B thì chương trình được dịch thành bao nhiêu lệnh để cả hai máy có cùng thời gian thực hiện của CPU?

Hướng dẫn giải:

- **Chú ý bẫy đơn vị:** Máy A cho $f_A = 750$ MHz, nhưng máy B cho $f_B = 1.5$ GHz. Ta phải đổi về cùng đơn vị MHz: $f_B = 1.5$ GHz $= 1500$ MHz.
- Lập phương trình $t_A = t_B$:

$$\frac{80\,000 \times 1.4}{750} = \frac{IC_B \times 2.8}{1500}$$

- Giải tìm IC_B:
$$IC_B = \frac{80\,000 \times 1.4 \times 1500}{750 \times 2.8} = 80\,000 \text{ lệnh}$$
- **Đáp số:** $80\,000$ **lệnh.**

Bài tập 1.31 (Đề Cuối kỳ 2021.2 - Mã đề B7, Câu 1)

Máy tính A có CPI trung bình là 2.4 và tần số xung nhịp là 800 MHz. Máy tính B có CPI trung bình là 3.6 và tần số xung nhịp là 1 GHz. Khi dịch trên máy A chương trình được dịch thành 70 000 lệnh. Hãy xác định khi dịch trên máy B thì chương trình được dịch thành bao nhiêu lệnh để cả hai máy có cùng thời gian thực hiện của CPU?

Hướng dẫn giải:

- Đổi về cùng đơn vị MHz: $f_A = 800$ MHz; $f_B = 1$ GHz $= 1000$ MHz.
- Lập phương trình $t_A = t_B$:
$$\frac{70\,000 \times 2.4}{800} = \frac{IC_B \times 3.6}{1000}$$
- Giải tìm IC_B:
$$IC_B = \frac{70\,000 \times 2.4 \times 1000}{800 \times 3.6} = \frac{168\,000\,000}{2880} = \frac{175\,000}{3} \approx 58\,333 \text{ lệnh}$$
- **Đáp số:** $\frac{175\,000}{3} \approx 58\,333$ **lệnh.**

Bài tập 1.32 (Đề Cuối kỳ 2021.2 - Mã đề B8, Câu 1)

Giả sử có 3 bộ xử lý P1, P2, P3 có cùng kiến trúc tập lệnh (ISA). Tần số xung nhịp và CPI của mỗi bộ xử lý được cho trong bảng sau:

Bộ xử lý	**Tần số xung nhịp (f_0)**	**CPI**
P1	2 GHz	1.5
P2	1.5 GHz	1.0
P3	3 GHz	2.5

1.1. Bộ xử lý nào có hiệu suất cao nhất?

1.2. Mỗi bộ xử lý thực thi một chương trình trong 10 s. Tính số chu kỳ và số lệnh của mỗi bộ xử lý.

1.3. Người ta muốn giảm thời gian thực thi đi 30%, nhưng điều này lại làm tăng CPI lên 20%. Tính tần số xung nhịp mới của mỗi bộ xử lý lúc này.

Hướng dẫn giải:

Giải 1.1 – Tìm bộ xử lý có hiệu suất cao nhất:

- Do cùng kiến trúc tập lệnh (ISA), ta tính tốc độ thực hiện lệnh (số tỷ lệnh thực hiện được trong 1 giây, tức $\dfrac{f_0}{CPI}$) của từng bộ xử lý:
  - Với P1: $\dfrac{f_1}{CPI_1} = \dfrac{2\text{ GHz}}{1.5} \approx 1.33 \times 10^9$ lệnh/giây (1333 MIPS).
  - Với P2: $\dfrac{f_2}{CPI_2} = \dfrac{1.5\text{ GHz}}{1.0} = 1.50 \times 10^9$ lệnh/giây (1500 MIPS).
  - Với P3: $\dfrac{f_3}{CPI_3} = \dfrac{3\text{ GHz}}{2.5} = 1.20 \times 10^9$ lệnh/giây (1200 MIPS).
- Vậy **bộ xử lý P2 có hiệu suất cao nhất** (thực hiện được nhiều lệnh nhất trong 1 giây).

Giải 1.2 – Tính số chu kỳ (n) và số lệnh (IC) trong $t = 10$ s:

- Công thức: Số chu kỳ $n = t \times f_0$; Số lệnh $IC = \dfrac{n}{CPI}$.
- **Bộ xử lý P1:**

$$n_1 = 10 \times 2 \times 10^9 = 20 \times 10^9 \text{ chu kỳ}; \quad IC_1 = \frac{20 \times 10^9}{1.5} \approx 13.33 \times 10^9 \text{ lệnh}$$

- **Bộ xử lý P2:**

$$n_2 = 10 \times 1.5 \times 10^9 = 15 \times 10^9 \text{ chu kỳ}; \quad IC_2 = \frac{15 \times 10^9}{1.0} = 15 \times 10^9 \text{ lệnh}$$

- **Bộ xử lý P3:**

$$n_3 = 10 \times 3 \times 10^9 = 30 \times 10^9 \text{ chu kỳ}; \quad IC_3 = \frac{30 \times 10^9}{2.5} = 12 \times 10^9 \text{ lệnh}$$

Giải 1.3 – Tính tần số xung nhịp mới ($f_{\text{mới}}$):

- Giảm thời gian đi 30% nghĩa là thời gian mới còn lại 70% so với ban đầu:

$$t_{\text{mới}} = (1 - 0.3) \times t_{\text{cũ}} = 0.7 \times 10 \text{ s} = 7 \text{ s}$$

- Tăng CPI lên 20% nghĩa là CPI mới bằng 120% so với ban đầu:

$$CPI_{\text{mới}} = (1 + 0.2) \times CPI_{\text{cũ}} = 1.2 \times CPI_{\text{cũ}}$$

- Vì số lệnh IC của chương trình không đổi, ta có công thức liên hệ giữa tần số mới và tần số cũ:

$$f_{\text{mới}} = \frac{IC \times CPI_{\text{mới}}}{t_{\text{mới}}} = \frac{IC \times 1.2 \times CPI_{\text{cũ}}}{0.7 \times t_{\text{cũ}}} = \frac{1.2}{0.7} \times f_{\text{cũ}} = \frac{12}{7} \times f_{\text{cũ}}$$

- Thay số cho từng bộ xử lý:
  - $f_{1\text{ mới}} = \frac{12}{7} \times 2\text{ GHz} = \frac{24}{7} \approx 3.43\text{ GHz}.$
  - $f_{2\text{ mới}} = \frac{12}{7} \times 1.5\text{ GHz} = \frac{18}{7} \approx 2.57\text{ GHz}.$
  - $f_{3\text{ mới}} = \frac{12}{7} \times 3\text{ GHz} = \frac{36}{7} \approx 5.14\text{ GHz}.$

1.2.6 Nhóm 6: CPI hiệu dụng và Phân loại kiến trúc máy tính

Lưu ý: Hai bài toán tính CPI hiệu dụng dưới đây kết hợp công thức CPI của Chương 1 với độ phạt bộ nhớ Cache của Chương 6, thường xuất hiện trong đề thi Cuối kỳ.

Bài tập 1.33 (Đề Cuối kỳ 2023.1 - #c7a320 & Ôn tập Cuối kỳ P2 - Câu 6)

Cho một CPU RISC-V có CPI cơ bản (base CPI) bằng 1. Bộ nhớ đệm (Cache) của nó có thời gian phạt khi bỏ lỡ (miss penalty) là 100 chu kỳ. Khi thực thi một chương trình, tỷ lệ bỏ lỡ lệnh (instruction miss rate) và tỷ lệ bỏ lỡ dữ liệu (data miss rate) lần lượt là 1% và 10%. Các lệnh truy cập bộ nhớ (load/store) chiếm 5% tổng số lệnh trong chương trình này. CPI hiệu dụng (effective CPI) khi thực thi chương trình này là bao nhiêu?

Hướng dẫn giải:

- **Hiểu bản chất:** Mọi lệnh (100% số lệnh) đều phải được đọc từ bộ nhớ lệnh (Instruction Cache), nhưng chỉ có các lệnh Load/Store (5% số lệnh) mới truy cập bộ nhớ dữ liệu (Data Cache).
- **Công thức tính CPI hiệu dụng:**

$$CPI_{\text{hiệu dụng}} = CPI_{\text{cơ bản}} + \text{Chu kỳ phạt trượt lệnh} + \text{Chu kỳ phạt trượt dữ liệu}$$

- Trong đó:

  - Số chu kỳ phạt do trượt lệnh (trên 1 lệnh) = $100\% \times 1\% \times 100 = 0.01 \times 100 = 1.0$ chu kỳ.
  - Số chu kỳ phạt do trượt dữ liệu (trên 1 lệnh) = $5\% \times 10\% \times 100 = 0.05 \times 0.10 \times 100 = 0.5$ chu kỳ.

- Cộng lại ta được:

$$CPI_{\text{hiệu dụng}} = 1 + 1.0 + 0.5 = 2.5$$

- **Đáp số: 2.5**

Bài tập 1.34 (Ôn tập Cuối kỳ P2 - Câu 58)

Cho biết:

- Tỷ lệ trượt Cache lệnh (I-cache miss rate) = 2%
- Tỷ lệ trượt Cache dữ liệu (D-cache miss rate) = 4%
- Thời gian phạt khi trượt (Miss penalty) = 100 chu kỳ
- CPI cơ bản khi Cache lý tưởng (Base CPI) = 2
- Các lệnh Load và Store chiếm 36% tổng số lệnh.

Hãy tính tỷ số: Actual CPI/Base CPI =?

Hướng dẫn giải:

- **Bước 1 – Tính số chu kỳ phạt trên mỗi lệnh:**
  - Phạt do trượt lệnh: $1 \times 2\% \times 100 = 0.02 \times 100 = 2.0$ chu kỳ.
  - Phạt do trượt dữ liệu: $36\% \times 4\% \times 100 = 0.36 \times 0.04 \times 100 = 1.44$ chu kỳ.
- **Bước 2 – Tính CPI thực tế (Actual CPI):**

$$\text{Actual CPI} = \text{Base CPI} + 2.0 + 1.44 = 2 + 3.44 = 5.44$$

- **Bước 3 – Tính tỷ số theo yêu cầu đề bài:**

$$\frac{\text{Actual CPI}}{\text{Base CPI}} = \frac{5.44}{2} = 2.72$$

- **Đáp số: 2.72**

Bài tập 1.35 (Đề Cuối kỳ 2020.2, 2022.2 - #c0ca61 & Ôn tập Cuối kỳ P2 - Câu 2, Câu 35)

a) Máy tính với 1 bộ xử lý đa lõi (multicore processor) thuộc kiến trúc nào?

- A. SISD B. SIMD C. MIMD với bộ nhớ phân tán D. MIMD với bộ nhớ dùng chung

b) Kiến trúc nào dưới đây chưa tồn tại máy tính thực tế?

- A. SIMD B. SISD C. MIMD D. MISD

Hướng dẫn giải:

- Theo phân loại kiến trúc của Flynn:
  - **SISD** (Single Instruction, Single Data): Máy tính đơn lõi cổ điển (1 dòng lệnh, 1 dòng dữ liệu).
  - **SIMD** (Single Instruction, Multiple Data): Xử lý vector, GPU (1 lệnh điều khiển nhiều dữ liệu cùng lúc).
  - **MISD** (Multiple Instruction, Single Data): Nhiều dòng lệnh tác động lên 1 dòng dữ liệu → **Chưa tồn tại máy tính thương mại thực tế**.
  - **MIMD** (Multiple Instruction, Multiple Data): Hệ thống đa xử lý. Một bộ xử lý đa lõi (multicore) có nhiều nhân CPU cùng nằm trên 1 chip và cùng truy cập chung một bộ nhớ chính (RAM), nên thuộc loại **MIMD với bộ nhớ dùng chung**.
- **Đáp án đúng:** Ý a chọn **D (MIMD với bộ nhớ dùng chung)**; Ý b chọn **D (MISD)**.

CHƯƠNG 2

HỆ THỐNG MÁY TÍNH

2.1 Phần 1: Lý thuyết trọng tâm

2.1.1 Chi tiết các thành phần cơ bản của máy tính

Mọi hệ thống máy tính đều hoạt động dựa trên sự phối hợp của 3 mô-đun phần cứng chính (CPU, Bộ nhớ, Mô-đun vào-ra) được kết nối với nhau qua Liên kết hệ thống (Bus).

1. Bộ xử lý trung tâm (CPU - Central Processing Unit)

CPU điều khiển toàn bộ hoạt động của máy tính theo chương trình nằm trong bộ nhớ chính và trực tiếp xử lý dữ liệu. Bên trong CPU gồm 3 bộ phận cốt lõi:

- **Đơn vị điều khiển (CU - Control Unit):** Giải mã lệnh và phát ra các tín hiệu điều khiển hoạt động của toàn bộ máy tính theo chương trình định sẵn.
- **Đơn vị số học và logic (ALU - Arithmetic and Logic Unit):** Trực tiếp thực hiện các phép toán số học (cộng, trừ, nhân, chia, so sánh) và các phép toán logic (AND, OR, XOR, dịch bit).
- **Tập thanh ghi (RF - Register File):** Các ô nhớ dung lượng nhỏ nhưng có tốc độ truy cập nhanh nhất trong máy tính, nằm ngay bên trong CPU để lưu trữ các thông tin tạm thời phục vụ hoạt động tức thời của CPU.

2. Bộ nhớ máy tính (Memory System)

Gồm 3 thành phần được phân cấp theo tốc độ và dung lượng:

- **Bộ nhớ chính (Main Memory - RAM):** Chứa các lệnh và dữ liệu của chương trình đang được thực hiện. Bộ nhớ chính được chia thành các ngăn nhớ (thường mỗi ngăn nhớ là 1 byte = 8 bit) và mỗi ngăn nhớ có một địa chỉ xác định.
- **Nguyên tắc vàng của ngăn nhớ (rất hay hỏi điền khuyết):** Mỗi ngăn nhớ trong máy tính có **nội dung** (hoặc giá trị) thay đổi được, nhưng **địa chỉ vật lý** của nó luôn cố định.

- **Bộ nhớ đệm (Cache):** Bộ nhớ bán dẫn tốc độ rất nhanh đặt đệm giữa CPU và Bộ nhớ chính nhằm tăng tốc độ CPU truy cập bộ nhớ. Thường chia thành các mức L1, L2, L3 tích hợp ngay trên chip xử lý.
- **Thiết bị lưu trữ (Storage / Bộ nhớ ngoài):** Lưu giữ lâu dài tài nguyên phần mềm và dữ liệu (ổ cứng từ tính HDD, ổ thể rắn bán dẫn SSD, thẻ nhớ). Về mặt cấu trúc kết nối, bộ nhớ ngoài được nối vào máy tính như một thiết bị vào-ra.

3. Hệ thống vào-ra (Input/Output System)

Có chức năng trao đổi thông tin giữa máy tính với thế giới bên ngoài, bao gồm 2 phần phân biệt rõ ràng:

- **Các thiết bị vào-ra (I/O Devices / Thiết bị ngoại vi - Peripherals):** Bàn phím, chuột, màn hình, máy in, ổ cứng, card mạng. Chúng không nối trực tiếp vào Bus hệ thống mà phải đi qua Mô-đun vào-ra.
- **Các mô-đun vào-ra (I/O Modules):** Là mạch ghép nối trung gian giữa Bus hệ thống (CPU/Bộ nhớ) với các thiết bị ngoại vi. Mỗi mô-đun vào-ra chứa một hoặc nhiều **cổng vào-ra (I/O Ports)**, mỗi cổng được gán một địa chỉ xác định. CPU muốn giao tiếp với thiết bị ngoại vi nào thì phải đọc/ghi thông qua địa chỉ của cổng vào-ra tương ứng.

2.1.2 Ba hoạt động cơ bản của máy tính

1. Hoạt động thực hiện chương trình (Chu trình lệnh)

Máy tính thực hiện chương trình bằng cách lặp đi lặp lại **Chu trình lệnh (Instruction Cycle)** gồm các giai đoạn theo đúng thứ tự:

1. **Tìm nạp lệnh (Fetch):** CPU lấy lệnh từ bộ nhớ chính vào CPU. Hai thanh ghi chuyên dụng phục vụ quá trình này là:
   - **Bộ đếm chương trình PC (Program Counter):** Luôn giữ **địa chỉ của lệnh tiếp theo** sẽ được nạp vào. (Trong kiến trúc Intel x86, thanh ghi này được gọi là IP - Instruction Pointer).
   - **Thanh ghi lệnh IR (Instruction Register):** Chứa **mã lệnh vừa được đọc** từ bộ nhớ chính lên.
   - Sau khi nạp lệnh, nếu thực hiện tuần tự, nội dung thanh ghi PC sẽ tự động tăng thêm 4 byte ($PC = PC + 4$ đối với lệnh 32-bit của RISC-V) để trỏ sang lệnh kế tiếp.

2. **Giải mã lệnh (Decode):** Diễn ra ngay sau giai đoạn lấy lệnh (Fetch). Đơn vị điều khiển (CU) phân tích mã lệnh trong IR để xác định thao tác cần làm và các toán hạng cần dùng.

3. **Thực hiện lệnh (Execute):** CPU phát tín hiệu điều khiển thực hiện 1 trong 3 nhóm thao tác cơ bản:

    - Trao đổi dữ liệu (Data Transfer): Đọc/ghi giữa CPU và Bộ nhớ hoặc Mô-đun vào-ra.
    - Xử lý số học / logic (Arithmetic / Logic): Tính toán trên ALU.
    - Điều khiển luồng (Control Flow): Rẽ nhánh có điều kiện (như `beq`, `bne`) hoặc nhảy không điều kiện (như `jal`), bản chất là nạp một địa chỉ đích mới vào thanh ghi PC.

2. Hoạt động Ngắt (Interrupt)

Ngắt là cơ chế cho phép CPU tạm dừng chương trình đang thực hiện để chuyển sang thực hiện một chương trình con xử lý ngắt (Interrupt Handler) có sẵn trong bộ nhớ, sau đó quay lại làm tiếp chương trình cũ.

- **Nguyên nhân gây ra ngắt:** Do lỗi thực hiện lệnh (tràn số, chia cho 0, sai mã lệnh), do lỗi phần cứng, do bộ định thời (Timer), hoặc do mô-đun vào-ra gửi tín hiệu yêu cầu trao đổi dữ liệu.
- **Quy trình 5 bước khi CPU xử lý ngắt từ thiết bị vào-ra:**
    1. Cuối mỗi chu trình lệnh, CPU kiểm tra xem có tín hiệu yêu cầu ngắt hay không.
    2. Nếu có ngắt: Tạm dừng (suspend) chương trình hiện tại và **cất ngữ cảnh** (lưu giá trị PC và các thanh ghi cần thiết ra ngăn xếp).
    3. Nạp địa chỉ bắt đầu của Chương trình con xử lý ngắt vào thanh ghi PC.
    4. Thực thi Chương trình con xử lý ngắt.
    5. **Khôi phục ngữ cảnh** và quay trở về tiếp tục chương trình đang bị tạm dừng.
- **Khi có nhiều yêu cầu ngắt cùng lúc:** Có 2 cách xử lý là *Xử lý ngắt tuần tự* (xử lý xong hết ngắt X mới sang ngắt Y) và *Xử lý ngắt ưu tiên* (ngắt Y có mức ưu tiên cao hơn được phép ngắt ngang ngắt X đang chạy; thường được quản lý bởi vi mạch điều khiển ngắt khả trình **PIC - Programmable Interrupt Controller**).

3. Hoạt động Vào-Ra (3 phương pháp trao đổi dữ liệu)

Đề thi rất hay hỏi so sánh hiệu suất và vai trò của CPU trong 3 phương pháp vào-ra sau:

1. **Vào-ra bằng chương trình (Programmed I/O hay Polling - Thăm dò): CPU** trực tiếp điều khiển toàn bộ quá trình vào-ra. Khi đợi thiết bị ngoại vi sẵn sàng, CPU phải nằm trong một vòng lặp kiểm tra trạng thái thiết bị liên tục (Polling), gây lãng phí thời gian của CPU nhất.

2. **Vào-ra điều khiển bằng ngắt (Interrupt-driven I/O):** CPU phát lệnh vào-ra rồi chuyển sang làm việc khác. Khi thiết bị sẵn sàng sẽ gửi tín hiệu ngắt cho CPU, lúc đó CPU mới tạm dừng việc đang làm để đọc/ghi dữ liệu. Hiệu suất sử dụng CPU cao hơn Polling.

3. **Truy cập bộ nhớ trực tiếp (DMA - Direct Memory Access):** CPU trao quyền điều khiển Bus cho vi mạch chuyên dụng **DMAC** (DMA Controller). DMAC tự điều khiển truyền toàn bộ khối dữ liệu lớn trực tiếp giữa Thiết bị vào-ra và Bộ nhớ chính (RAM) mà **không đi qua CPU**. Đây là phương pháp có hiệu suất sử dụng CPU cao nhất.

Thứ tự hiệu suất sử dụng CPU tăng dần (từ thấp đến cao):

Programmed I/O (Polling) → Interrupt-driven I/O → DMA

2.1.3 Liên kết hệ thống (Cấu trúc Bus và Không gian địa chỉ)

1. Ba loại Bus chức năng trong Bus hệ thống

Độ rộng của Bus là số đường dây truyền bit song song cùng một lúc (chỉ tính độ rộng cho Bus địa chỉ và Bus dữ liệu).

- **Bus địa chỉ (Address Bus):**
  - **Chức năng:** Vận chuyển địa chỉ để xác định vị trí ngăn nhớ hoặc cổng vào-ra.
  - **Hướng truyền:** Là bus **một chiều (unidirectional)**, chỉ truyền địa chỉ từ CPU (hoặc DMAC) tới Bộ nhớ chính và Mô-đun vào-ra.
  - **Công thức tính Không gian địa chỉ:** Nếu Bus địa chỉ có độ rộng là N bit (ký hiệu từ A_0 đến A_{N-1}), thì số lượng địa chỉ tối đa mà hệ thống quản lý được là:

    Không gian địa chỉ $= 2^N$ địa chỉ (nếu đánh địa chỉ theo byte thì bằng 2^N byte)

- **Bus dữ liệu (Data Bus):**
  - **Chức năng:** Vận chuyển cả **các lệnh** (từ bộ nhớ vào CPU) và **dữ liệu** (giữa CPU, bộ nhớ, vào-ra).

  - **Hướng truyền:** Là bus **hai chiều (bidirectional)**.
  - **Số lượng băng nhớ vật lý (Memory Banks):** Khi bộ nhớ chính được thiết kế kiểu đan xen theo byte (Low-order interleaving), số lượng băng nhớ vật lý đúng bằng số byte truyền đồng thời trên Bus dữ liệu:

$$\text{Số băng nhớ } K = \frac{\text{Độ rộng Bus dữ liệu (bit)}}{8}$$

    Một byte nhớ có địa chỉ X sẽ nằm trên băng nhớ thứ: (X mod K).

- **Bus điều khiển (Control Bus):** Vận chuyển các tín hiệu điều khiển đọc/ghi, ngắt và xin nhượng bus:
  - Điều khiển đọc/ghi (từ CPU phát ra): `MEMR` (đọc bộ nhớ), `MEMW` (ghi bộ nhớ), `IOR` (đọc cổng vào-ra), `IOW` (ghi cổng vào-ra).
  - Điều khiển ngắt: `INTR` (yêu cầu ngắt có thể che), `INTA` (chấp nhận ngắt), `NMI` (ngắt không thể che), `Reset` (khởi động lại).
  - Điều khiển nhượng bus (cho DMA): `BRQ` (Bus Request - xin nhượng bus), `BGT` (Bus Grant - chấp nhận nhượng bus).

2. Bảng tra nhanh lũy thừa của 2 để tính độ rộng Bus địa chỉ (N bit)

Dung lượng bộ nhớ	**Số bit địa chỉ (N)**	**Dung lượng bộ nhớ**	**Số bit địa chỉ (N)**
1 KiB = 2^{10} B	10 bit	1 GiB = 2^{30} B	30 bit
64 KiB = $2^6 \times 2^{10}$ B	16 bit	2 GiB = $2^1 \times 2^{30}$ B	31 bit
1 MiB = 2^{20} B	20 bit	4 GiB = $2^2 \times 2^{30}$ B	32 bit
8 MiB = $2^3 \times 2^{20}$ B	23 bit	8 GiB = $2^3 \times 2^{30}$ B	33 bit

3. Liên kết điểm-điểm hiện đại (Point-to-Point Interconnect)

Bus dùng chung (Shared bus) truyền thống bị nghẽn cổ chai vì tại một thời điểm chỉ phục vụ được một yêu cầu và cần bộ phân xử bus. Các máy tính hiện đại thay thế bằng liên kết điểm-điểm tốc độ cao, truyền dữ liệu dạng gói:

- **QPI (Quick Path Interconnect):** Kết nối trực tiếp giữa các nhân xử lý (Cores), các CPU và Chipset.
- **PCI Express (PCIe):** Liên kết điểm-điểm tốc độ cao thay thế cho PCI bus cũ để kết nối với các thiết bị vào-ra tốc độ cao (như Card đồ họa rời, ổ cứng SSD NVMe, Card mạng Gigabit).

2.2 Phần 2: Toàn bộ bài tập Chương 2 và Hướng dẫn giải

2.2.1 Nhóm 1: Chu trình lệnh, các thanh ghi PC/IR/SP và Ngăn nhớ

Bài tập 2.1 (Đề Giữa kỳ 2022.2 - Đề 0009, Câu 11)

Điền từ còn thiếu vào 2 ô trống trong phát biểu sau: "Mỗi ngăn nhớ trong máy tính có ..... thay đổi được, nhưng ..... của nó luôn cố định."

Hướng dẫn giải:

- Đối chiếu trực tiếp với định nghĩa Bộ nhớ chính (Slide trang 9, Chương 2): Mỗi ngăn nhớ có nội dung (dữ liệu lưu bên trong) có thể đọc/ghi thay đổi liên tục, nhưng địa chỉ vật lý của ngăn nhớ đó là cố định.
- **Đáp số:** Ô trống thứ nhất điền **nội dung** (hoặc **giá trị**); Ô trống thứ hai điền **địa chỉ** (hoặc **địa chỉ vật lý**).

Bài tập 2.2 (Đề Giữa kỳ 2022.2 - Đề 0008 & 0009, Câu 10)

Tên của 2 thanh ghi phục vụ cho quá trình nhận lệnh (tìm nạp lệnh) từ bộ nhớ chính là gì? Viết tên đầy đủ, hoặc viết tắt.

Hướng dẫn giải:

- Trong giai đoạn Tìm nạp lệnh (Fetch), CPU dùng thanh ghi **PC** để phát địa chỉ lệnh ra bộ nhớ, và nhận mã lệnh từ bộ nhớ nạp vào thanh ghi **IR**.
- **Đáp số: PC (Program Counter - Bộ đếm chương trình)** và **IR (Instruction Register - Thanh ghi lệnh)**.

Bài tập 2.3 (Ôn tập Giữa kỳ - Câu 60)

Thanh ghi nào trong CPU luôn giữ địa chỉ của lệnh tiếp theo sẽ được lấy từ bộ nhớ?

A. Thanh ghi Địa chỉ Bộ nhớ (Memory Address Register - MAR)

B. Thanh ghi Bộ đếm Chương trình (Program Counter - PC)

C. Thanh ghi Lệnh (Instruction Register - IR)

D. Thanh ghi Dữ liệu Bộ nhớ (Memory Buffer Register - MBR)

Hướng dẫn giải:

- Thanh ghi giữ địa chỉ của lệnh tiếp theo sẽ được nạp vào CPU là Bộ đếm chương trình **PC (Program Counter)**.
- **Đáp án đúng: B.**

Bài tập 2.4 (Đề Cuối kỳ 2022.2 - #6f0a57, Giữa kỳ 2022.2 - Đề 0006 Câu 4 & Ôn tập Cuối kỳ P2 - Câu 14)

Hãy chỉ ra tên của thành phần trong máy tính thực hiện chức năng chứa địa chỉ của ngăn nhớ đỉnh ngăn xếp?

A. Stack Pointer (SP)

B. Base Register

C. Program Counter (PC)

D. Instruction Register (IR)

Hướng dẫn giải:

- Con trỏ ngăn xếp **Stack Pointer (SP)** (trong RISC-V là thanh ghi `sp` hay `x2`) luôn chứa địa chỉ của ngăn nhớ đỉnh ngăn xếp.
- **Đáp án đúng: A (Stack Pointer - SP).**

Bài tập 2.5 (Ôn tập Giữa kỳ - Câu 40)

Trong chu trình thực hiện lệnh cơ bản (Instruction Cycle), giai đoạn nào diễn ra ngay sau khi CPU lấy lệnh (Fetch) từ bộ nhớ?

A. Giai đoạn Thực thi (Execute), nơi lệnh được xử lý bởi ALU.

B. Giai đoạn Ghi lại kết quả (Write Back), nơi kết quả được lưu vào thanh ghi hoặc bộ nhớ.

C. Giai đoạn Điều khiển ngắt (Interrupt Check), để xử lý các yêu cầu I/O.

D. Giai đoạn Giải mã (Decode), nơi CPU xác định loại lệnh và các toán hạng.

Hướng dẫn giải:

- Ngay sau khi lấy mã lệnh từ bộ nhớ vào thanh ghi IR (Fetch), CPU phải tiến hành **Giải mã lệnh (Decode)** để biết lệnh đó yêu cầu thực hiện phép toán gì và lấy toán hạng ở đâu, rồi mới sang bước Thực thi (Execute).
- **Đáp án đúng: D.**

Bài tập 2.6 (Ôn tập Giữa kỳ - Câu 43)

Lệnh `beq rs1, rs2, offset` là ví dụ điển hình cho loại hoạt động cơ bản nào của máy tính?

A. Hoạt động truyền dữ liệu (Data Transfer Operation)

B. Hoạt động số học (Arithmetic Operation)

C. Hoạt động điều khiển luồng (Control Flow Operation)

D. Hoạt động I/O (I/O Operation)

Hướng dẫn giải:

- Lệnh `beq` (branch if equal) kiểm tra điều kiện để thay đổi trình tự thực hiện lệnh (rẽ nhánh sang nhãn khác thay vì chạy tuần tự), nên thuộc nhóm **Hoạt động điều khiển luồng (Control Flow Operation)**.
- **Đáp án đúng: C.**

2.2.2 Nhóm 2: Hệ thống Bus và Tính toán độ rộng Bus địa chỉ

Bài tập 2.7 (Ôn tập Giữa kỳ - Câu 14)

Bus Địa chỉ (Address Bus) có đặc điểm nào sau đây, đặc biệt là về hướng truyền?

A. Là bus hai chiều, truyền tín hiệu điều khiển và dữ liệu.

B. Là bus hai chiều (bidirectional), truyền cả địa chỉ và dữ liệu.

C. Là bus một chiều, truyền tín hiệu điều khiển từ bộ nhớ đến CPU.

D. Là bus một chiều (unidirectional), truyền địa chỉ từ CPU/DMA đến bộ nhớ/I/O.

Hướng dẫn giải:

- Chỉ có CPU (hoặc bộ điều khiển DMA khi được nhượng bus) mới phát địa chỉ ra để tìm ngăn nhớ hoặc cổng vào-ra. Bộ nhớ và I/O chỉ nhận địa chỉ chứ không phát địa chỉ ngược lại. Do đó Address Bus là **bus một chiều (unidirectional)**.
- **Đáp án đúng: D.**

Bài tập 2.8 (Ôn tập Giữa kỳ - Câu 56)

Bus dữ liệu (Data Bus) trong kiến trúc máy tính chịu trách nhiệm truyền tải loại thông tin nào?

A. Các lệnh và dữ liệu giữa các thành phần trong hệ thống máy tính

B. Địa chỉ bộ nhớ của dữ liệu cần truy cập

C. Tín hiệu điều khiển để đồng bộ hóa hoạt động

D. Các dữ liệu giữa các thành phần trong hệ thống máy tính

Hướng dẫn giải:

- Theo Slide trang 34 (Chương 2), chức năng của Bus dữ liệu gồm 2 ý: (1) Vận chuyển **lệnh** từ bộ nhớ đến CPU, và (2) Vận chuyển **dữ liệu** giữa các thành phần của máy tính với nhau. Do đó phương án A đầy đủ và chính xác nhất (nếu biểu mẫu trắc nghiệm chỉ cài đặt đáp án hẹp theo tên gọi thì mới xét D, nhưng về mặt lý thuyết chuẩn của bài giảng là cả lệnh và dữ liệu).
- **Đáp án chuẩn theo giáo trình: A (Các lệnh và dữ liệu giữa các thành phần trong hệ thống máy tính).**

Bài tập 2.9 (Đề Giữa kỳ 2022.2 - Đề 0008, Câu 3)

Một máy tính có bus hệ thống gồm bus dữ liệu 32 bit và bus địa chỉ 23 bit, quản lý bộ nhớ theo byte. Hỏi không gian địa chỉ bộ nhớ có kích thước bao nhiêu byte?

Hướng dẫn giải:

- Không gian địa chỉ chỉ phụ thuộc vào độ rộng của **Bus địa chỉ** ($N = 23$ bit), không phụ thuộc vào Bus dữ liệu.
- Vì máy tính quản lý bộ nhớ theo từng byte nên kích thước không gian địa chỉ là:

$$2^{23} \text{ byte} = 2^3 \times 2^{20} \text{ byte} = 8 \text{ MiB (hoặc } 8 \text{ MB} = 8\,388\,608 \text{ byte)}$$

- **Đáp số:** 2^{23} **byte** $= 8$ **MiB (hoặc** 8 **MB).**

Bài tập 2.10 (Đề Giữa kỳ 2022.2 - Đề 0006, Câu 3)

Nếu như `0x3FF` là địa chỉ bộ nhớ lớn nhất mà CPU có thể quản lý được, thì bus địa chỉ có độ rộng bằng bao nhiêu bit?

Hướng dẫn giải:

- Đổi địa chỉ lớn nhất `0x3FF` từ hệ Hexa sang hệ nhị phân (mỗi chữ số Hexa ứng với 4 bit nhị phân):

$$\texttt{0x3FF} = \underbrace{0011}_{3}\ \underbrace{1111}_{\text{F}}\ \underbrace{1111}_{\text{F}}{}_{(2)} = 11\,1111\,1111_{(2)}$$

- Đếm số lượng bit 1 liên tiếp: có tất cả $2+4+4 = 10$ bit 1 (tương ứng giá trị $2^{10}-1 = 1023$).
- Như vậy địa chỉ chạy từ `0x000` đến `0x3FF` gồm $2^{10} = 1024$ địa chỉ, suy ra độ rộng Bus địa chỉ là 10 bit.
- **Đáp số:** 10 **bit.**

Bài tập 2.11 (Đề Cuối kỳ 2020.2 & Ôn tập Cuối kỳ P2 - Câu 28)

Cho biết không gian địa chỉ bộ nhớ là 2 GiB, hãy cho biết độ rộng bus địa chỉ tương ứng của máy tính là bao nhiêu bit?

A. 31 bit

B. 30 bit

C. 2 bit

D. 21 bit

Hướng dẫn giải:

- Ta có 1 GiB = 2^{30} byte.
- Suy ra 2 GiB = $2^1 \times 2^{30}$ byte = 2^{31} byte.
- Để đánh địa chỉ cho 2^{31} byte nhớ, bus địa chỉ cần có độ rộng là 31 bit.
- **Đáp án đúng: A (31 bit).**

Bài tập 2.12 (Đề Cuối kỳ 2022.2 - #bb55ef & Ôn tập Cuối kỳ P2 - Câu 36)

Một hệ thống máy tính gồm có 4 MiB bộ nhớ cache, 8 GiB bộ nhớ chính, 1 TiB bộ nhớ thể rắn SSD. Hỏi bus địa chỉ của hệ thống máy tính cần có độ rộng tối thiểu bao nhiêu bit?

Hướng dẫn giải:

- **Phân tích bản chất vật lý (theo chuẩn đề thi SoICT):**
  - Bộ nhớ Cache là bộ nhớ đệm trong suốt, tự động ánh xạ từ địa chỉ bộ nhớ chính sang nên không cần đánh địa chỉ riêng trên Bus hệ thống.
  - Ổ cứng thể rắn SSD là thiết bị lưu trữ ngoài (bộ nhớ ngoài), giao tiếp qua cổng của mô-đun vào-ra theo khối (block), không được CPU đánh địa chỉ trực tiếp từng byte trên Bus địa chỉ hệ thống.
  - Bus địa chỉ của hệ thống chỉ dùng để đánh địa chỉ trực tiếp cho từng byte của **Bộ nhớ chính (RAM)** có dung lượng 8 GiB:

$$8 \text{ GiB} = 2^3 \times 2^{30} \text{ byte} = 2^{33} \text{ byte} \implies N = 33 \text{ bit}$$

- **Lưu ý về đáp án trong file Ôn tập P2:**
  - Trong kỳ thi thật trên hệ thống `daotao.ai` của trường, đáp án đúng là **33** (tính theo 8 GiB bộ nhớ chính).
  - Tuy nhiên, trong file Microsoft Forms *Ôn tập P2 (Câu 36)* do sinh viên tự tạo, người làm form đã hiểu nhầm là cộng cả ổ cứng SSD (1 TiB = 2^{40}) nên cài đáp án là 41. Khi đi thi thật trên trường, bạn nhớ điền **33**.
- **Đáp số chuẩn kiến trúc máy tính: 33 bit.**

Bài tập 2.13 (Ôn tập Cuối kỳ P2 - Câu 16)

Không gian địa chỉ bộ nhớ dùng chung (Memory-mapped I/O) dùng để đánh địa chỉ cho các đối tượng nào?

A. Tất cả đáp án trên

B. Các ngăn nhớ và mô-đun vào-ra

C. Các ngăn nhớ và các thiết bị ngoại vi

D. Các ngăn nhớ

E. Các ngăn nhớ và các cổng vào-ra

Hướng dẫn giải:

- Theo Slide trang 31 (Chương 3) và trang 33 (Chương 2): Không gian địa chỉ bộ nhớ dùng chung đánh địa chỉ trực tiếp cho **các ngăn nhớ** (trong bộ nhớ chính) và **các cổng vào-ra** (trên mô-đun vào-ra). Thiết bị ngoại vi nối vào cổng vào-ra chứ không trực tiếp mang địa chỉ bộ nhớ.
- **Đáp án đúng: E (Các ngăn nhớ và các cổng vào-ra).**

Bài tập 2.14 (Ôn tập Giữa kỳ - Câu 27)

Cổng truy cập (Port) của thiết bị Vào/Ra (I/O) được ánh xạ vào không gian địa chỉ bộ nhớ (Memory-Mapped I/O) có ý nghĩa gì?

A. Thiết bị I/O có bus dữ liệu riêng biệt với bộ nhớ chính.

B. Thiết bị I/O có thể tự truy cập bộ nhớ mà không cần sự can thiệp của CPU.

C. Thiết bị I/O chỉ có thể được truy cập thông qua các lệnh LOAD/STORE, giống như truy cập bộ nhớ.

D. CPU cần một tập lệnh I/O đặc biệt (ví dụ: IN/OUT) để giao tiếp với thiết bị.

Hướng dẫn giải:

- Trong cơ chế vào-ra ánh xạ bộ nhớ (Memory-Mapped I/O) mà RISC-V sử dụng, các cổng vào-ra được coi như các ngăn nhớ đặc biệt. CPU không có lệnh `IN/OUT` riêng mà dùng chính các lệnh đọc/ghi bộ nhớ (`lw`, `sw`, `lb`, `sb`) để giao tiếp với thiết bị I/O.
- **Đáp án đúng: C.**

Bài tập 2.15 (Đề Cuối kỳ 2021.2 - Mã đề B3, Câu 4.1 & 4.2)

Máy tính dùng 36 bit địa chỉ để đánh địa chỉ cho bộ nhớ theo byte, 16 bit để đánh địa chỉ cho cổng vào-ra riêng biệt; bus dữ liệu để kết nối với bộ nhớ chính là 64 bit. Hãy cho biết:

4.1. Số byte nhớ tối đa được đánh địa chỉ? Địa chỉ đầu và địa chỉ cuối dưới dạng Hexa?

4.2. Số cổng vào-ra tối đa được đánh địa chỉ vào-ra riêng biệt? Địa chỉ đầu và địa chỉ cuối dưới dạng Hexa?

Hướng dẫn giải:

Giải 4.1 – Không gian địa chỉ bộ nhớ (36 bit):

- Số byte nhớ tối đa được đánh địa chỉ: 2^{36} byte $= 2^6 \times 2^{30}$ byte $= 64$ GiB.
- Vì 36 bit chia cho 4 được đúng 9 chữ số Hexa, nên:
  - Địa chỉ đầu (36 bit 0): `0x000000000` (9 chữ số 0).
  - Địa chỉ cuối (36 bit 1): `0xFFFFFFFFF` (9 chữ số F).

Giải 4.2 – Không gian địa chỉ cổng vào-ra (16 bit):

- Số cổng vào-ra tối đa được đánh địa chỉ: $2^{16} = 65\,536$ cổng (64K cổng).
- Vì 16 bit ứng với 4 chữ số Hexa, nên:
  - Địa chỉ cổng đầu: `0x0000`.
  - Địa chỉ cổng cuối: `0xFFFF`.

2.2.3 Nhóm 3: Bài toán xác định Băng nhớ vật lý (Memory Banks)

Phương pháp giải chung cực dễ nhớ:

1. Tính số lượng băng nhớ K từ độ rộng Bus dữ liệu:
   - Nếu Bus dữ liệu = 32 bit = 4 byte $\implies$ Có $K = 4$ băng nhớ (đánh số từ Băng 0, Băng 1, Băng 2, Băng 3).
   - Nếu Bus dữ liệu = 64 bit = 8 byte $\implies$ Có $K = 8$ băng nhớ (đánh số từ Băng 0 đến Băng 7).

2. Để biết byte nhớ có địa chỉ Hexa nằm ở băng nào, bạn chỉ cần lấy **chữ số Hexa cuối cùng bên phải** của địa chỉ đó đem **chia lấy dư cho** K (4 hoặc 8). Số dư chính là số thứ tự của băng nhớ!

Bài tập 2.16 (Đề Cuối kỳ 2022.2 - #38a80d, 2024.1 - #cbb2f4 & Ôn tập Cuối kỳ P2 - Câu 32)

Cho một máy tính có bus địa chỉ 32 bit và bus dữ liệu 32 bit. Bộ nhớ chính được đánh địa chỉ theo byte và được thiết kế theo kiểu băng nhớ đan xen. Hãy cho biết byte nhớ có địa chỉ `0x10011647` nằm trên băng nhớ nào?

A. Băng 1

B. Băng 3

C. Băng 2

D. Băng 0

Hướng dẫn giải:

- Bước 1: Bus dữ liệu là 32 bit = 4 byte, nên bộ nhớ được chia thành $K = 4$ băng nhớ (Băng 0, 1, 2, 3).
- Bước 2: Lấy chữ số Hexa cuối cùng của địa chỉ `0x10011647` là chữ số 7.
- Bước 3: Thực hiện phép chia lấy dư cho 4:

$$7 \div 4 = 1 \text{ dư } \mathbf{3}$$

- Vậy byte nhớ này nằm trên **Băng 3**.
- **Đáp án đúng: B (Băng 3).**

Bài tập 2.17 (Đề Cuối kỳ 2021.2 - Mã đề B3, Câu 4.3)

Máy tính có bus dữ liệu để kết nối với bộ nhớ chính là 64 bit (bộ nhớ đánh địa chỉ theo byte). Hãy cho biết các byte nhớ có địa chỉ sau đây: `0x0FE12C39` và `0x10ABCD05` được bố trí ở băng nhớ nào?

Hướng dẫn giải:

- Bước 1: Bus dữ liệu là 64 bit = 8 byte $\implies$ Bộ nhớ chính gồm $K = 8$ băng nhớ (đánh số từ Băng 0 đến Băng 7).
- Bước 2: Xét địa chỉ `0x0FE12C39`, chữ số Hexa cuối cùng là 9. Chia lấy dư cho 8:

$$9 \bmod 8 = \mathbf{1} \implies \text{Nằm ở } \mathbf{Băng\ 1}$$

- Bước 3: Xét địa chỉ `0x10ABCD05`, chữ số Hexa cuối cùng là 5. Chia lấy dư cho 8:

$$5 \bmod 8 = \mathbf{5} \implies \text{Nằm ở } \mathbf{Băng\ 5}$$

- **Đáp số:** Địa chỉ `0x0FE12C39` ở **Băng 1**; Địa chỉ `0x10ABCD05` ở **Băng 5**.

Bài tập 2.18 (Đề Cuối kỳ 2021.2 - Mã đề B1, Câu 4 & Ôn tập Cuối kỳ P2 - Câu 26, Câu 44)

Trên máy tính dùng bộ xử lý có bus địa chỉ 32 bit, bus dữ liệu 64 bit, các thanh ghi bên trong có độ dài 32 bit. Giả sử có một mảng dữ liệu hai chiều B có kích thước 5×7 (5 hàng, 7 cột) của các phần tử 32 bit (4 byte). Mảng B được lưu trữ theo hàng trong bộ nhớ chính, cấp phát ban đầu từ địa chỉ `0x000A5020`.

a) (Ôn tập P2 - Câu 26): Nếu chỉ số hàng và cột tính từ 0 ($i = 0..4, j = 0..6$), cho biết địa chỉ của phần tử $B[1, 4]$?

b) (Ôn tập P2 - Câu 44 & Đề 2021.2 B1): Hỏi phần tử $B[1, 5]$ (tính chỉ số từ 0, tương ứng $B[2, 6]$ nếu tính từ 1) và phần tử $B[1, 4]$ (tương ứng $B[2, 5]$ nếu tính từ 1) nằm trên các băng nhớ nào của bộ nhớ vật lý?

Hướng dẫn giải:

Giải ý a) Tính địa chỉ phần tử mảng 2 chiều lưu theo hàng:

- Mảng có 7 cột (mỗi hàng có 7 phần tử), mỗi phần tử kích thước 32 bit = 4 byte.
- Với chỉ số bắt đầu từ 0 ($i = 0..4, j = 0..6$), số phần tử đứng trước $B[i, j]$ là $(i \times 7 + j)$.
- Xét phần tử $B[1, 4]$ (hàng $i = 1$, cột $j = 4$):
  - Số phần tử đứng trước nó: $1 \times 7 + 4 = 11$ phần tử.
  - Độ lệch địa chỉ (Offset) tính theo byte: 11×4 byte = 44 byte = `0x2C` byte.
  - Địa chỉ của $B[1, 4]$: `0x000A5020` + `0x2C` = `0x000A504C`.

Giải ý b) Xác định các băng nhớ chứa phần tử 4 byte:

- Bus dữ liệu 64 bit = 8 byte $\implies$ Có $K = 8$ băng nhớ (từ Băng 0 đến Băng 7). Mỗi phần tử 32 bit chiếm **4 byte liên tiếp**, tức sẽ nằm trải trên **4 băng nhớ liên tiếp**.
- **Đối với phần tử $B[1, 4]$ (hoặc $B[2, 5]$ khi đếm từ 1):**
  - Địa chỉ bắt đầu là `0x000A504C`, chiếm 4 byte ở các địa chỉ kết thúc bằng: `C` (12), `D` (13), `E` (14), `F` (15).
  - Chia lấy dư lần lượt cho 8: 12 mod 8 = 4; 13 mod 8 = 5; 14 mod 8 = 6; 15 mod 8 = 7.
  - Vậy phần tử này nằm trên các băng nhớ: **4, 5, 6, 7**.
- **Đối với phần tử $B[1, 5]$ (trong Câu 44 - Ôn tập P2):**
  - Phần tử $B[1, 5]$ đứng ngay sau $B[1, 4]$, nên có địa chỉ bắt đầu là `0x000A504C`+4 = `0x000A5050`.
  - Chiếm 4 byte ở các địa chỉ kết thúc bằng: `0`, `1`, `2`, `3`.
  - Chia lấy dư cho 8 được các số dư: 0, 1, 2, 3.
  - Vậy phần tử $B[1, 5]$ nằm trên các băng nhớ: **0, 1, 2, 3**.

2.2.4 Nhóm 4: Các phương pháp Vào-Ra (Polling, Interrupt, DMA) và Thiết bị lưu trữ

Bài tập 2.19 (Đề Cuối kỳ 2022.2 - #169b6e & Ôn tập Giữa kỳ - Câu 21)

Với phương pháp Vào-ra bằng chương trình (Programmed I/O), thành phần nào của máy tính trực tiếp điều khiển vào-ra?

A. Thiết bị vào-ra

B. CPU

C. DMAC

D. RAM

Hướng dẫn giải:

- Trong phương pháp Vào-ra bằng chương trình, **CPU** phải trực tiếp thực hiện các lệnh kiểm tra trạng thái và đọc/ghi dữ liệu với cổng vào-ra.
- **Đáp án đúng: B (CPU).**

Bài tập 2.20 (Ôn tập Cuối kỳ P2 - Câu 22)

Trong phương pháp vào/ra bằng chương trình (Programmed I/O), CPU dành phần lớn thời gian để làm gì khi đợi thiết bị ngoại vi sẵn sàng?

A. Giải phóng Bus hệ thống cho các thiết bị khác.

B. Thực hiện các tiến trình khác của người dùng.

C. Kiểm tra trạng thái thiết bị trong một vòng lặp (Polling).

D. Tự động chuyển sang chế độ nghỉ (Sleep mode).

Hướng dẫn giải:

- Vì thiết bị ngoại vi rất chậm so với CPU, trong Programmed I/O, CPU phải liên tục đọc thanh ghi trạng thái của thiết bị trong một vòng lặp (gọi là **Polling** - hỏi vòng/thăm dò) cho đến khi thiết bị báo sẵn sàng.
- **Đáp án đúng: C.**

Bài tập 2.21 (Ôn tập Cuối kỳ P2 - Câu 27)

Thứ tự nào sau đây phản ánh đúng sự phát triển về hiệu suất (từ thấp đến cao) đối với việc sử dụng CPU trong I/O?

A. Interrupt-driven → Programmed I/O → DMA

B. DMA → Interrupt-driven → Programmed I/O

C. Programmed I/O → DMA → Interrupt-driven

D. Programmed I/O → Interrupt-driven → DMA

Hướng dẫn giải:

- Programmed I/O làm CPU bị bận 100% thời gian chờ đợi (hiệu suất thấp nhất) → Interrupt-driven giải phóng CPU trong lúc chờ đợi → DMA giải phóng CPU cả trong lúc truyền khối dữ liệu (hiệu suất cao nhất).
- **Đáp án đúng: D (Programmed I/O → Interrupt-driven → DMA).**

Bài tập 2.22 (Ôn tập Cuối kỳ P2 - Câu 62)

Tại sao phương pháp DMA (Direct Memory Access) lại hiệu quả hơn khi truyền các khối dữ liệu lớn?

A. Vì dữ liệu được truyền trực tiếp giữa thiết bị và bộ nhớ mà không đi qua CPU.

B. Vì nó không cần sử dụng bộ nhớ chính (RAM).

C. Vì DMA sử dụng phần mềm thay vì phần cứng.

D. Vì nó cho phép CPU thực hiện lệnh truyền dữ liệu nhanh hơn.

Hướng dẫn giải:

- DMA sử dụng bộ điều khiển phần cứng DMAC để truyền dữ liệu trực tiếp giữa mô-đun vào-ra và bộ nhớ chính (RAM) mà không cần CPU phải can thiệp vào từng từ dữ liệu.
- **Đáp án đúng: A.**

Bài tập 2.23 (Ôn tập Cuối kỳ P2 - Câu 45 & Câu 57)

a) (Câu 45) Bộ điều khiển DMA (DMAC) gồm những thành phần nào dưới đây? (Chọn nhiều đáp án)

- 1. Thanh ghi con trỏ lệnh 2. Logic điều khiển 3. Bộ đếm dữ liệu 4. Thanh ghi địa chỉ 5. Thanh ghi dữ liệu

b) (Câu 57) Cách thực hiện DMA nào dưới đây là tối ưu và khó thực hiện hơn cả?

- A. DMA lấy chu kỳ B. DMA trong suốt C. Ba phương pháp tương tự nhau D. DMA truyền theo khối

Hướng dẫn giải:

- **Ý a):** DMAC không thực thi chương trình nên **không có** Thanh ghi con trỏ lệnh (PC). DMAC gồm 4 thành phần: **Logic điều khiển, Bộ đếm dữ liệu (Data counter), Thanh ghi địa chỉ, và Thanh ghi dữ liệu**.
- **Ý b): DMA trong suốt (Transparent DMA)** chỉ tranh thủ sử dụng Bus hệ thống vào những chu kỳ mà CPU không dùng đến Bus (CPU đang xử lý nội bộ), nên không làm chậm CPU chút nào (tối ưu nhất), nhưng mạch đồng bộ phần cứng khó thiết kế nhất. (Chọn **B**).

Bài tập 2.24 (Ôn tập Cuối kỳ P2 - Câu 54)

Những khẳng định nào dưới đây đúng về PIC? (Chọn nhiều đáp án)

A. Trong PIC, yêu cầu ngắt có mức ưu tiên cao nhất sẽ không thể bị cấm và do đó được gửi tới CPU đầu tiên.

B. Các đường vào yêu cầu ngắt của PIC có quy định mức ưu tiên.

C. Đây là 1 phương pháp nối ghép ngắt (cụ thể là hỏi vòng bằng phần cứng).

D. PIC là viết tắt của Programmable Interrupt Controller.

Hướng dẫn giải:

- **PIC** là viết tắt của **Programmable Interrupt Controller** (Bộ điều khiển ngắt khả trình).
- Các đường vào yêu cầu ngắt ($IR_0 \dots IR_7$) của PIC **có quy định mức ưu tiên**.
- Phương án A sai vì mọi ngắt đi qua PIC đều nối vào chân `INTR` của CPU nên đều là ngắt *có thể bị che (cấm)*. Phương án C sai vì PIC là phương pháp ngắt ưu tiên phần cứng độc lập, không phải hỏi vòng (daisy chain/polling).
- **Đáp án đúng: B và D.**

Bài tập 2.25 (Ôn tập Cuối kỳ P2 - Câu 24 & Câu 51)

a) (Câu 51) Những đặc điểm nào sau đây là ưu điểm của ổ cứng bán dẫn SSD so với ổ đĩa từ HDD?

- 1. Giá thành trên mỗi đơn vị lưu trữ (GB) rẻ hơn.

- 2. Không gây tiếng ồn khi hoạt động.
- 3. Tốc độ truy xuất dữ liệu nhanh hơn và độ trễ thấp hơn.
- 4. Khả năng chống sốc vật lý tốt hơn.

b) (Câu 24) Trong thực tế hiện nay, những nhận định nào sau đây là đúng về việc sử dụng HDD và SSD trên các thiết bị?

- 1. HDD vẫn được sử dụng phổ biến trong các hệ thống Server lưu trữ (NAS) hoặc trung tâm dữ liệu nhờ ưu thế về dung lượng lớn và giá thành rẻ.
- 2. Camera giám sát (CCTV) ghi hình 24/7 thường ưu tiên dùng SSD vì SSD có tuổi thọ ghi xóa dữ liệu cao hơn HDD khi hoạt động liên tục.
- 3. Các máy chơi game thế hệ mới (như PS5, Xbox Series X) sử dụng SSD để giảm thời gian chờ tải màn chơi (loading screen).
- 4. SSD thường được ưu tiên dùng làm ổ cài đặt hệ điều hành trên Laptop và PC để tăng tốc độ khởi động máy.
- 5. Các thiết bị di động nhỏ gọn như Smartphone và Tablet sử dụng chip nhớ có nguyên lý tương tự HDD.

Hướng dẫn giải:

- **Ý a):** SSD dùng chip nhớ bán dẫn Flash (không có đĩa quay cơ học như HDD) nên có 3 ưu điểm vượt trội: **Không gây tiếng ồn (2), Tốc độ truy xuất nhanh và độ trễ thấp (3), Chống sốc vật lý tốt hơn (4)**. Nhược điểm của SSD là giá thành trên mỗi GB đắt hơn HDD (nên ý 1 sai).
- **Ý b):** Các nhận định đúng là **1, 3 và 4**. Nhận định 2 sai vì chip nhớ Flash của SSD có giới hạn số lần ghi xóa nên camera ghi đè 24/7 thường dùng ổ HDD chuyên dụng. Nhận định 5 sai vì điện thoại dùng bộ nhớ bán dẫn Flash (nguyên lý giống SSD chứ không phải HDD).

CHƯƠNG 3

KIẾN TRÚC TẬP LỆNH RISC-V

3.1 Phần 1: Lý thuyết trọng tâm

3.1.1 Tập 32 thanh ghi của RISC-V 32-bit

Bộ xử lý RISC-V 32-bit có tất cả **32 thanh ghi đa dụng**, mỗi thanh ghi có độ dài 32 bit. Vì $32 = 2^5$, nên trong mã máy, mỗi thanh ghi được mã hóa bằng đúng **5 bit** (từ `00000` ứng với `x0` đến `11111` ứng với `x31`).

Khi lập trình và làm bài thi dịch mã máy, bạn bắt buộc phải tra cứu hoặc thuộc bảng ánh xạ giữa **Tên gợi nhớ** và **Số hiệu thanh ghi** dưới đây:

Tên	Số hiệu	Chức năng sử dụng	Bảo toàn
`zero`	`x0`	Thanh ghi hằng số, luôn bằng 0 (chỉ đọc)	Không áp dụng
`ra`	`x1`	Return Address: chứa địa chỉ trở về hàm	Caller-saved
`sp`	`x2`	Stack Pointer: con trỏ đỉnh ngăn xếp	Callee-saved
`gp`	`x3`	Global Pointer: con trỏ dữ liệu toàn cục	Không áp dụng
`tp`	`x4`	Thread Pointer: con trỏ luồng	Không áp dụng
`t0 - t2`	`x5 - x7`	Temporaries: thanh ghi biến tạm thời	Caller-saved
`s0 / fp`	`x8`	Saved register / Frame pointer: lưu biến	Callee-saved
`s1`	`x9`	Saved register: thanh ghi lưu biến	Callee-saved
`a0 - a1`	`x10 - x11`	Arguments / Return values: tham số & kết quả	Caller-saved
`a2 - a7`	`x12 - x17`	Arguments: tham số truyền vào hàm con	Caller-saved
`s2 - s11`	`x18 - x27`	Saved registers: thanh ghi lưu biến	Callee-saved
`t3 - t6`	`x28 - x31`	Temporaries: thanh ghi biến tạm thời	Caller-saved

Giải thích quy ước bảo toàn thanh ghi khi gọi hàm (rất hay hỏi trắc nghiệm):

- **Các thanh ghi tạm thời (`t0 - t6`):** Thuộc loại **Caller-saved** (bên gọi hàm phải tự cất vào ngăn xếp nếu muốn giữ lại giá trị sau khi hàm con chạy xong).
- **Các thanh ghi lưu trữ (`s0 - s11`):** Thuộc loại **Callee-saved** (hàm con được gọi nếu muốn dùng các thanh ghi `s0 - s11` thì bắt buộc phải cất giá trị cũ vào ngăn xếp ở đầu hàm và khôi phục lại trước khi thoát hàm).

3.1.2 Tổ chức dữ liệu trong bộ nhớ và Quy tắc Little-Endian

1. Đơn vị dữ liệu trong RISC-V

- **Byte:** 8 bit (chiếm 1 ngăn nhớ). Địa chỉ có thể chẵn hoặc lẻ bất kỳ.
- **Halfword (Nửa từ):** 16 bit = 2 byte. Địa chỉ bắt buộc phải là **số chẵn** (chia hết cho 2, kết thúc bằng `0, 2, 4, 6, 8, A, C, E`).
- **Word (Từ nhớ):** 32 bit = 4 byte. Địa chỉ bắt buộc phải là **bội số của 4** (chia hết cho 4, chữ số Hexa cuối cùng phải là `0, 4, 8, C`). Địa chỉ của một Word nhớ chính là địa chỉ của byte đầu tiên (byte có địa chỉ nhỏ nhất) thuộc Word đó.

2. Quy tắc lưu trữ Đầu nhỏ (Little-Endian)

Kiến trúc RISC-V và Intel x86 sử dụng quy tắc **Little-Endian** (Đầu nhỏ):

- **Nguyên tắc:** Byte có ý nghĩa thấp nhất (2 chữ số Hexa ngoài cùng bên phải) được lưu ở ngăn nhớ có **địa chỉ nhỏ nhất**. Byte có ý nghĩa cao nhất (2 chữ số Hexa ngoài cùng bên trái) được lưu ở ngăn nhớ có **địa chỉ lớn nhất**.
- **Ví dụ minh họa:** Giả sử lưu Word 32-bit có giá trị `0x12345678` vào địa chỉ `0x10012000`:
  - Địa chỉ `0x10012000` chứa byte `0x78` (byte thấp nhất – đứng ở độ lệch +0).
  - Địa chỉ `0x10012001` chứa byte `0x56` (đứng ở độ lệch +1).
  - Địa chỉ `0x10012002` chứa byte `0x34` (đứng ở độ lệch +2).
  - Địa chỉ `0x10012003` chứa byte `0x12` (byte cao nhất – đứng ở độ lệch +3).

3.1.3 Các nhóm lệnh hợp ngữ RISC-V cốt lõi

1. Nhóm lệnh số học và khởi tạo hằng số

- `add rd, rs1, rs2`: Cộng 2 thanh ghi (rd = rs1 + rs2).
- `sub rd, rs1, rs2`: Trừ 2 thanh ghi (rd = rs1 − rs2).
- `addi rd, rs1, imm`: Cộng thanh ghi với hằng số 12-bit có dấu (rd = rs1 + SignExt(imm)).
- **Lưu ý quan trọng về `addi`:** Vì hằng số `imm` là số nguyên có dấu 12-bit, nên giá trị hợp lệ của `imm` phải nằm trong đoạn $[-\mathbf{2048}, +\mathbf{2047}]$ (tức từ `0xFFFFF800` đến

`0x7FF`). Nếu viết `addi x1, x0, 2048` hoặc `0x800`, phần mềm RARS sẽ báo lỗi biên dịch! Ngoài ra, RISC-V **không có lệnh** `subi`, muốn trừ hằng số ta dùng lệnh `addi` với số âm.

- `lui rd, upimm`: Nạp hằng số 20-bit vào 20 bit cao (từ bit 31 đến bit 12) của thanh ghi `rd`, đồng thời xóa 12 bit thấp (bit 11 đến bit 0) về 0.

Quy tắc nạp hằng số 32-bit bất kỳ vào thanh ghi (Lệnh giả `li`):

Để nạp một số 32-bit dạng `0xABCDE123` vào thanh ghi `t0`, trình dịch tách làm 2 lệnh là `lui` (nạp 5 chữ số Hexa đầu) và `addi` (cộng thêm 3 chữ số Hexa cuối). Tuy nhiên cần đặc biệt chú ý **bẫy mở rộng dấu ở bit thứ 11** (chữ số Hexa thứ 3 từ phải sang):

- **Trường hợp 1:** Chữ số Hexa thứ 3 từ phải sang nhỏ hơn `8` (tức từ `0` đến `7`, bit 11 bằng 0): Giữ nguyên 5 chữ số Hexa đầu.

  Ví dụ: `li t0, 0x20241120` → `lui t0, 0x20241` rồi `addi t0, t0, 0x120`.

- **Trường hợp 2:** Chữ số Hexa thứ 3 từ phải sang từ `8` đến `F` (bit 11 bằng 1, làm cho hằng số 12-bit trong `addi` bị hiểu là số âm): Phải **cộng thêm 1** vào 5 chữ số Hexa đầu trong lệnh `lui` để bù lại phần bị trừ đi!

  Ví dụ: `li s1, 0x12345EDC` → `lui s1, 0x12346` rồi `addi s1, s1, -292` (với `0xEDC` $= -292$).

2. Nhóm lệnh Đọc/Ghi bộ nhớ (Load / Store)

- `lw rd, imm(rs1)` và `sw rs2, imm(rs1)`: Đọc và ghi 1 Word (32-bit = 4 byte). Địa chỉ hiệu dụng rs1 + imm phải chia hết cho 4.

- `lh rd, imm(rs1)` và `lhu rd, imm(rs1)`: Đọc 1 Halfword (16-bit = 2 byte) vào 16 bit thấp của `rd`. Lệnh `lh` mở rộng dấu (Sign-extend: nếu bit 15 là 1 thì điền toàn `FFFF` vào 16 bit cao), còn `lhu` mở rộng không dấu (Zero-extend: luôn điền `0000` vào 16 bit cao). Lệnh ghi tương ứng là `sh rs2, imm(rs1)`.

- `lb rd, imm(rs1)` và `lbu rd, imm(rs1)`: Đọc 1 Byte (8-bit) vào 8 bit thấp của `rd`. Lệnh `lb` mở rộng dấu (nếu byte đọc lên từ `0x80` đến `0xFF` thì 24 bit cao điền toàn `FFFFFF`), còn `lbu` luôn điền `000000` vào 24 bit cao). Lệnh ghi tương ứng là `sb rs2, imm(rs1)`.

3. Nhóm lệnh Logic, Dịch bit và So sánh

- **Phép AND (`and`, `andi`):** 1 AND 1 = 1, còn lại bằng 0. Dùng để xóa một số bit về 0 và giữ nguyên các bit khác.

- **Phép OR (`or`, `ori`):** 0 OR 0 = 0, còn lại bằng 1. Dùng để ép một số bit lên 1.
- **Phép XOR (`xor`, `xori`):** Hai bit khác nhau thì bằng 1, giống nhau thì bằng 0. Dùng để đảo bit (thay cho lệnh NOT bằng cách `xori rd, rs1, -1`) hoặc xóa thanh ghi về 0 (`xor s0, s0, s0`).
- **Lưu ý cực kỳ quan trọng về `andi, ori, xori` trong RISC-V:** Khác với MIPS cũ (mở rộng không dấu cho logic immediate), trong RISC-V, hằng số 12-bit của `andi, ori, xori` **luôn được mở rộng có dấu (Sign-extend)** thành 32-bit trước khi thực hiện phép toán!
- **Dịch trái logic (`sll`, `slli`):** Dịch các bit sang trái k vị trí, chèn 0 vào bên phải. Tương đương nhân với 2^k.
- **Dịch phải logic (`srl`, `srli`):** Dịch các bit sang phải k vị trí, **luôn chèn bit 0** vào bên trái.
- **Dịch phải số học (`sra`, `srai`):** Dịch các bit sang phải k vị trí, nhưng **giữ nguyên bit dấu** (nếu số đang âm, bit 31 là 1 thì chèn toàn bit 1 vào bên trái; nếu dương thì chèn bit 0). Tương đương chia số có dấu cho 2^k.
- **So sánh nhỏ hơn (`slt, slti` và `sltu, sltiu`):** Nếu rs1 $<$ rs2 (hoặc imm) thì thanh ghi đích rd = 1, ngược lại rd = 0. Lệnh `slt`/`slti` so sánh theo số **có dấu**, còn `sltu`/`sltiu` so sánh theo số **không dấu** (khi không dấu, các số bắt đầu bằng bit 1 như `0xFFFFFFF0` là những số dương cực lớn).

4. Nhóm lệnh Rẽ nhánh, Nhảy và Chương trình con

- **Rẽ nhánh có điều kiện (Kiểu B):** `beq` (bằng nhau), `bne` (khác nhau), `blt` (nhỏ hơn có dấu), `bge` (lớn hơn hoặc bằng có dấu), `bltu` (nhỏ hơn không dấu), `bgeu` (lớn hơn hoặc bằng không dấu). Bản chất phần cứng dùng phép **trừ** trong ALU để so sánh 2 thanh ghi.
- **Gọi chương trình con (`jal rd, Label` – Kiểu J):** Thực hiện đồng thời 2 việc:
  1. Cất địa chỉ lệnh kế tiếp vào thanh ghi đích: rd = PC + 4 (thường dùng `rd` là `ra` tức `x1`).
  2. Nhảy tới nhãn `Label`: PC = PC + imm. (Nếu dùng `jal x0, Label` thì trở thành lệnh nhảy không điều kiện `j Label` vì `x0` bỏ qua địa chỉ trả về).
- **Trở về từ chương trình con (`jalr rd, rs1, imm` – Kiểu I):** Cất rd = PC + $4 Haibngtrnbtrnlphildocuhnhct`l`v`c`trongLaTeX khngtØngxungdngkhivnbndihoccnhiuct[cite : 501, 502]$.

Tôi đã sửa lại bằng cách sử dụng định dạng cột cố định độ rộng ‘p...‘ cho bảng thanh ghi[cite: 501] và thu gọn tiêu đề kết hợp cỡ chữ ‘‘ cho bảng 7 cột khuôn dạng lệnh[cite: 502] để cả hai bảng nằm gọn tuyệt đối trong khổ giấy A4.

Dưới đây là toàn bộ tệp **‘C3.tex‘** đã được chỉnh sửa hoàn chỉnh để bạn dán đè lên Overleaf:

“‘latex

CHƯƠNG 4

KIẾN TRÚC TẬP LỆNH RISC-V

4.1 Phần 1: Lý thuyết trọng tâm

4.1.1 Tập 32 thanh ghi của RISC-V 32-bit

Bộ xử lý RISC-V 32-bit có tất cả **32 thanh ghi đa dụng**, mỗi thanh ghi có độ dài 32 bit. Vì $32 = 2^5$, nên trong mã máy, mỗi thanh ghi được mã hóa bằng đúng **5 bit** (từ `00000` ứng với `x0` đến `11111` ứng với `x31`).

Khi lập trình và làm bài thi dịch mã máy, bạn bắt buộc phải tra cứu hoặc thuộc bảng ánh xạ giữa **Tên gợi nhớ** và **Số hiệu thanh ghi** dưới đây:

Tên gọi	**Số hiệu**	**Chức năng sử dụng**	**Bảo toàn**
`zero`	`x0`	Thanh ghi hằng số, luôn bằng 0 (chỉ đọc)	Không áp dụng
`ra`	`x1`	Return Address: chứa địa chỉ trở về hàm	Caller-saved
`sp`	`x2`	Stack Pointer: con trỏ đỉnh ngăn xếp	Callee-saved
`gp`	`x3`	Global Pointer: con trỏ dữ liệu toàn cục	Không áp dụng
`tp`	`x4`	Thread Pointer: con trỏ luồng	Không áp dụng
`t0 - t2`	`x5 - x7`	Temporaries: thanh ghi biến tạm thời	Caller-saved
`s0 / fp`	`x8`	Saved register / Frame pointer: lưu biến	Callee-saved
`s1`	`x9`	Saved register: thanh ghi lưu biến	Callee-saved
`a0 - a1`	`x10 - x11`	Arguments / Return: tham số và kết quả trả về	Caller-saved
`a2 - a7`	`x12 - x17`	Arguments: tham số truyền vào chương trình con	Caller-saved
`s2 - s11`	`x18 - x27`	Saved registers: thanh ghi lưu biến chương trình	Callee-saved
`t3 - t6`	`x28 - x31`	Temporaries: thanh ghi biến tạm thời	Caller-saved

Giải thích quy ước bảo toàn thanh ghi khi gọi hàm (rất hay hỏi trắc nghiệm):

- **Các thanh ghi tạm thời (`t0 - t6`):** Thuộc loại **Caller-saved** (bên gọi hàm phải tự cất vào ngăn xếp nếu muốn giữ lại giá trị sau khi hàm con chạy xong).

– **Các thanh ghi lưu trữ (`s0 - s11`):** Thuộc loại **Callee-saved** (hàm con được gọi nếu muốn dùng các thanh ghi `s0 - s11` thì bắt buộc phải cất giá trị cũ vào ngăn xếp ở đầu hàm và khôi phục lại trước khi thoát hàm).

4.1.2 Tổ chức dữ liệu trong bộ nhớ và Quy tắc Little-Endian

1. Đơn vị dữ liệu trong RISC-V

– **Byte:** 8 bit (chiếm 1 ngăn nhớ). Địa chỉ có thể chẵn hoặc lẻ bất kỳ.

– **Halfword (Nửa từ):** 16 bit = 2 byte. Địa chỉ bắt buộc phải là **số chẵn** (chia hết cho 2, kết thúc bằng `0, 2, 4, 6, 8, A, C, E`).

– **Word (Từ nhớ):** 32 bit = 4 byte. Địa chỉ bắt buộc phải là **bội số của 4** (chia hết cho 4, chữ số Hexa cuối cùng phải là `0, 4, 8, C`). Địa chỉ của một Word nhớ chính là địa chỉ của byte đầu tiên (byte có địa chỉ nhỏ nhất) thuộc Word đó.

2. Quy tắc lưu trữ Đầu nhỏ (Little-Endian)

Kiến trúc RISC-V và Intel x86 sử dụng quy tắc **Little-Endian** (Đầu nhỏ):

– **Nguyên tắc:** Byte có ý nghĩa thấp nhất (2 chữ số Hexa ngoài cùng bên phải) được lưu ở ngăn nhớ có **địa chỉ nhỏ nhất**. Byte có ý nghĩa cao nhất (2 chữ số Hexa ngoài cùng bên trái) được lưu ở ngăn nhớ có **địa chỉ lớn nhất**.

– **Ví dụ minh họa:** Giả sử lưu Word 32-bit có giá trị `0x12345678` vào địa chỉ `0x10012000`:

  * Địa chỉ `0x10012000` chứa byte `0x78` (byte thấp nhất – đứng ở độ lệch +0).
  * Địa chỉ `0x10012001` chứa byte `0x56` (đứng ở độ lệch +1).
  * Địa chỉ `0x10012002` chứa byte `0x34` (đứng ở độ lệch +2).
  * Địa chỉ `0x10012003` chứa byte `0x12` (byte cao nhất – đứng ở độ lệch +3).

4.1.3 Các nhóm lệnh hợp ngữ RISC-V cốt lõi

1. Nhóm lệnh số học và khởi tạo hằng số

– `add rd, rs1, rs2`: Cộng 2 thanh ghi (rd = rs1 + rs2).

– `sub rd, rs1, rs2`: Trừ 2 thanh ghi (rd = rs1 − rs2).

– `addi rd, rs1, imm`: Cộng thanh ghi với hằng số 12-bit có dấu (rd = rs1 + SignExt(imm)).

– **Lưu ý quan trọng về `addi`:** Vì hằng số `imm` là số nguyên có dấu 12-bit, nên giá trị hợp lệ của `imm` phải nằm trong đoạn $[-\mathbf{2048}, +\mathbf{2047}]$ (tức từ `0xFFFFF800` đến `0x7FF`). Nếu viết `addi x1, x0, 2048` hoặc `0x800`, phần mềm RARS sẽ báo lỗi biên dịch! Ngoài ra, RISC-V **không có lệnh `subi`**, muốn trừ hằng số ta dùng lệnh `addi` với số âm.

– `lui rd, upimm`: Nạp hằng số 20-bit vào 20 bit cao (từ bit 31 đến bit 12) của thanh ghi `rd`, đồng thời xóa 12 bit thấp (bit 11 đến bit 0) về 0.

Quy tắc nạp hằng số 32-bit bất kỳ vào thanh ghi (Lệnh giả `li`):

Để nạp một số 32-bit dạng `0xABCDE123` vào thanh ghi `t0`, trình dịch tách làm 2 lệnh là `lui` (nạp 5 chữ số Hexa đầu) và `addi` (cộng thêm 3 chữ số Hexa cuối). Tuy nhiên cần đặc biệt chú ý **bẫy mở rộng dấu ở bit thứ 11** (chữ số Hexa thứ 3 từ phải sang):

– **Trường hợp 1:** Chữ số Hexa thứ 3 từ phải sang nhỏ hơn `8` (tức từ `0` đến `7`, bit 11 bằng 0): Giữ nguyên 5 chữ số Hexa đầu.

Ví dụ: `li t0, 0x20241120` → `lui t0, 0x20241` rồi `addi t0, t0, 0x120`.

– **Trường hợp 2:** Chữ số Hexa thứ 3 từ phải sang từ `8` đến `F` (bit 11 bằng 1, làm cho hằng số 12-bit trong `addi` bị hiểu là số âm): Phải **cộng thêm 1** vào 5 chữ số Hexa đầu trong lệnh `lui` để bù lại phần bị trừ đi!

Ví dụ: `li s1, 0x12345EDC` → `lui s1, 0x12346` rồi `addi s1, s1, -292` (với `0xEDC` $= -292$).

2. Nhóm lệnh Đọc/Ghi bộ nhớ (Load / Store)

– `lw rd, imm(rs1)` và `sw rs2, imm(rs1)`: Đọc và ghi 1 Word (32-bit = 4 byte). Địa chỉ hiệu dụng rs1 + imm phải chia hết cho 4.

– `lh rd, imm(rs1)` và `lhu rd, imm(rs1)`: Đọc 1 Halfword (16-bit = 2 byte) vào 16 bit thấp của `rd`. Lệnh `lh` mở rộng dấu (Sign-extend: nếu bit 15 là 1 thì điền toàn `FFFF` vào 16 bit cao), còn `lhu` mở rộng không dấu (Zero-extend: luôn điền `0000` vào 16 bit cao). Lệnh ghi tương ứng là `sh rs2, imm(rs1)`.

– `lb rd, imm(rs1)` và `lbu rd, imm(rs1)`: Đọc 1 Byte (8-bit) vào 8 bit thấp của `rd`. Lệnh `lb` mở rộng dấu (nếu byte đọc lên từ `0x80` đến `0xFF` thì 24 bit cao điền toàn `FFFFFF`), còn `lbu` luôn điền `000000` vào 24 bit cao. Lệnh ghi tương ứng là `sb rs2, imm(rs1)`.

3. Nhóm lệnh Logic, Dịch bit và So sánh

– **Phép AND (`and`, `andi`):** 1 AND 1 = 1, còn lại bằng 0. Dùng để xóa một số bit về 0 và giữ nguyên các bit khác.

– **Phép OR (`or`, `ori`):** 0 OR 0 = 0, còn lại bằng 1. Dùng để ép một số bit lên 1.

– **Phép XOR (`xor`, `xori`):** Hai bit khác nhau thì bằng 1, giống nhau thì bằng 0. Dùng để đảo bit (thay cho lệnh NOT bằng cách `xori rd, rs1, -1`) hoặc xóa thanh ghi về 0 (`xor s0, s0, s0`).

– **Lưu ý cực kỳ quan trọng về `andi, ori, xori` trong RISC-V:** Khác với MIPS cũ (mở rộng không dấu cho logic immediate), trong RISC-V, hằng số 12-bit của `andi, ori, xori` **luôn được mở rộng có dấu (Sign-extend)** thành 32-bit trước khi thực hiện phép toán!

- **Dịch trái logic (`sll`, `slli`):** Dịch các bit sang trái k vị trí, chèn 0 vào bên phải. Tương đương nhân với 2^k.
- **Dịch phải logic (`srl`, `srli`):** Dịch các bit sang phải k vị trí, **luôn chèn bit 0** vào bên trái.
- **Dịch phải số học (`sra`, `srai`):** Dịch các bit sang phải k vị trí, nhưng **giữ nguyên bit dấu** (nếu số đang âm, bit 31 là 1 thì chèn toàn bit 1 vào bên trái; nếu dương thì chèn bit 0). Tương đương chia số có dấu cho 2^k.
- **So sánh nhỏ hơn (`slt`, `slti` và `sltu`, `sltiu`):** Nếu rs1 $<$ rs2 (hoặc imm) thì thanh ghi đích rd $= 1$, ngược lại rd $= 0$. Lệnh `slt`/`slti` so sánh theo số **có dấu**, còn `sltu`/`sltiu` so sánh theo số **không dấu** (khi không dấu, các số bắt đầu bằng bit 1 như `0xFFFFFFF0` là những số dương cực lớn).

4. Nhóm lệnh Rẽ nhánh, Nhảy và Chương trình con

- **Rẽ nhánh có điều kiện (Kiểu B):** `beq` (bằng nhau), `bne` (khác nhau), `blt` (nhỏ hơn có dấu), `bge` (lớn hơn hoặc bằng có dấu), `bltu` (nhỏ hơn không dấu), `bgeu` (lớn hơn hoặc bằng không dấu). Bản chất phần cứng dùng phép **trừ** trong ALU để so sánh 2 thanh ghi.
- **Gọi chương trình con (`jal rd, Label` – Kiểu J):** Thực hiện đồng thời 2 việc:
  1. Cất địa chỉ lệnh kế tiếp vào thanh ghi đích: rd $=$ PC $+ 4$ (thường dùng `rd` là `ra` tức `x1`).
  2. Nhảy tới nhãn `Label`: PC $=$ PC $+$ imm. (Nếu dùng `jal x0, Label` thì trở thành lệnh nhảy không điều kiện `j Label` vì `x0` bỏ qua địa chỉ trả về).
- **Trở về từ chương trình con (`jalr rd, rs1, imm` – Kiểu I):** Cất rd $=$ PC $+ 4$ và nhảy tới PC $=$ rs1 $+$ imm. Lệnh giả `jr ra` (trở về hàm) được dịch thành `jalr x0, ra, 0`.
- **Thao tác với Ngăn xếp (Stack):** Ngăn xếp của RISC-V phát triển theo chiều **địa chỉ giảm dần**.
  - Khi **cất (push)** k thanh ghi (mỗi thanh ghi 4 byte) vào ngăn xếp: Phải **trừ** `sp` trước (`addi sp, sp, -4*k`), rồi dùng `sw` lưu từng thanh ghi.
  - Khi **khôi phục (pop)** k thanh ghi từ ngăn xếp: Dùng `lw` đọc từng thanh ghi ra trước, sau đó mới **cộng** trả lại `sp` (`addi sp, sp, 4*k`).

4.1.4 Mã máy 32-bit của RISC-V và 4 Phương thức định địa chỉ

1. Sáu khuôn dạng lệnh máy 32-bit của RISC-V

Mọi lệnh chuẩn RV32I đều dài 32 bit (4 byte). 7 bit cuối cùng (`bit 6:0`) luôn là trường **opcode**.

Kiểu	31:25 (7b)	24:20 (5b)	19:15 (5b)	14:12 (3b)	11:7 (5b)	6:0 (7b)
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	imm[11:0] (12 bit)		rs1	funct3	rd	opcode
S-type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B-type	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode
U-type	imm (20 bit)				rd	opcode
J-type	imm[20,10:1,11,19:12] (20 bit)				rd	opcode

Bảng tra nhanh Opcode và Funct3/Funct7 của các lệnh thường thi nhất:

- **Kiểu R** (opcode = 0110011): add (funct3=000, funct7=0000000), sub (funct3=000, funct7=0100000), sll (001), slt (010), sltu (011), xor (100), srl (101, 0000000), sra (101, 0100000), or (110), and (111).
- **Kiểu I – Tính toán** (opcode = 0010011): addi (funct3=000), slli (001), slti (010), xori (100), srli/srai (101), ori (110), andi (111).
- **Kiểu I – Đọc bộ nhớ Load** (opcode = 0000011): lb (funct3=000), lh (001), lw (010), lbu (100), lhu (101).
- **Kiểu S – Ghi bộ nhớ Store** (opcode = 0100011): sb (funct3=000), sh (001), sw (010).
- **Kiểu B – Rẽ nhánh** (opcode = 1100011): beq (funct3=000), bne (001), blt (100), bge (101), bltu (110), bgeu (111).
- **Kiểu U & J:** lui (opcode = 0110111), auipc (0010111), jal (1101111), jalr (Kiểu I, 1100111).

2. Công thức tính địa chỉ đích rẽ nhánh (BTA) và nhảy (JTA) trong RISC-V

Khác với kiến trúc MIPS cũ (tính độ lệch từ PC + 4), trong **RISC-V**, mọi độ lệch rẽ nhánh và nhảy đều tính trực tiếp từ **địa chỉ của chính câu lệnh đó (PC)**:

$$\text{Địa chỉ đích (BTA hoặc JTA)} = \text{PC}_{\text{hiện tại}} + \text{imm} \implies \text{imm} = \text{Địa chỉ đích} - \text{PC}_{\text{hiện tại}}$$

Lưu ý: Vì các lệnh luôn nằm ở địa chỉ chẵn, nên bit số 0 (imm[0]) luôn bằng 0 và **không được lưu trong mã máy**. Mã máy kiểu B chỉ lưu từ imm[12] đến imm[1], mã máy kiểu J lưu từ imm[20] đến imm[1].

3. Bốn phương thức định địa chỉ trong RISC-V

1. **Định địa chỉ thanh ghi (Register Addressing):** Toán hạng nằm hoàn toàn trong các thanh ghi (dùng cho các lệnh kiểu R như add, sub, and, or...).
2. **Định địa chỉ tức thì (Immediate Addressing):** Toán hạng là hằng số imm nằm ngay trong câu lệnh (dùng cho toán hạng hằng số của addi, ori, andi, lui...).

3. **Định địa chỉ cơ sở (Base Addressing):** Địa chỉ của toán hạng bộ nhớ bằng nội dung thanh ghi cơ sở cộng với hằng số độ lệch (Address = rs1 + imm, dùng cho các lệnh Load/Store như `lw, sw, lb, lh...`).
4. **Định địa chỉ tương đối với PC (PC-relative Addressing):** Địa chỉ đích bằng thanh ghi `PC` cộng với hằng số độ lệch (dùng cho các lệnh rẽ nhánh kiểu B như `beq, bne, bge` và lệnh nhảy `jal`).

4.2 Phần 2: Toàn bộ bài tập Chương 3 và Hướng dẫn giải

4.2.1 Nhóm 1: Thanh ghi, Phương thức định địa chỉ và Kiểm tra hợp lệ

Bài tập 4.1 (Đề Cuối kỳ 2023.1 - #08de08 & Ôn tập Cuối kỳ P2 - Câu 13)

Phát biểu nào sau đây là đúng về các thanh ghi tạm thời (`t0 - t6`) và các thanh ghi lưu trữ (`s0 - s11`) trong RISC-V?

A. Trong một số quy ước gọi hàm, thanh ghi tạm thời là "callee-saved", còn thanh ghi lưu trữ là "caller-saved".

B. Trong một số quy ước gọi hàm, thanh ghi tạm thời là "caller-saved", còn thanh ghi lưu trữ là "callee-saved".

C. Giá trị của các thanh ghi lưu trữ luôn phải được lưu bởi bên được gọi (callee).

D. Giá trị của các thanh ghi tạm thời luôn phải được lưu bởi bên gọi (caller).

Hướng dẫn giải:

- Theo quy ước gọi hàm chuẩn của RISC-V: các thanh ghi tạm thời (`t0 - t6`) là **caller-saved** (bên gọi chỉ lưu khi thực sự cần dùng tiếp sau lời gọi hàm, không phải "luôn luôn" phải lưu), còn các thanh ghi lưu trữ (`s0 - s11`) là **callee-saved** (bên được gọi chỉ lưu nếu nó có sửa đổi các thanh ghi này).
- **Đáp án đúng: B.**

Bài tập 4.2 (Đề Giữa kỳ 2022.2 - Đề 0008 Câu 10 & Đề 0006 Câu 5)

a) Trong 32 thanh ghi được đánh địa chỉ của bộ xử lý 32-bit, tên của 1 thanh ghi chỉ đọc (read-only) luôn chứa hằng số 0 là gì?

b) Trong khuôn dạng lệnh R của bộ xử lý 32-bit, tại sao độ rộng của vùng thông tin thanh ghi đích `rd` phải là 5 bit?

Hướng dẫn giải:

- **Ý a):** Thanh ghi chỉ đọc luôn có giá trị bằng 0 là thanh ghi `zero` (hoặc `x0`). Mọi thao tác ghi vào `x0` đều không làm thay đổi giá trị của nó.
- **Ý b):** Vì bộ xử lý có tất cả **32 thanh ghi** đa dụng. Để mã hóa và phân biệt được 32 thanh ghi khác nhau, ta cần số bit là $\log_2(32) = \mathbf{5}$ bit (vì $2^5 = 32$).

Bài tập 4.3 (Đề Giữa kỳ 2022.2 - Đề 0006, Câu 8)

Một phần mềm viết cho bộ xử lý RISC-V 32-bit có 150 lệnh hợp ngữ chuẩn (không dùng lệnh giả và không dùng lệnh nén) thì chiếm bao nhiêu byte lệnh mã máy trong bộ nhớ?

Hướng dẫn giải:

- Mỗi lệnh mã máy chuẩn của kiến trúc 32-bit có độ dài cố định là 32 bit = 4 byte.
- Vậy 150 lệnh sẽ chiếm dung lượng là: 150×4 byte = **600** byte.
- **Đáp số:** 600 **byte.**

Bài tập 4.4 (Đề Cuối kỳ 2022.2 - #17e1a7, 2024.1 - #366b03 & Ôn tập Cuối kỳ P2 - Câu 29)

a) Phương pháp định địa chỉ nào được sử dụng mã hóa cho toán hạng hằng số của lệnh hợp ngữ RISC-V sau: `addi x18, x19, -5`?

b) Phương pháp định địa chỉ nào được sử dụng để xác định địa chỉ đích của lệnh rẽ nhánh `beq` hoặc `jal`?

- A. Định địa chỉ cơ sở B. Định địa chỉ tương đối với PC C. Định địa chỉ tức thì D. Định địa chỉ thanh ghi

Hướng dẫn giải:

- **Ý a):** Toán hạng hằng số `-5` nằm trực tiếp ngay trong mã lệnh của lệnh `addi`, nên sử dụng phương pháp **Định địa chỉ tức thì (Immediate Addressing)** → Chọn **C**.
- **Ý b):** Các lệnh rẽ nhánh (`beq, bne...`) và lệnh nhảy (`jal`) tính địa chỉ đích bằng cách lấy PC + imm, nên sử dụng phương pháp **Định địa chỉ tương đối với PC (PC-relative Addressing)** → Chọn **B**.

Bài tập 4.5 (Đề Cuối kỳ 2020.2 & Cuối kỳ 2022.2 - #3ace1f)

Trong các giá trị địa chỉ được cho dưới đây, giá trị nào có thể là địa chỉ của một **word nhớ** trong bộ nhớ chính của máy tính 32-bit?

- **Câu A (Đề 2022.2):** 1. `0x00404508` 2. `0x0050601A` 3. `0x00601002` 4. `0x00402346`
- **Câu B (Đề 2020.2):** 1. `0x1001A00C` 2. `0x10800016` 3. `0x100020CE` 4. `0x1050009A`

Hướng dẫn giải:

- **Quy tắc:** Mỗi word nhớ 32-bit chiếm 4 byte, nên địa chỉ của word nhớ bắt buộc phải **chia hết cho 4**.
- Một số Hexa chia hết cho 4 khi và chỉ khi chữ số Hexa tận cùng bên phải của nó chia hết cho 4 (tức tận cùng phải là `0, 4, 8` hoặc `C` (= 12)). Các chữ số tận cùng là `2, 6, A` (= 10), `E` (= 14) đều là số chẵn nhưng không chia hết cho 4!
- **Xét Câu A:** Chỉ có `0x00404508` tận cùng bằng `8` (chia hết cho 4) → Chọn **1** (`0x00404508`).
- **Xét Câu B:** Chỉ có `0x1001A00C` tận cùng bằng `C` ($12 \div 4 = 3$) → Chọn **1** (`0x1001A00C`).

Bài tập 4.6 (Đề Cuối kỳ 2023.1 - Trang 15 & Ôn tập Giữa kỳ - Câu 18)

Toán hạng đích trong lệnh `sw` của đoạn chương trình RISC-V dưới đây có hợp lệ không?

```
lui  s0, 0x10010
addi t0, zero, 100
sw   t0, 2(s0)
```

A. Có, hoàn toàn hợp lệ (Yes, it is).

B. Có, nhưng đoạn mã này cần quyền đặc biệt để thực thi.

C. Không, vì nó không có địa chỉ hợp lệ (No, it does not have a valid address).

D. Không, vì nó không nằm trong phân đoạn dữ liệu hợp lệ.

Hướng dẫn giải:

- Lệnh `lui s0, 0x10010` nạp giá trị `0x10010000` vào thanh ghi `s0`.
- Lệnh `sw t0, 2(s0)` yêu cầu lưu một **word 32-bit** (4 byte) vào địa chỉ `0x10010000` + 2 = `0x10010002`.
- Vì địa chỉ `0x10010002` tận cùng là `2` (**không chia hết cho 4** – vi phạm quy tắc căn chỉnh biên từ nhớ *word alignment*), nên địa chỉ này không hợp lệ đối với lệnh `sw`.
- **Đáp án đúng: C (No, it does not have a valid address).**

Bài tập 4.7 (Ôn tập Giữa kỳ - Câu 11)

Phát biểu nào là đúng về thứ tự lưu trữ các byte trong bộ nhớ chính?

A. Intel x86 dùng big-endian

B. SunSPARC dùng middle-endian

C. ARM dùng little-endian hoặc big-endian tùy phiên bản

D. RISC-V dùng big-endian

Hướng dẫn giải:

- Theo Slide trang 16 (Chương 3): Intel x86 và RISC-V dùng **little-endian**; Motorola 680x0 và SunSPARC dùng **big-endian**; còn kiến trúc **ARM dùng little-endian hoặc big-endian (Bi-endian) tùy phiên bản/cấu hình**.
- **Đáp án đúng: C.**

Bài tập 4.8 (Ôn tập Giữa kỳ - Câu 62)

(Những) câu lệnh nào sau đây bị lỗi biên dịch trong phần mềm mô phỏng RARS? (Chọn nhiều đáp án)

1. `lui x1, 0x1  addi x1, x0, 0xFFFFF800`
2. `lui x1, 0x1  addi x1, x0, -2048`
3. `li x1, 0x800`
4. `lui x1, 0x1  addi x1, x0, 2048`
5. `lui x1, 0x1  addi x1, x0, 0xFFFF800`
6. `lui x1, 0x1  addi x1, x0, 0x800`
7. `addi x1, x0, 0x800`

Hướng dẫn giải:

- **Quy tắc kiểm tra lỗi biên dịch lệnh `addi`:** Toán hạng hằng số của lệnh `addi` là số nguyên có dấu 12-bit, nên chỉ nhận giá trị từ $-$**2048** đến $+$**2047** (viết ở hệ Hexa 32-bit mở rộng dấu là từ `0xFFFFF800` đến `0x000007FF`).
- Kiểm tra từng lệnh:
  - Lệnh 1 (`0xFFFFF800` $= -2048$) và Lệnh 2 (-2048): Hợp lệ.
  - Lệnh 3 (`li x1, 0x800`): Hợp lệ vì `li` là lệnh giả, trình dịch tự động chuyển thành `lui` + `addi`.
  - **Lệnh 4** (`2048`), **Lệnh 6** (`0x800` $= +2048$), và **Lệnh 7** (`0x800` $= +2048$): Đều vượt quá giới hạn $+2047$ của lệnh `addi` $\rightarrow$ **Bị lỗi biên dịch**.
  - **Lệnh 5** (`0xFFFF800` – chỉ có 4 chữ F ở đầu): Là số dương `0x0FFFF800` $= 268\,433\,408 > 2047 \rightarrow$ **Bị lỗi biên dịch**.
- **Đáp án đúng: Chọn các ý 4, 5, 6 và 7.**

4.2.2 Nhóm 2: Thực thi lệnh Số học, Logic, Dịch bit và Khởi tạo thanh ghi

Bài tập 4.9 (Đề Giữa kỳ 2024.1 - #4610c2 & Ôn tập Giữa kỳ - Câu 10)

Những lệnh nào sau đây có thể dùng để xóa nội dung của thanh ghi `s0` về 0 (clear the content of `s0`)?

A. `AND s0, s0, zero`

B. `XOR s0, s0, s0`

C. `XOR s0, s0, zero`

D. `AND s0, s0, s0`

Hướng dẫn giải:

- Xét từng lệnh:
  - `AND s0, s0, zero`: Bất kỳ bit nào AND với 0 đều bằng 0 $\implies$ `s0` $= 0$ (**Đúng**).
  - `XOR s0, s0, s0`: Hai bit giống hệt nhau khi XOR với nhau đều bằng 0 $\implies$ `s0` $= 0$ (**Đúng**).
  - `XOR s0, s0, zero` và `AND s0, s0, s0`: Đều giữ nguyên giá trị cũ của `s0` (**Sai**).
- **Đáp án đúng: Chọn cả A và B.**

Bài tập 4.10 (Đề Giữa kỳ 2024.1 - #99ef6 & Ôn tập Giữa kỳ - Câu 25)

Đoạn lệnh nào sau đây tương đương với lệnh giả `li t0, 0x20241120` trong RISC-V?

A.
```
lui t0, 0x202411
ori t0, t0, 0x20
```

B.
```
lui t0, 0x2024
addi t0, t0, 0x1120
```

C.
```
lui t0, 0x2024
ori t0, t0, 0x1120
```

D.
```
lui t0, 0x20241
addi t0, t0, 0x120
```

Hướng dẫn giải:

- Hằng số 32-bit `0x20241120` được tách thành 20 bit cao (5 chữ số Hexa đầu là `0x20241`) và 12 bit thấp (3 chữ số Hexa cuối là `0x120`).
- Vì chữ số Hexa đầu của phần 12 bit thấp là `1` (< 8, số dương) nên không cần cộng 1 vào phần cao.
- Ta dùng `lui t0, 0x20241` (được `0x20241000`) rồi cộng 12 bit thấp bằng `addi t0, t0, 0x120`.

– **Đáp án đúng: D.**

Bài tập 4.11 (Đề Giữa kỳ 2024.1 - #e10492)

Chọn đáp án đúng để thực hiện câu lệnh C sau trong RISC-V: `a += 0x12345678;` (giả sử biến `a` được lưu trong thanh ghi `s0`):

A. `li t0, 0x12345` → `addi t0, t0, 0x678` → `add s0, s0, t0`

B. `lui t0, 0x1234` → `addi t0, t0, 0x5678` → `add s0, s0, t0`

C. `lui t0, 0x12345` → `addi t0, t0, 0x678` → `add s0, zero, t0`

D. `lui t0, 0x12345` → `addi t0, t0, 0x678` → `add s0, s0, t0`

Hướng dẫn giải:

– Bước 1: Nạp hằng số `0x12345678` vào thanh ghi tạm `t0` bằng cặp lệnh `lui t0, 0x12345` (nạp 20 bit cao) và `addi t0, t0, 0x678` (cộng 12 bit thấp).

– Bước 2: Cộng dồn vào `s0` (`a = a + t0`) bằng lệnh `add s0, s0, t0`. (Phương án C sai vì cộng với `zero` làm mất giá trị cũ của `a`).

– **Đáp án đúng: D.**

Bài tập 4.12 (Đề Giữa kỳ 2024.1 - #1b55cf)

Cho đoạn mã hợp ngữ RISC-V dưới đây. Hãy cho biết giá trị của thanh ghi `s2` ở dạng số Hexa sau khi đoạn mã thực hiện xong:

```
li  s0, 1
lui s1, 0xFFFFF
ori s1, s1, 2047
add s2, s0, s1
```

Hướng dẫn giải:

– Lệnh 1 (`li s0, 1`): `s0` = `0x00000001`.

– Lệnh 2 (`lui s1, 0xFFFFF`): Nạp 5 chữ F vào 20 bit cao, 12 bit thấp bằng 0 $\implies$ `s1` = `0xFFFFF000`.

– Lệnh 3 (`ori s1, s1, 2047`): Đổi 2047 sang hệ Hexa: $2047 = 2048 - 1 =$ `0x7FF` (gồm bit 11 là 0 và 11 bit 1, nên mở rộng dấu 32-bit là `0x000007FF`). Thực hiện phép OR:

$$\texttt{s1} = \texttt{0xFFFFF000} \mid \texttt{0x000007FF} = \texttt{0xFFFFF7FF}$$

– Lệnh 4 (`add s2, s0, s1`): Cộng thêm 1 vào `s1`:

$$\texttt{s2} = \texttt{0xFFFFF7FF} + 1 = \texttt{0xFFFFF800} \quad (\text{vì } \texttt{7FF} + 1 = \texttt{800})$$

– **Đáp số:** `0xFFFFF800`.

Bài tập 4.13 (Đề Cuối kỳ 2024.1 - #d9f3db & Ôn tập Giữa kỳ - Câu 23)

Cho biết nội dung thanh ghi `s1` viết theo số Hexa sau khi thực hiện đoạn chương trình hợp ngữ RISC-V dưới đây:

```
lui s0, 0x11223
ori s0, s0, 0x4AB
ori s1, s0, 0xFF
```

Hướng dẫn giải:

– Lệnh 1 (`lui s0, 0x11223`): Đưa `0x11223` vào 20 bit cao $\implies$ `s0` = `0x11223000`.

– Lệnh 2 (`ori s0, s0, 0x4AB`): Hằng số `0x4AB` có bit 11 bằng 0 (vì $4 < 8$), mở rộng 32-bit là `0x000004AB`. Thực hiện phép OR:

$$\texttt{s0} = \texttt{0x11223000} \mid \texttt{0x000004AB} = \texttt{0x112234AB}$$

– Lệnh 3 (`ori s1, s0, 0xFF`): Hằng số `0x0FF` có bit 11 bằng 0, mở rộng 32-bit là `0x000000FF`. Phép OR với `0x000000FF` sẽ giữ nguyên 6 chữ số Hexa đầu và bật 2 chữ số Hexa cuối thành `FF`:

$$\texttt{s1} = \texttt{0x112234AB} \mid \texttt{0x000000FF} = \texttt{0x112234FF}$$

– **Đáp số:** `0x112234FF`.

Bài tập 4.14 (Đề Cuối kỳ 2022.2 - #585834 & Giữa kỳ 2023.2 - Đề 2 Câu 1)

a) (Đề 2022.2): Giả sử thanh ghi `s0` đang có giá trị `0x223389CD`. Thực hiện lệnh `andi s1, s0, 0xFF`. Hãy cho biết nội dung thanh ghi `s1` viết theo số Hexa (8 chữ số)?

b) (Đề 2023.2): Thực hiện đoạn lệnh RISC-V sau:

```
lui  s0, 0x4567C
addi s0, s0, -802     # s0 = 0x4567BCDE
addi s1, zero, -16
and  s2, s0, s1
```

Hãy cho biết nội dung thanh ghi `s1` và `s2` dưới dạng số Hexa?

Hướng dẫn giải:

- **Giải ý a):** Hằng số `0x0FF` mở rộng thành `0x000000FF`. Phép `AND` với `0x000000FF` sẽ xóa 6 chữ số Hexa bên trái về `0` và chỉ giữ lại 2 chữ số Hexa cuối cùng:

$$\texttt{s1} = \texttt{0x223389CD} \ \& \ \texttt{0x000000FF} = \texttt{0x000000CD}$$

  (Lưu ý: Thí sinh làm bài trong ảnh chụp đề 2022.2 đã nhầm phép `andi` thành `ori` nên điền sai thành `0x223389FF`).

- **Giải ý b):**
  - Thanh ghi `s1` nhận giá trị -16. Đổi -16 sang mã bù 2 32-bit: $+16$ = `0x00000010` $\Longrightarrow$ bù 2 là `s1 = 0xFFFFFFF0`.
  - Thực hiện `and s2, s0, s1`: Phép AND với `0xFFFFFFF0` giữ nguyên 7 chữ số Hexa đầu của `s0` và xóa chữ số Hexa cuối cùng về `0`:

$$\texttt{s2} = \texttt{0x4567BCDE} \ \& \ \texttt{0xFFFFFFF0} = \texttt{0x4567BCD0}$$

Bài tập 4.15 (Đề Giữa kỳ 2024.1 - #5e7591 & Ôn tập Giữa kỳ - Câu 16)

Giá trị thập phân của thanh ghi `s4` sau khi thực hiện các lệnh dưới đây là bao nhiêu?

```
addi s1, zero, 87
addi s2, zero, 53
xor  s3, s2, zero
addi s3, s3, 1
add  s4, s1, s3
```

Hướng dẫn giải:

- Lệnh 1: `s1` = 87.
- Lệnh 2: `s2` = 53.
- Lệnh 3 (`xor s3, s2, zero`): Bất kỳ số nào XOR với 0 đều bằng chính nó $\Longrightarrow$ `s3` = 53.
- Lệnh 4 (`addi s3, s3, 1`): `s3` = 53 + 1 = 54.
- Lệnh 5 (`add s4, s1, s3`): `s4` = 87 + 54 = **141**.
- **Đáp số: 141.**

Bài tập 4.16 (Đề Cuối kỳ 2022.2 - #7a3120, 2024.1 - #25c9d6 & Ôn tập Giữa kỳ - Câu 52)

Cho đoạn chương trình hợp ngữ RISC-V sau:

```
addi t0, zero, 6
addi t1, zero, 12
and  t2, t0, t1
or   t3, t0, t1
xor  t4, t0, t1
```

Giá trị của `t2, t3, t4` là gì sau khi thực hiện đoạn chương trình này?

A. $\texttt{t2} = 4, \texttt{t3} = 14, \texttt{t4} = 10$

B. $\texttt{t2} = 4, \texttt{t3} = 14, \texttt{t4} = 5$

C. $\texttt{t2} = 14, \texttt{t3} = 4, \texttt{t4} = 10$

D. $\texttt{t2} = 14, \texttt{t3} = 4, \texttt{t4} = 5$

Hướng dẫn giải:

- Đổi `t0` $= 6$ và `t1` $= 12$ sang nhị phân 4-bit:

$$\texttt{t0} = 6 = \texttt{0110}_{(2)}; \quad \texttt{t1} = 12 = \texttt{1100}_{(2)}$$

- Tính từng phép toán theo từng cột bit:
  - `t2` $=$ `0110` AND `1100` $= \texttt{0100}_{(2)} = \mathbf{4}$.
  - `t3` $=$ `0110` OR `1100` $= \texttt{1110}_{(2)} = 8 + 4 + 2 = \mathbf{14}$.
  - `t4` $=$ `0110` XOR `1100` $= \texttt{1010}_{(2)} = 8 + 2 = \mathbf{10}$.
- **Đáp án đúng: A (**$\texttt{t2} = 4, \texttt{t3} = 14, \texttt{t4} = 10$**).**

Bài tập 4.17 (Đề Cuối kỳ 2022.2 - #5070f9 & Ôn tập Cuối kỳ P2 - Câu 8)

Cho nội dung các thanh ghi: `t0` (`x5`) = `0x5A6B7C8D`, `t1` (`x6`) = `0x000FFF00`. Hãy viết theo số Hexa nội dung các thanh ghi `t2` (`x7`) và `t3` (`x28`) sau khi thực hiện đoạn lệnh hợp ngữ RISC-V dưới đây:

```
xor x7,  x5, x6
xor x28, x7, x6
```

Hướng dẫn giải:

- **Quy tắc cực nhanh cho phép XOR với mặt nạ `0` và `F`:**
  - Chữ số Hexa nào XOR với `0` thì **giữ nguyên**.
  - Chữ số Hexa nào XOR với `F` $(= 15)$ thì lấy 15 trừ đi chữ số đó (đảo bit).
- **Tính `x7` (`t2`):** Mặt nạ `x6 = 0x000FFF00` có 3 chữ `F` ở giữa ứng với 3 chữ số `B, 7, C` của `x5`.
  - Giữ nguyên 3 chữ đầu (`5A6`) và 2 chữ cuối (`8D`).

  - Đảo 3 chữ giữa: `B` (11) $\rightarrow 15-11=$ **4**; `7` $\rightarrow 15-7=$ **8**; `C` (12) $\rightarrow 15-12=$ **3**.
  - Suy ra: `x7 (t2)` = `0x5A64838D`.
- **Tính `x28 (t3)`:** Lấy kết quả `x7` đem XOR một lần nữa với chính `x6` sẽ đảo ngược lại về giá trị ban đầu của `x5`:

$$\texttt{x28 (t3)} = (\texttt{x5} \oplus \texttt{x6}) \oplus \texttt{x6} = \texttt{x5} = \texttt{0x5A6B7C8D}$$

Bài tập 4.18 (Đề Cuối kỳ 2020.2 & Giữa kỳ 2023.2 - Đề 2 Câu 4)

a) (Đề 2020.2): Cho `s0` = `0x1234ABCD` và `s1` = `0xFFFF0000`. Tìm `s2` sau lệnh `xor s2, s0, s1`.

b) (Đề 2023.2): Cho nội dung thanh ghi `t0` = `0xA1B2C3D4`, hãy viết một lệnh logic hợp ngữ để biến đổi nội dung `t0` thành `0x286CB0F5` (giả sử thanh ghi hằng số mặt nạ đã có sẵn trong `t1`).

Hướng dẫn giải:

- **Giải ý a):** XOR với `0xFFFF0000` sẽ đảo 4 chữ số Hexa đầu (`1234`) và giữ nguyên 4 chữ số Hexa cuối (`ABCD`):
  - $15-1=14$ (`E`); $15-2=13$ (`D`); $15-3=12$ (`C`); $15-4=11$ (`B`).
  - Kết quả: `s2` = `0xEDCBABCD`.
- **Giải ý b):** Muốn biến số A thành số B bất kỳ chỉ bằng 1 phép logic, ta dùng phép **XOR** với mặt nạ $M = A \oplus B$:

$$M = \texttt{0xA1B2C3D4} \oplus \texttt{0x286CB0F5} = \texttt{0x89DE7321}$$

  Với `t1` = `0x89DE7321`, lệnh cần viết là: `xor t0, t0, t1`.

Bài tập 4.19 (Đề Cuối kỳ 2024.1 - #2a8241, 2022.2 - Trang 14 & Ôn tập Giữa kỳ - Câu 39)

Cho đoạn chương trình hợp ngữ RISC-V sau:

```
addi t0, x0, 0x1234
srli t1, t0, 3
```

Giá trị của `t1` bằng bao nhiêu sau khi thực hiện đoạn chương trình này?

A. `0x000091A0`

B. `0x01234000`

C. `0x00000001`

D. `0x00000246`

Hướng dẫn giải:

- (Lưu ý: Hằng số trong bài nếu dùng `addi` thực tế nhỏ hơn `0x800`, hoặc khởi tạo qua lệnh giả `li t0, 0x1234`).
- Đổi `t0` = `0x00001234` sang nhị phân:

$$\texttt{0x1234} = \texttt{0001 0010 0011 0100}_{(2)}$$

- Lệnh `srli t1, t0, 3` dịch **phải** logic 3 bit (bỏ đi 3 bit cuối `100` ở bên phải và thêm 3 bit `000` vào bên trái):

$$\texttt{t1} = \texttt{0000 0010 0100 0110}_{(2)} = \texttt{0x00000246}$$

  (Hoặc bấm máy tính nhẩm nhanh: Dịch phải 3 bit là chia lấy nguyên cho $2^3 = 8$: $\texttt{0x1234} = 4660 \implies 4660 \div 8 = 582 = \texttt{0x246}$).
- **Đáp án đúng: D (`0x00000246`).**

Bài tập 4.20 (Đề Cuối kỳ 2022.2 - #75a1b2, 2024.1 - #b68d32 & Ôn tập Giữa kỳ - Câu 30)

Hãy viết theo số Hexa (8 chữ số) nội dung thanh ghi `t1` sau khi chạy 3 câu lệnh dịch bit trong 3 đề thi dưới đây:

a) **(Đề Cuối kỳ 2022.2):** Cho `t0` = `0x61728394`. Thực hiện: `srli t1, t0, 4` → `slli t1, t1, 24` → `srli t1, t1, 8`.

b) **(Đề Cuối kỳ 2024.1):** Cho `t0` = `0x8192A3B4`. Thực hiện: `srli t1, t0, 4` → `slli t1, t1, 20` → `srli t1, t1, 4`.

c) **(Ôn tập Giữa kỳ - Câu 30):** Cho `t0` = `0x8192A3B4`. Thực hiện: `srai t1, t0, 4` → `slli t1, t1, 24` → `srli t1, t1, 8`.

Hướng dẫn giải:

Mẹo làm cực nhanh: Dịch bit đi 4 bit tương đương dịch đúng **1 chữ số Hexa**! Dịch 8 bit là dịch **2 chữ số Hexa**, dịch 20 bit là **5 chữ số Hexa**, dịch 24 bit là **6 chữ số Hexa**.

- **Giải ý a) Với `t0` = `0x61728394`:**
  - `srli t1, t0, 4` (dịch phải logic 1 chữ Hexa): `t1` = `0x06172839` (mất chữ `4`).
  - `slli t1, t1, 24` (dịch trái 6 chữ Hexa): Đẩy 2 chữ cuối `39` lên đầu, chèn 6 chữ `0` vào đuôi $\implies$ `t1` = `0x39000000`.

    - `srli t1, t1, 8` (dịch phải logic 2 chữ Hexa): Đẩy lùi sang phải 2 chữ, chèn `00` vào đầu $\implies$ `t1` = `0x00390000`.

- **Giải ý b) Với** `t0` = `0x8192A3B4`**:**
    - `srli t1, t0, 4` (dịch phải logic 1 chữ Hexa): `t1` = `0x08192A3B`.
    - `slli t1, t1, 20` (dịch trái 5 chữ Hexa): Đẩy 3 chữ cuối `A3B` lên đầu, chèn 5 chữ `0` vào đuôi $\implies$ `t1` = `0xA3B00000`.
    - `srli t1, t1, 4` (dịch phải logic 1 chữ Hexa): `t1` = `0x0A3B0000`.
- **Giải ý c) Với** `t0` = `0x8192A3B4`**:**
    - `srai t1, t0, 4` (dịch phải **số học** 1 chữ Hexa, vì chữ đầu là `8` âm nên chèn chữ `F`): `t1` = `0xF8192A3B`.
    - `slli t1, t1, 24` (dịch trái 6 chữ Hexa, làm rơi mất 6 chữ đầu kể cả chữ `F` vừa chèn): `t1` = `0x3B000000`.
    - `srli t1, t1, 8` (dịch phải logic 2 chữ Hexa): `t1` = `0x003B0000`.

Bài tập 4.21 (Đề Giữa kỳ 2023.2 - Đề 1 Câu 4, Cuối kỳ 2020.2 & Cuối kỳ 2021.2 - B8 Câu 2.2)

a) (Giữa kỳ 2023.2): Cho `t0` = `0xAAAAAAAA`, `t1` = `0x00FF0000`. Tìm `t2, t3` sau 2 lệnh: `srli t2, t0, 3` và `or t3, t2, t1`.

b) (Cuối kỳ 2020.2): Cho `t0` = `0xAAAAAAAA`, `t1` = `0x0000FFC0`. Tìm `t3` sau lệnh: `and t3, t0, t1`.

c) (Cuối kỳ 2020.2): Lệnh nào thay đổi `t0` từ `0x000000FF` thành `0x00000FF0`? (Và điền số còn thiếu vào chỗ trống: `addi t3, zero, 0x15` → `slli t2, t3, 4` → `addi s0, t2, 0x...` để `s0` = `0x15F`).

d) (Cuối kỳ 2021.2 - B8): Cho `t0` = `0xBEADFEED` và `t1` = `0xDEADFADE`. Tìm `t2` sau 2 lệnh: `slli t2, t0, 4` và `or t2, t2, t1`.

Hướng dẫn giải:

- **Giải ý a):** Chữ `A` = $1010_{(2)}$, nên `t0` = `1010 1010 ...` $1010_{(2)}$.
    - Dịch phải logic 3 bit (`srli t2, t0, 3`): 3 bit `000` chèn vào đầu ghép với bit `1` đầu tiên thành `0001` (`1`), các nhóm 4-bit tiếp theo đều là `0101` (`5`) $\implies$ `t2` = `0x15555555`.
    - Thực hiện `or t3, t2, t1` với `t1` = `0x00FF0000`: Bật 2 chữ số Hexa thứ 3, 4 thành `FF` $\implies$ `t3` = `0x15FF5555`.
- **Giải ý b):** Với `t0` = `0xAAAAAAAA` và `t1` = `0x0000FFC0`:
    - 4 chữ Hexa đầu AND với `0000` ra `0000`. 2 chữ tiếp theo AND với `FF` giữ nguyên là `AA`.

    - Chữ thứ 7: `A` (1010_2) & `C` (1100_2) = 1000_2 = `8`. Chữ cuối AND với `0` ra `0`.
    - Vậy `t3` = `0x0000AA80`.
- **Giải ý c):** Để dịch `0x000000FF` sang trái 1 chữ số Hexa thành `0x00000FF0`, ta cần dịch trái 4 bit: `slli t0, t0, 4`. Tương tự, `0x15` dịch trái 4 bit thành `0x150`, muốn cộng thêm để thành `0x15F` thì chỗ trống cần điền là `0xF` (hoặc `0x0F`).
- **Giải ý d):**
    - `slli t2, t0, 4`: Dịch trái 1 chữ số Hexa của `0xBEADFEED` được `t2` = `0xEADFEED0`.
    - `or t2, t2, t1`: Thực hiện phép OR từng chữ số Hexa giữa `0xEADFEED0` và `0xDEADFADE` được kết quả: `t2` = `0xFEFFFEFE`.

4.2.3 Nhóm 3: Lệnh so sánh (`slt, sltu, blt, bltu`) và Rẽ nhánh cơ bản

Bài tập 4.22 (Đề Cuối kỳ 2022.2 - #09f26f & Cuối kỳ 2024.1 - #b46119)

Cho đoạn chương trình hợp ngữ RISC-V sau:

```
addi s0, zero, 50
addi s1, zero, -51
slt  t0, s0, s1
sltu t1, s0, s1
```

Hãy cho biết nội dung `t0` và `t1` sau khi thực hiện đoạn chương trình này?

A. `t0` = 1 và `t1` = 1

B. `t0` = 0 và `t1` = 1

C. `t0` = 1 và `t1` = 0

D. `t0` = 0 và `t1` = 0

Hướng dẫn giải:

- Ta có `s0` = +50 (`0x00000032`) và `s1` = −51 (`0xFFFFFFCD`).
- **Xét lệnh `slt t0, s0, s1` (so sánh có dấu):** So sánh +50 < −51 là **Sai**, nên `t0` = **0**.
- **Xét lệnh `sltu t1, s0, s1` (so sánh không dấu):** Khi coi là số không dấu, `s1` = `0xFFFFFFCD` là một số dương cực lớn (≈ 4.29 tỷ). Vì 50 < 4.29 tỷ là **Đúng**, nên `t1` = **1**.
- **Đáp án đúng: B (`t0` = 0 và `t1` = 1).**

Bài tập 4.23 (Đề Cuối kỳ 2021.2 - Mã đề B8, Câu 2.1)

Cho câu lệnh hợp ngữ: `slti t2, t0, X`. Giả sử thanh ghi `t0` có giá trị `0xAD100002`. Với những giá trị nào của hằng số `X` thì thanh ghi `t2` sẽ có giá trị bằng 1?

Hướng dẫn giải:

- Lệnh `slti` thực hiện phép so sánh **có dấu** giữa thanh ghi `t0` (32-bit) với hằng số `X` (12-bit có dấu, chỉ nhận giá trị trong đoạn $[-2048, +2047]$).
- Xét thanh ghi `t0 = 0xAD100002`: Chữ số Hexa đầu tiên là `A` $= 1010_{(2)}$ có bit dấu bằng 1, nên `t0` là một số nguyên âm rất nhỏ ($\approx -1.39 \times 10^9$), luôn nhỏ hơn -2048.
- Do đó, với **mọi giá trị hợp lệ của hằng số 12-bit có dấu** $X \in [-2048, +2047]$ (hoặc $[-32768, +32767]$ nếu xét chuẩn 16-bit cũ), điều kiện `t0` $< X$ luôn luôn đúng, làm cho `t2` $= 1$.

Bài tập 4.24 (Ôn tập Giữa kỳ - Câu 13, 20, 38, 42, 44 & Giữa kỳ 2024.1 - #8d33e2, #484325)

Tính giá trị thanh ghi kết quả sau khi chạy từng đoạn mã RISC-V dưới đây:

a) **(Câu 13):** Cho `x22` $= 10$, `x23` $= 5$, `x24` $= 0$. Chạy: `slt x24, x22, x23` → `bne x24, x0, L1` → `addi x24, x24, 10` → `L1: addi x24, x24, 1`. Tìm `x24`?

b) **(Câu 38):** Cho `x5` $= 10$, `x6` $= 10$. Chạy: `addi x7, x0, 100` → `bne x5, x6, Skip` → `addi x7, x7, 50` → `Skip: addi x7, x7, 1`. Tìm `x7`?

c) **(Câu 20):** Cho `x10` $=$ `0xFFFFFFF0` (-16) và `x11` $=$ `0x00000005`. Chạy: `addi x12, x0, 20` → `blt x10, x11, TakeBranch` → `addi x12, x12, 10` → `j Done` → `TakeBranch: addi x12, x12, 5` → `Done: addi x12, x12, 1`. Tìm `x12`?

d) **(Câu 42):** Cũng với `x10` $=$ `0xFFFFFFF0` và `x11` $= 5$, nhưng thay lệnh `blt` ở câu c bằng lệnh không dấu `bltu x10, x11, TakeBranchU`. Tìm giá trị cuối cùng của `x13`?

e) **(Đề GK 2024.1 - #8d33e2 & #484325):**

- Bài 1: `t0` $= -2$, `t1` $= 2$. Nếu `blt t0, t1, else` đúng thì `a0` $= 2 + 7$, sai thì `a0` $= 1 + 5$.
- Bài 2: `t0` $= 10$, `t1` $= 10$. Nếu `bltu t0, t1, else` đúng thì `a0` $= 20 + 7$, sai thì `a0` $= 10 + 5$.

Hướng dẫn giải:

- **Giải ý a):** Vì $10 < 5$ là Sai nên `x24` $= 0$. Lệnh `bne x24, x0, L1` không nhảy, thực hiện tiếp `x24 = 0 + 10 = 10`, sau đó chạy luôn lệnh ở nhãn `L1`: `x24` $= 10 + 1 =$ **11**.
- **Giải ý b):** Vì `x5` == `x6` $(10 == 10)$ nên lệnh `bne` không nhảy. Thanh ghi `x7` được tính tuần tự: `x7` $= 100 + 50 + 1 =$ **151**.

- **Giải ý c):** Lệnh `blt` so sánh có dấu: $-16 < 5$ là **Đúng** $\Longrightarrow$ Nhảy tới `TakeBranch` (`x12` $= 20 + 5 = 25$), rồi xuống `Done`: `x12` $= 25 + 1 =$ **26**.
- **Giải ý d):** Lệnh `bltu` so sánh không dấu: `0xFFFFFFF0` > 5 nên điều kiện nhỏ hơn là **Sai** $\Longrightarrow$ Không nhảy, thực hiện `x13` $= 20 + 10 = 30$, nhảy tới `DoneU`: `x13` $= 30 + 1 =$ **31**.
- **Giải ý e):**
  - Bài 1: Vì $-2 < 2$ là Đúng nên nhảy xuống `else`: `a0` $= 2 + 7 =$ **9**.
  - Bài 2: Vì $10 < 10$ là Sai nên không nhảy: `a0` $= 10 + 5 =$ **15**.

4.2.4 Nhóm 4: Truy cập bộ nhớ (`lw, lh, lb, sw`) theo thứ tự Little-Endian

Bài tập 4.25 (Đề Cuối kỳ 2022.2 - #fe0e91 & Ôn tập Cuối kỳ P2 - Câu 9)

Cho vùng nhớ dữ liệu lưu trữ theo thứ tự đầu nhỏ (Little-Endian) như bảng dưới. Giả thiết nội dung thanh ghi cơ sở `s6` (`x16`) = `0x10012000`.

Dữ liệu (Word 32-bit)	Địa chỉ
0x12345678	0x10012000
0xA1B2C3D4	0x10012004
0x11223344	0x10012008
0xAABBCCDD	0x1001200C
0x66778899	0x10012010

Hãy cho biết nội dung các thanh ghi `t0` (`x5`) và `t1` (`x6`) dưới dạng số Hexa (8 chữ số) sau khi thực hiện các lệnh sau:

```
lw t0, 8(s6)
lb t1, 14(s6)
```

Hướng dẫn giải:

- **Bước 1 – Tìm `t0` từ lệnh `lw t0, 8(s6)`:**
  - Địa chỉ cần đọc: `0x10012000` $+ 8 =$ `0x10012008`.
  - Vì bảng đề bài đã viết sẵn dữ liệu theo từng Word 32-bit ở các địa chỉ bội của 4, nên lệnh `lw` tại địa chỉ `0x10012008` sẽ nạp nguyên vẹn cả Word đó vào `t0`:

$$\texttt{t0} = \texttt{0x11223344}$$

- **Bước 2 – Tìm `t1` từ lệnh `lb t1, 14(s6)`:**

* Đổi độ lệch 14 ra tổng của bội số 4 gần nhất: $14 = 12 + 2 = \texttt{0x0C} + 2$.
* Như vậy byte cần đọc nằm trong Word ở địa chỉ `0x10012000` + `0x0C` = `0x1001200C` (Word này có giá trị `0xAABBCCDD`), tại vị trí độ lệch +2.
* Theo quy tắc Little-Endian (đếm từ phải sang trái):
  - Độ lệch +0 (`0x1001200C`): `0xDD`
  - Độ lệch +1 (`0x1001200D`): `0xCC`
  - Độ lệch +2 (`0x1001200E`): `0xBB`
  - Độ lệch +3 (`0x1001200F`): `0xAA`
* Lệnh `lb` nạp byte `0xBB` vào thanh ghi 32-bit `t1`. Vì chữ `B` = $1011_{(2)}$ có bit dấu bằng 1 (số âm), nên `lb` mở rộng dấu bằng cách điền toàn chữ `F` vào 6 chữ số Hexa bên trái:

$$\texttt{t1} = \texttt{0xFFFFFFBB}$$

Bài tập 4.26 (Đề Giữa kỳ 2023.2 - Đề 2, Câu 2)

Cho địa chỉ và nội dung các word nhớ dữ liệu lưu trữ theo kiểu Little-Endian như bảng dưới. Giả thiết thanh ghi `s3` = `0x10012010`. Hãy cho biết nội dung các thanh ghi `t0, t1` dưới dạng số Hexa sau khi thực hiện các lệnh:

```
lw t0, 20(s3)
lh t1, 14(s3)
```

Địa chỉ	Dữ liệu	Địa chỉ	Dữ liệu
0x10012010	0x2F3B4A5B	0x10012020	0xC2D1E0F1
0x10012014	0x6E7D8C9B	0x10012024	0xE2D3C4B5
0x10012018	0xAAB9C8D7	0x10012028	0xA6978879
0x1001201C	0xE6B5A4B3	0x1001202C	0x6A5B4C3B

Hướng dẫn giải:

– **Tính `t0` (`lw t0, 20(s3)`):**

* Đổi độ lệch 20 sang hệ Hexa: $20 = 16 + 4 = \texttt{0x14}$.
* Địa chỉ cần đọc: `0x10012010` + `0x14` = `0x10012024`.
* Tra bảng tại địa chỉ `0x10012024`, ta được: `t0` = `0xE2D3C4B5`.

– **Tính `t1` (`lh t1, 14(s3)`):**

* Tách độ lệch $14 = 12 + 2 = \texttt{0x0C} + 2$.
* Word chứa dữ liệu nằm ở địa chỉ `0x10012010` + `0x0C` = `0x1001201C` (có giá trị `0xE6B5A4B3`).
* Trong Word `0xE6B5A4B3`, 2 byte ở độ lệch $+0, +1$ là `A4B3`; còn 2 byte ở độ lệch $+2, +3$ là `E6B5`.

- Vì `E6B5` bắt đầu bằng chữ `E` ≥ 8 (số âm), lệnh `lh` mở rộng dấu thành 32-bit: `t1` = `0xFFFFE6B5`.

Bài tập 4.27 (Đề Cuối kỳ 2023.1 - #d4c0fb, Giữa kỳ 2024.1 - #2f8025 & Ôn tập Giữa kỳ - Câu 54)

Cho một hệ thống RISC-V dùng thứ tự đầu nhỏ (Little-Endian) với các byte dữ liệu trong bộ nhớ như sau:

ADDRESS	VALUE	ADDRESS	VALUE	ADDRESS	VALUE
0x10010000	0x1a	0x10010002	0x3c	0x10010004	0x5e
0x10010001	0x2b	0x10010003	0x4d	0x10010005	0x6f

a) (Đề CK 2023.1 & Ôn tập GK Câu 54): Tìm giá trị của thanh ghi `s1` sau khi chạy: `lui t0, 0x10010` → `addi t0, t0, 2` → `lh s1, 2(t0)`.

b) (Đề GK 2024.1 - #2f8025): Tìm giá trị của thanh ghi `s0` sau khi chạy: `li t1, 0x10010004` → `addi t1, t1, -4` → `lh s0, 2(t1)`.

Hướng dẫn giải:

- **Giải ý a):**
  - Sau 2 lệnh đầu, thanh ghi `t0` chứa địa chỉ `0x10010002`.
  - Lệnh `lh s1, 2(t0)` đọc 1 halfword (2 byte) bắt đầu từ địa chỉ `0x10010002`+2 = `0x10010004` (gồm 2 byte ở địa chỉ `0x10010004` và `0x10010005`).
  - Theo quy tắc Little-Endian: byte ở địa chỉ nhỏ (`0x10010004` = `0x5e`) đứng bên phải (byte thấp), byte ở địa chỉ lớn (`0x10010005` = `0x6f`) đứng bên trái (byte cao) $\implies$ ghép lại thành `0x6f5e`.
  - Vì chữ số đầu là `6` < 8 (số dương), mở rộng dấu thành 32-bit được: `s1` = `0x00006f5e`.
- **Giải ý b):**
  - Sau `addi t1, t1, -4`, thanh ghi `t1` = `0x10010000`.
  - Lệnh `lh s0, 2(t1)` đọc 2 byte tại địa chỉ `0x10010002` (`0x3c`) và `0x10010003` (`0x4d`).
  - Ghép theo Little-Endian (địa chỉ lớn đứng trước, địa chỉ nhỏ đứng sau) và mở rộng dấu dương: `s0` = `0x00004d3c`.

Bài tập 4.28 (Đề Cuối kỳ 2022.2 - #ad4a6e)

Bộ nhớ máy tính được đánh địa chỉ cho từng byte và lưu trữ theo thứ tự đầu nhỏ (Little-Endian). Cho A là mảng các số nguyên 32-bit, được lưu trữ tại địa chỉ cơ sở `0x10012500`. Cho biết phần tử $A[1]$ = `0x22334455`. Hãy chỉ ra nội dung byte nhớ có địa chỉ `0x10012506`?

A. `0x33` B. `0x22` C. `0x44` D. `0x55`

Hướng dẫn giải:

- Phần tử $A[0]$ chiếm 4 byte từ `0x10012500` đến `0x10012503`.
- Phần tử $A[1]$ = `0x22334455` bắt đầu tại địa chỉ `0x10012500` + 4 = `0x10012504`.
- Theo Little-Endian, 4 byte của $A[1]$ được phân bố lần lượt từ phải sang trái như sau:
  - Địa chỉ `0x10012504`: `0x55`
  - Địa chỉ `0x10012505`: `0x44`
  - Địa chỉ `0x10012506`: `0x33`
  - Địa chỉ `0x10012507`: `0x22`
- **Đáp án đúng: A (`0x33`).**

Bài tập 4.29 (Đề Giữa kỳ 2023.2 - Đề 1, Câu 3)

Cho biết bộ nhớ chính được đánh địa chỉ cho từng byte và lưu trữ theo kiểu đầu nhỏ (Little-Endian). Giả sử thanh ghi `s6` chứa địa chỉ của một ngăn nhớ dữ liệu. Hãy chỉ ra nội dung của các thanh ghi `s0, s1` theo số Hexa sau khi thực hiện đoạn chương trình RISC-V dưới đây:

```
lui  s0, 0x1B32C
addi s0, s0, 0x2E5    # s0 = 0x1B32C2E5
sw   s0, 0(s6)
lb   s1, 1(s6)
```

Hướng dẫn giải:

- Thanh ghi `s0` được khởi tạo giá trị 32-bit là: `s0` = `0x1B32C2E5`.
- Lệnh `sw s0, 0(s6)` lưu 4 byte của `s0` ra bộ nhớ theo thứ tự Little-Endian:
  - Tại `0(s6)` lưu byte thấp nhất: `0xE5`.
  - Tại `1(s6)` lưu byte tiếp theo: `0xC2`.
  - Tại `2(s6)` lưu byte: `0x32`; tại `3(s6)` lưu byte cao nhất: `0x1B`.
- Lệnh `lb s1, 1(s6)` đọc byte `0xC2` tại địa chỉ `1(s6)`. Vì chữ `C` = $1100_{(2)}$ có bit dấu bằng 1 (số âm), lệnh `lb` mở rộng dấu thành 32-bit:

  `s1` = `0xFFFFFFC2`

Bài tập 4.30 (Đề Giữa kỳ 2024.1 - #f5f091 & #cd0189)

a) **(Câu #f5f091):** Giá trị của `s0` sau khi thực thi đoạn mã sau là gì?

```
.data
x: .byte -24
.text
la t0, x
lb s0, 0(t0)
```

b) **(Câu #cd0189):** Tìm giá trị Hexa lưu trong thanh ghi `s2` sau khi thực thi đoạn mã sau trên bộ xử lý RISC-V Little-Endian:

```
.data
A: .word 0xa1b2c3d4
.text
la   t0, A
lb   s1, 0(t0)
andi s2, s1, 0xf0
```

Hướng dẫn giải:

- **Giải ý a):** Lệnh `lb` nạp giá trị -24 vào thanh ghi 32-bit `s0` và mở rộng dấu thành số nguyên 32-bit có dấu -24. Đổi -24 sang hệ Hexa 32-bit: $+24 =$ `0x00000018` $\Longrightarrow$ bù 2 là `0xffffffe8`.
- **Giải ý b):**
  - Trong bộ nhớ Little-Endian, tại địa chỉ đầu tiên `0(t0)` của word `0xa1b2c3d4` chứa byte thấp nhất là `0xd4`.
  - Lệnh `lb s1, 0(t0)` nạp `0xd4` và mở rộng dấu âm thành `s1` = `0xffffffd4`.
  - Lệnh `andi s2, s1, 0xf0`: Hằng số `0x0f0` là số dương 12-bit, mở rộng 32-bit là `0x000000f0`. Thực hiện phép AND:

$$\texttt{s2} = \texttt{0xffffffd4} \ \& \ \texttt{0x000000f0} = \texttt{0x000000d0}$$

Bài tập 4.31 (Đề Giữa kỳ 2022.2 - Đề 0008 & Đề 0007, Câu 2)

a) (Đề 0008): Một biến 32-bit được lưu vào bộ nhớ chính theo thứ tự đầu nhỏ (Little-Endian) từ địa chỉ `0x100110F8` đến `0x100110FB` lần lượt có các byte giá trị là: `0x5C, 0x52, 0x0E, 0xEF`. Biến 32-bit này có giá trị bằng bao nhiêu? Hãy viết các lệnh hợp ngữ RISC-V để nạp biến 32-bit trên vào thanh ghi `s1`.

b) (Đề 0007): Một biến 32-bit có giá trị bằng `0x4CDC001A` được lưu vào bộ nhớ chính ở địa chỉ `0x1001272C`. Hãy điền các byte vào 4 địa chỉ từ `0x1001272C` đến `0x1001272F`.

Hướng dẫn giải:

- **Giải ý a):**

    - Ghép các byte theo thứ tự từ địa chỉ cao (`0x100110FB`) về địa chỉ thấp (`0x100110F8`): Giá trị biến 32-bit là `0xEF0E525C`.
    - Để nạp biến ở địa chỉ `0x100110F8` vào thanh ghi `s1` trong RISC-V:

```
lui  t0, 0x10011
lw   s1, 0x0F8(t0)    # Đọc word tại 0x10011000 + 0x0F8 = 0x100110F8
```

- **Giải ý b):** Với biến `0x4CDC001A` lưu tại `0x1001272C` theo Little-Endian:
    - `0x1001272C`: `0x1A` | `0x1001272D`: `0x00`
    - `0x1001272E`: `0xDC` | `0x1001272F`: `0x4C`

Bài tập 4.32 (Đề Giữa kỳ 2024.1 - #6a94a2 & Ôn tập Giữa kỳ - Câu 2)

Địa chỉ của toán hạng đích trong lệnh `sw` dưới đây là bao nhiêu (viết ở dạng số Hexa)?

```
li   s0, 0x1001BCDE
addi s1, s0, 0x321
sw   s0, 1(s1)
```

Hướng dẫn giải:

- Lệnh 1: `s0` = `0x1001BCDE`.
- Lệnh 2: `s1` = `0x1001BCDE` + `0x321` = `0x1001BFFF`.
- Lệnh 3 (`sw s0, 1(s1)`): Địa chỉ của toán hạng đích trong bộ nhớ là:

$$\text{Địa chỉ} = \texttt{s1} + 1 = \texttt{0x1001BFFF} + 1 = \texttt{0x1001C000}$$

  (Lưu ý: Trong ảnh chụp file đề GK 2024.1, chữ `C` bị nhận diện OCR nhầm thành số `0`, kết quả đúng phải là `0x1001C000`).

4.2.5 Nhóm 5: Chuyển đổi câu lệnh C sang Hợp ngữ RISC-V (Mảng, Con trỏ, Rẽ nhánh)

Bài tập 4.33 (Đề Cuối kỳ 2022.2 - Trang 14, 2024.1 - #f4, 2020.2 & Ôn tập Giữa kỳ - Câu 9)

Đoạn lệnh hợp ngữ RISC-V dưới đây tương ứng với câu lệnh C nào (biết rằng các thanh ghi `s0, s1, s2, s3` được sử dụng tương ứng cho các biến `f, g, h, i`)?

```
slli t0, s1, 2
sub  s3, s0, s2
add  s3, s3, t0
addi s3, s3, 5
```

A. `i = f + h + 4*g - 5`

B. `i = f - h + 4*g + 5`

C. `i = f - h + 4*g - 5`

D. `i = f + h - 4*g - 5`

Hướng dẫn giải:

- Lệnh 1 (`slli t0, s1, 2`): Dịch trái `s1` (`g`) đi 2 bit, tương đương nhân với $2^2 = 4 \implies$ `t0` $= 4 \times g$.
- Lệnh 2 (`sub s3, s0, s2`): `s3` $= i = f - h$.
- Lệnh 3 (`add s3, s3, t0`): $i = (f - h) + 4 \times g$.
- Lệnh 4 (`addi s3, s3, 5`): $i = f - h + 4 \times g + 5$.
- **Đáp án đúng: B (`i = f - h + 4*g + 5`).**

Bài tập 4.34 (Đề Giữa kỳ 2024.1 - #f96f43 & #12416e & Ôn tập Giữa kỳ - Câu 47)

a) **(Câu #f96f43):** Chọn đoạn lệnh RISC-V thực hiện câu lệnh C: `*a = b + c;` (giả sử `a` là con trỏ nằm trong thanh ghi `a1`, còn `b, c` là các biến nằm trong `a2, a3`).

b) **(Câu #12416e & Câu 47):** Chọn đoạn lệnh RISC-V thực hiện câu lệnh C: `a = b + *c;` (giả sử `a, b` là biến trong `a1, a2`, còn `c` là con trỏ trong `a3`).

Hướng dẫn giải:

- **Giải ý a) `*a = b + c;`:** Ký hiệu `*a` ở vế trái nghĩa là **ghi kết quả ra vùng nhớ** mà con trỏ `a` (`a1`) đang trỏ tới.
  - Bước 1: Tính tổng `b + c` vào thanh ghi tạm: `add t0, a2, a3`.
  - Bước 2: Lưu kết quả vào địa chỉ trong `a1`: `sw t0, 0(a1)`.
- **Giải ý b) `a = b + *c;`:** Ký hiệu `*c` ở vế phải nghĩa là **đọc giá trị từ vùng nhớ** mà con trỏ `c` (`a3`) đang trỏ tới.
  - Bước 1: Đọc dữ liệu từ địa chỉ trong `a3` vào thanh ghi tạm: `lw t0, 0(a3)`.
  - Bước 2: Cộng với `b` (`a2`) rồi cất vào `a` (`a1`): `add a1, a2, t0`.

Bài tập 4.35 (Đề Cuối kỳ 2022.2 - #757298, 2024.1 - #c99f87 & Ôn tập Giữa kỳ - Câu 64)

Biến `x` thuộc kiểu word 32-bit nằm ở địa chỉ `0x10010200`. Đoạn chương trình RISC-V nào dưới đây tương đương với câu lệnh ngôn ngữ C: `x = x & 15;`?

A. `lui t3, 0x10010` → `addi t3, t3, 0x200` → `lw s3, 0(t3)` → `andi s3, s3, 15` → `sw s3, 0(t3)`

B. `lui t3, 0x10200` → `addi t3, t3, 0x100` → `lw s3, 0(t3)` → `andi s3, s3, 15` → `sw s3, 0(t3)`

C. `lui t3, 0x10010` → `addi t3, t3, 0x200` → `sw s3, 0(t3)` → `andi s3, s3, 15` → `lw s3, 0(t3)`

Hướng dẫn giải:

- Bước 1: Tạo địa chỉ `0x10010200` trong thanh ghi con trỏ `t3` bằng cặp lệnh `lui t3, 0x10010` và `addi t3, t3, 0x200`.
- Bước 2: Vì `x` là biến nằm trong bộ nhớ, ta phải **đọc (`lw`)** giá trị của `x` lên thanh ghi `s3` trước: `lw s3, 0(t3)`.
- Bước 3: Thực hiện phép AND với 15: `andi s3, s3, 15`.
- Bước 4: **Ghi (`sw`)** kết quả từ `s3` trả lại bộ nhớ: `sw s3, 0(t3)`.
- **Đáp án đúng: A (Tùy chọn 1).**

Bài tập 4.36 (Đề Cuối kỳ 2022.2 - #13504d, 2024.1 - Trang 7 & Giữa kỳ 2024.1 - #d0125d)

a) (Đề CK 2022.2 & 2024.1): Giả thiết biến `k` ở thanh ghi `s1`; `P, Q` là hai mảng số nguyên 32-bit có địa chỉ cơ sở nằm trong `s6` và `s7`. Đoạn lệnh sau tương ứng với câu lệnh C nào?

```
lw  t0, 160(s7)
add t0, s1, t0
sw  t0, 80(s6)
```

b) (Đề GK 2024.1 - #d0125d): Cho `s1, s2` lần lượt giữ địa chỉ cơ sở của mảng số nguyên 32-bit `A` và `B`. Đoạn lệnh sau tương ứng với câu lệnh C nào?

```
lw   t0, 24(s1)
addi s0, t0, 1
sw   s0, 20(s2)
```

Hướng dẫn giải:

- **Quy tắc đổi Độ lệch byte (Offset) sang Chỉ số mảng (Index):** Vì mỗi phần tử số nguyên 32-bit chiếm **4** byte, nên muốn tìm chỉ số phần tử mảng, ta lấy **Offset chia cho 4**! (Không bao giờ được giữ nguyên số 160 hay 80).
- **Giải ý a):**

    - `lw t0, 160(s7)`: Đọc từ mảng `Q` tại chỉ số $160 \div 4 = 40 \implies$ `t0` $= Q[40]$.
    - `add t0, s1, t0`: `t0` $= k + Q[40]$.
    - `sw t0, 80(s6)`: Ghi vào mảng `P` tại chỉ số $80 \div 4 = 20 \implies$ `P[20] = k + Q[40]`.
  - **Giải ý b):**
    - `lw t0, 24(s1)`: Đọc từ mảng `A` tại chỉ số $24 \div 4 = 6 \implies$ `t0` $= A[6]$.
    - `addi s0, t0, 1`: `s0` $= A[6] + 1$.
    - `sw s0, 20(s2)`: Ghi vào mảng `B` tại chỉ số $20 \div 4 = 5 \implies$ `B[5] = A[6] + 1`.

Bài tập 4.37 (Đề Giữa kỳ 2024.1 - #3602ec & Ôn tập Giữa kỳ - Câu 67)

Cho mảng số nguyên 32-bit `A[]` được cấp phát trong bộ nhớ bắt đầu tại địa chỉ cơ sở `0x20230200`. Giá trị của phần tử nào trong mảng `A` sẽ nằm trong thanh ghi `t0` sau khi thực hiện đoạn mã dưới đây?

- **Biến thể 1 (Ôn tập GK - Câu 67):** `lui s0, 0x20230` → `addi s0, s0, 0x20C` → `lw t0, 4(s0)`.
- **Biến thể 2 (Đề GK 2024.1 - #3602ec):** `lui s0, 0x20230` → `addi s0, s0, 0x200` → `lw t0, 4(s0)`.

Hướng dẫn giải:

- **Xét Biến thể 1:**
  - Sau 2 lệnh đầu, thanh ghi `s0` = `0x2023020C`.
  - Lệnh `lw t0, 4(s0)` đọc phần tử tại địa chỉ: `0x2023020C` + 4 = `0x20230210`.
  - Độ lệch so với địa chỉ cơ sở `0x20230200` là: `0x20230210` − `0x20230200` = `0x10` = 16 byte.
  - Chỉ số phần tử: $16 \div 4 = \mathbf{4} \implies$ Đó là phần tử `A[4]`.
- **Xét Biến thể 2:** Nếu lệnh `addi` cộng `0x200` thì `s0` = `0x20230200`, khi đó `lw t0, 4(s0)` có độ lệch là 4 byte $\implies 4 \div 4 = 1$, ứng với `A[1]` (trên hệ thống daotao.ai năm 2024.1 đề gốc là `0x20C` nên đáp án hệ thống chấm là `A[4]`).

Bài tập 4.38 (Đề Giữa kỳ 2022.2 - Đề 0008 Câu 8 & Đề 0007 Câu 7)

Cho mảng các số nguyên 32-bit `A` ở địa chỉ cơ sở `0x1000CDC0`.

a) (Đề 0008): Thực hiện 3 lệnh: `lui s1, 0x1000D` → `addi s1, s1, -572` (để `s1` = `0x1000CDC4`) → `lw t1, 4(s1)`. Hỏi các địa chỉ byte nhớ nào được nạp vào `t1` và `t1` chứa phần tử nào của mảng `A`?

b) (Đề 0007): Nếu `s1` = `0x1000CDC8` và thực hiện lệnh `lw t1, -4(s1)`, thì `t1` chứa phần tử nào của mảng `A`?

Hướng dẫn giải:

- **Giải ý a):**
  - Địa chỉ word được nạp vào `t1` là: `0x1000CDC4` + 4 = `0x1000CDC8`.
  - Bốn địa chỉ byte nhớ được nạp vào `t1` là: `0x1000CDC8, 0x1000CDC9, 0x1000CDCA, 0x1000CDCB`.
  - Độ lệch so với đầu mảng `0x1000CDC0` là `0xCDC8` − `0xCDC0` = 8 byte $\implies$ Chỉ số phần tử là $8 \div 4 = 2$, tức phần tử `A[2]`.
- **Giải ý b):**
  - Địa chỉ word được nạp: `0x1000CDC8` − 4 = `0x1000CDC4`.
  - Độ lệch so với đầu mảng là `0xCDC4` − `0xCDC0` = 4 byte $\implies$ Chỉ số là $4 \div 4 = 1$, tức phần tử `A[1]`.

Bài tập 4.39 (Đề Cuối kỳ 2022.2 - #0542f & #70e201)

a) **(Câu #0542f):** Cho mảng `M[]` gồm 100 phần tử số nguyên 32-bit bắt đầu từ địa chỉ `0x10020400`. Đoạn lệnh hoán đổi $M[i]$ và $M[j]$ sử dụng `s6` chứa địa chỉ của $M[i]$ và `s7` chứa địa chỉ của $M[j]$. Nếu $i = 9, j = 6$, nội dung các thanh ghi `s6` và `s7` cần có giá trị bao nhiêu (viết theo số Hexa)?

b) **(Câu #70e201):** Cho mảng 2 chiều `M` kích thước (4×5) gồm các phần tử 32-bit, lưu trữ theo hàng bắt đầu tại địa chỉ `0x10024000`. Để lệnh `lw t0, 0(s6)` đọc phần tử $M[i, j]$ với $i = 2, j = 2$ (chỉ số hàng và cột tính từ 0), nội dung thanh ghi `s6` phải bằng bao nhiêu?

Hướng dẫn giải:

- **Giải ý a):**
  - Với $i = 9$: Độ lệch = $9 \times 4 = 36$ byte = `0x24` byte $\implies$ `s6` = `0x10020400` + `0x24` = `0x10020424`.
  - Với $j = 6$: Độ lệch = $6 \times 4 = 24$ byte = `0x18` byte $\implies$ `s7` = `0x10020400` + `0x18` = `0x10020418`.
- **Giải ý b):**
  - Mảng có 5 cột (mỗi hàng có 5 phần tử). Với $i = 2, j = 2$, số phần tử đứng trước $M[2, 2]$ là: $2 \times 5 + 2 = 12$ phần tử.
  - Độ lệch tính theo byte: $12 \times 4 = 48$ byte = `0x30` byte.
  - Vậy địa chỉ trong thanh ghi `s6` là: `0x10024000` + `0x30` = `0x10024030`.

Bài tập 4.40 (Đề Cuối kỳ 2022.2 - #bcfd00 & Ôn tập Cuối kỳ P2 - Câu 38)

Cho đoạn hợp ngữ RISC-V như dưới đây, biết rằng các biến `a, b, c` được gán tương ứng trong các thanh ghi `x18 (s2), x19 (s3), x20 (s4)`:

```
      slt x5, x20, x19
      bne x5, x0, L1
      add x18, x18, x19
      j   L2
L1:   add x18, x18, x20
L2:
```

Hãy chỉ ra câu lệnh C tương ứng:

A. `if (b > c) a = a + b; else a = a + c;`

B. `if (b <= c) a = a + b; else a = a + c;`

C. `if (b < c) a = a + b; else a = a + c;`

Hướng dẫn giải:

- Lệnh 1 (`slt x5, x20, x19`): Kiểm tra xem `x20` < `x19` (tức $c < b$, hay $b > c$). Nếu đúng thì `x5` = 1, nếu sai (tức $b \leq c$) thì `x5` = 0.
- Lệnh 2 (`bne x5, x0, L1`): Nếu `x5` $\neq 0$ (tức $c < b$), nhảy xuống nhãn `L1` để thực hiện $a = a + c$.
- Ngược lại, nếu không nhảy (tức khi $\mathbf{b \leq c}$), chương trình chạy lệnh ngay bên dưới là `add x18, x18, x19` ($a = a + b$).
- Vậy đoạn mã tương đương với: `if (b <= c) a = a + b; else a = a + c;`.
- **Đáp án đúng: B.**

Bài tập 4.41 (Tổng hợp các bài tự luận viết hợp ngữ từ biểu thức C trong đề Giữa kỳ 2022.2, 2023.2 & Cuối kỳ 2021.2)

Hãy viết các đoạn mã hợp ngữ RISC-V tối thiểu (không dùng lệnh giả) cho các yêu cầu sau:

a) (GK 2023.2 - Đề 1 Câu 5): `f = g + h - A[20];` (với `f, g, h` ở `s0, s1, s2`; địa chỉ cơ sở mảng 32-bit `A` ở `s7`).

b) (GK 2023.2 - Đề 2 Câu 3): Dịch ngược 2 đoạn mã hợp ngữ sau sang C (với `f, g, h, i, j` ở `s0..s4`; mảng `A, B` ở `s6, s7`):

- Đoạn 1: `lw t1, 20(s7)` → `sub s2, s3, t1`.
- Đoạn 2: `add t0, s1, s0` → `slli t1, s4, 2` → `add t1, s6, t1` → `sw t0, 0(t1)`.

c) (GK 2023.2 - Đề 2 Câu 5): Viết hợp ngữ cho câu lệnh: `if (i <= j) f = f + g; else f = f + h;` (với `f, g, h, i, j` ở `s0..s4`).

d) (GK 2022.2 - Đề 0009, 0008, 0006): Viết hợp ngữ cho các biểu thức: (1) `t0` $= 4 \times$ (`t1` + `t2`) $+ 5$; (2) `t0` $=$ `t1` $+ 8 \times$ (`t2` $- 7$); (3) Điền 1 lệnh còn thiếu vào giữa `add t2, s3, s4` và `sub s3, t3, s4` để thực hiện $y = (x + y) \times 16 - x$ (với y ở `s3`, x ở `s4`).

e) (CK 2021.2 - B2 & B3, Câu 2): Thực hiện câu lệnh `x[2] = x[7] + a;` (với `a` ở `s2`, mảng 32-bit `x` có địa chỉ cơ sở là `0x00283A00`).

Hướng dẫn giải:

Giải ý a) `f = g + h - A[20];`:

```
lw   t0, 80(s7)       # t0 = A[20] (vì 20 * 4 = 80 byte)
add  t1, s1, s2       # t1 = g + h
sub  s0, t1, t0       # f = (g + h) - A[20]
```

Giải ý b) Dịch hợp ngữ sang C:

- Đoạn 1: `20(s7)` là $B[20/4] = B[5]$, `s2` là h, `s3` là $i \implies$ Câu lệnh C: `h = i - B[5];`.
- Đoạn 2: `t0` $= g + f$; `t1` $=$ `s6` $+ 4 \times j$ (địa chỉ của $A[j]$) $\implies$ Câu lệnh C: `A[j] = g + f;`.

Giải ý c) `if (i <= j) f = f + g; else f = f + h;`:

```
    blt  s4, s3, Else     # Nếu j < i (tức i > j) thì nhảy xuống Else
    add  s0, s0, s1       # f = f + g
    jal  x0, Exit         # Nhảy không điều kiện tới Exit
Else:
    add  s0, s0, s2       # f = f + h
Exit:
```

Giải ý d):

- Biểu thức 1 (`t0` $= 4 \times$ (`t1` + `t2`) $+ 5$): `add t0, t1, t2` $\rightarrow$ `slli t0, t0, 2` $\rightarrow$ `addi t0, t0, 5`.
- Biểu thức 2 (`t0` $=$ `t1` $+ 8 \times$ (`t2` $- 7$)): `addi t0, t2, -7` $\rightarrow$ `slli t0, t0, 3` $\rightarrow$ `add t0, t1, t0`.
- Biểu thức 3 ($y = (x + y) \times 16 - x$): Nhân với $16 = 2^4$ là dịch trái 4 bit. Lệnh còn thiếu ở giữa là: `slli t3, t2, 4`. (Và nếu đề bài cho 3 lệnh `add s0, t2, t2` $\rightarrow$ `slli s0, s0, 1` $\rightarrow$ `slli s0, s0, 2` tương đương nhân $2 \times 2 \times 4 = 16$, thì thay thế bằng 1 lệnh duy nhất là: `slli s0, t2, 4`).

Giải ý e) `x[2] = x[7] + a;` **với địa chỉ cơ sở** `0x00283A00`**:**

– Chú ý: Chữ số Hexa thứ 3 từ phải sang của `0x00283A00` là `A` ≥ 8 (bit 11 bằng 1). Khi dùng `addi` phải cộng 1 vào `lui`:

```
lui  t0, 0x00284      # t0 = 0x00284000
addi t0, t0, -1536    # t0 = 0x00284000 - 0x600 = 0x00283A00 (địa chỉ cơ sở x)
lw   t1, 28(t0)       # t1 = x[7] (vì 7 * 4 = 28)
add  t1, t1, s2       # t1 = x[7] + a
sw   t1, 8(t0)        # x[2] = x[7] + a (vì 2 * 4 = 8)
```

4.2.6 Nhóm 6: Phân tích Vòng lặp và Đếm tổng số lệnh thực thi

Bài tập 4.42 (Đề Giữa kỳ 2024.1 - #f4fd9f)

Giá trị của thanh ghi `s5` ở hệ thập phân sau khi thực hiện đoạn mã RISC-V dưới đây là bao nhiêu (giả sử ban đầu `s5` = 0)?

```
.data
A: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
.text
    li   s1, 0
    li   s3, 10
    la   s2, A
loop:
    add  t1, s1, s1
    add  t1, t1, t1
    add  t1, t1, s2
    lw   t0, 0(t1)
    add  s5, s5, t0
    addi s1, s1, 2
    bne  s1, s3, loop
```

Hướng dẫn giải:

– Hai lệnh `add t1, s1, s1` (`t1` = 2 × `s1`) và `add t1, t1, t1` (`t1` = 4 × `s1`) tính độ lệch địa chỉ của phần tử A[`s1`].

– Lệnh `lw t0, 0(t1)` đọc phần tử A[`s1`] rồi cộng dồn vào `s5`.

– Biến chạy `s1` bắt đầu từ 0, mỗi vòng tăng thêm 2 (`addi s1, s1, 2`), và dừng lại khi `s1` == 10.

– Vậy `s1` lần lượt nhận các giá trị: $0, 2, 4, 6, 8$.

- Các phần tử mảng được cộng vào `s5` là (lưu ý chỉ số mảng bắt đầu từ 0, với $A[0] = 1$):

$$\texttt{s5} = A[0] + A[2] + A[4] + A[6] + A[8] = 1 + 3 + 5 + 7 + 9 = \mathbf{25}$$

- **Đáp số: 25.**

Bài tập 4.43 (Đề Cuối kỳ 2020.2)

Hãy cho biết nội dung thanh ghi `a0` (trong đề cũ là `$v0`) nhận giá trị bằng bao nhiêu sau khi thực hiện đoạn chương trình dưới đây:

```
      addi a1, x0, 14
      addi s0, x0, 0
      add  s1, x0, x0
loop: slt  t0, a1, s1
      bne  t0, x0, finish
      add  s0, s0, s1
      addi s1, s1, 2
      j    loop
finish:
      add  a0, s0, x0
```

A. 54 B. 14 C. 56 D. 28

Hướng dẫn giải:

- Khởi tạo: `a1` = 14, tổng `s0` = 0, biến chạy `s1` = 0.
- Điều kiện thoát vòng lặp: `slt t0, a1, s1` kiểm tra 14 < `s1`. Khi `s1` ≤ 14 thì `t0` = 0 (tiếp tục cộng `s1` vào `s0` và tăng `s1` thêm 2). Khi `s1` = 16 (14 < 16 đúng) thì thoát.
- Vậy chương trình tính tổng các số chẵn từ 0 đến 14:

$$\texttt{a0} = \texttt{s0} = 0 + 2 + 4 + 6 + 8 + 10 + 12 + 14 = \mathbf{56}$$

- **Đáp án đúng: C** (56)**.**

Bài tập 4.44 (Đề Cuối kỳ 2022.2 - #526d03 & Cuối kỳ 2024.1 - #115355)

Cho chương trình con `count` viết bằng hợp ngữ RISC-V sau:

```
count:  addi s0, zero, 0
L1:     beq  a0, zero, done
        andi t0, a0, 1
```

```
        beq  t0, zero, L2
        addi s0, s0, 1
L2:     srli a0, a0, 1
        jal  x0, L1
done:   add  a1, s0, zero
        jalr x0, ra, 0
```

Giả thiết trước khi thực hiện chương trình con `count`, thanh ghi `a0 = 0x5678ABCD`. Hãy cho biết nội dung thanh ghi kết quả `a1` (hoặc `$v0`) sau khi thực hiện chương trình con đó (viết theo số thập phân)?

Hướng dẫn giải:

- **Hiểu thuật toán của hàm `count`:**
  - Trong mỗi vòng lặp, lệnh `andi t0, a0, 1` lấy ra bit cuối cùng bên phải của `a0`.
  - Nếu bit đó bằng 1 (khác 0), lệnh `addi s0, s0, 1` tăng biến đếm `s0` lên 1.
  - Sau đó lệnh `srli a0, a0, 1` dịch phải `a0` đi 1 bit để xét bit tiếp theo, lặp lại cho đến khi `a0` $== 0$.
  - **Kết luận:** Chương trình con này dùng để **đếm tổng số bit 1** có trong biểu diễn nhị phân của thanh ghi `a0`!
- **Đếm nhanh số bit 1 trong `a0 = 0x5678ABCD`:**
  - $5 = 0101_2 \rightarrow$ **2** bit 1 | $A = 1010_2 \rightarrow$ **2** bit 1
  - $6 = 0110_2 \rightarrow$ **2** bit 1 | $B = 1011_2 \rightarrow$ **3** bit 1
  - $7 = 0111_2 \rightarrow$ **3** bit 1 | $C = 1100_2 \rightarrow$ **2** bit 1
  - $8 = 1000_2 \rightarrow$ **1** bit 1 | $D = 1101_2 \rightarrow$ **3** bit 1
- Cộng tổng số bit 1: $(2 + 2 + 3 + 1) + (2 + 3 + 2 + 3) = 8 + 10 =$ **18**.
- **Đáp số: 18.**

Bài tập 4.45 (Tổng hợp 6 bài toán phân tích Vòng lặp trong Đề Cuối kỳ 2021.2 - Mã đề B1 đến B7)

Hãy xác định tổng số lệnh thực thi và giá trị thanh ghi sau khi kết thúc các vòng lặp dưới đây:

a) **(Mã đề B1, Câu 2):** Khởi tạo `addi t1, zero, 100`. Vòng lặp `LOOP` gồm 5 lệnh: `lw s1, 0(s0)` → `add s2, s2, s1` → `addi s0, s0, 4` → `addi t1, t1, -1` → `bne t1, x0, LOOP`. Có bao nhiêu lệnh được thực hiện và chuyển sang C?

b) **(Mã đề B5, Câu 2):** Khởi tạo `addi t1, s0, 600`. Vòng lặp `LOOP` gồm 6 lệnh: `lw s1, 0(s0)` → `add s2, s2, s1` → `lw s1, 4(s0)` → `add s2, s2, s1` → `addi s0, s0, 8` → `bne t1, s0, LOOP`. Có bao nhiêu lệnh được thực hiện?

c) **(Mã đề B2 & B3, Câu 3):** Cho vòng lặp:

```
LOOP: slt  t2, zero, t1
      bne  t2, zero, ELSE
      jal  x0, DONE
ELSE: addi s2, s2, 2
      addi t1, t1, -1
      jal  x0, LOOP
DONE:
```

Tính `s2` khi khởi tạo `t1` $= 20$, `s2` $= 0$ (Đề B2) hoặc `t1` $= 18$, `s2` $= 0$ (Đề B3). Tổng quát với `t1` $= N$ ($N > 0$), có bao nhiêu lệnh được thực hiện?

d) **(Mã đề B4, Câu 3):** Cho 2 vòng lặp lồng nhau:

```
LOOP:  addi t2, zero, 0xA
LOOP2: addi s2, s2, 14
       addi t2, t2, -1
       bne  t2, x0, LOOP2
       addi t1, t1, -1
       bne  t1, x0, LOOP
```

Tính `s2` khi ban đầu `t1` $= 18$, `s2` $= 0$. Tổng quát với `t1` $= N$, có bao nhiêu lệnh được thực hiện?

e) **(Mã đề B6 & B7, Câu 2):** Khởi tạo `addi t1, zero, K` và `add s2, x0, x0`. Vòng lặp `LOOP` gồm 6 lệnh: `slt t2, x0, t1` → `beq t2, x0, DONE` → `addi s2, s2, M` → `slli s2, s2, L` → `addi t1, t1, -1` → `jal x0, LOOP`. Tính tổng số lệnh thực thi và giá trị `s2` với:

- Đề B6: $K = 5, M = 3, L = 2$.
- Đề B7: $K = 7, M = 1, L = 1$.

Hướng dẫn giải:

- **Giải ý a) (Đề B1):**
  - Có 1 lệnh khởi tạo bên ngoài + 100 vòng lặp × 5 lệnh/vòng = **501** lệnh.
  - Câu lệnh C tương ứng (tính tổng 100 phần tử đầu của mảng `A`): `for (i = 0; i < 100; i++) a = a + A[i];`.
- **Giải ý b) (Đề B5):**
  - Mỗi vòng lặp địa chỉ `s0` tăng thêm 8 byte. Để tăng thêm 600 byte, số vòng lặp là $600 \div 8 = 75$ vòng.
  - Tổng số lệnh thực hiện: 1 lệnh ngoài + $75 \times 6 =$ **451** lệnh.
- **Giải ý c) (Đề B2 & B3):**
  - Mỗi vòng lặp khi `t1` > 0, thanh ghi `s2` được cộng thêm 2. Vậy với `t1` $= 20 \implies$ `s2` $= 20 \times 2 =$ **40** (Đề B2); với `t1` $= 18 \implies$ `s2` $= 18 \times 2 =$ **36** (Đề B3).

  - Đếm số lệnh theo N: Trong N lần lặp đầu (khi `t1` $= N, N-1, \ldots, 1$), mỗi lần thực hiện 5 lệnh (`slt, bne, addi, addi, jal LOOP`). Đến lần kiểm tra cuối cùng khi `t1` $= 0$, thực hiện thêm 3 lệnh (`slt, bne, jal DONE`) để thoát. Vậy tổng số lệnh là: $\mathbf{5N + 3}$ lệnh.

- **Giải ý d) (Đề B4):**
  - Vòng lặp trong (`LOOP2`) lặp `0xA` $= 10$ lần, mỗi lần cộng 14 vào `s2` (tức mỗi vòng ngoài tăng $10 \times 14 = 140$). Với `t1` $= 18$ vòng ngoài $\implies$ `s2` $= 18 \times 140 =$ **2520**.
  - Đếm số lệnh theo N: Mỗi vòng ngoài gồm 1 lệnh (`addi t2`) + 10×3 lệnh (trong `LOOP2`) + 2 lệnh cuối vòng ngoài = 33 lệnh. Lặp N vòng ngoài sẽ thực hiện: **33N** lệnh.

- **Giải ý e) (Đề B6 & B7):**
  - Công thức đếm số lệnh với K vòng lặp: 2 lệnh khởi tạo ngoài + $K \times 6$ lệnh trong vòng lặp + 2 lệnh kiểm tra thoát (`slt, beq`) $= 6K + 4$ lệnh.
  - **Với Đề B6 ($K = 5$):** Số lệnh $= 6 \times 5 + 4 =$ **34** lệnh. Tính `s2` qua 5 vòng (mỗi vòng cộng 3 rồi nhân 4): Vòng 1: $(0+3) \times 4 = 12$; Vòng 2: $(12+3) \times 4 = 60$; Vòng 3: $(60+3) \times 4 = 252$; Vòng 4: $(252+3) \times 4 = 1020$; Vòng 5: $(1020+3) \times 4 =$ **4092**.
  - **Với Đề B7 ($K = 7$):** Số lệnh $= 6 \times 7 + 4 =$ **46** lệnh. Tính `s2` qua 7 vòng (mỗi vòng cộng 1 rồi nhân 2): Vòng 1: 2; Vòng 2: 6; Vòng 3: 14; Vòng 4: 30; Vòng 5: 62; Vòng 6: 126; Vòng 7: **254** (tổng quát là $2^{K+1} - 2$).

4.2.7 Nhóm 7: Chương trình con, Ngăn xếp (Stack) và Lệnh `jal` / `jalr`

Bài tập 4.46 (Đề Giữa kỳ 2024.1 - #5ebf59 & Ôn tập Giữa kỳ - Câu 22)

Những lệnh nào trong RISC-V lưu địa chỉ PC + 4 vào một thanh ghi trước khi thực hiện rẽ nhánh/nhảy?

A. `jal` B. `j` C. `beq` D. `jalr` E. `jr`

Hướng dẫn giải:

- Hai lệnh nhảy và liên kết (Jump and Link) dùng để gọi chương trình con và lưu địa chỉ trở về PC + 4 vào thanh ghi đích là `jal` và `jalr`.
- **Đáp án đúng: Chọn cả A (`jal`) và D (`jalr`).**

Bài tập 4.47 (Đề Giữa kỳ 2024.1 - #907ec8 & Ôn tập Giữa kỳ - Câu 46, Câu 49)

a) (Câu #907ec8 & Câu 46): Ngăn xếp (stack) của RISC-V phát triển theo chiều giảm địa chỉ bộ nhớ. Đoạn mã nào sau đây phù hợp để cấp phát và lưu 1 word (`s0`) vào ngăn xếp?

- A) `addi sp, sp, -4` → `sw s0, 0(sp)` B) `addi sp, sp, 0` → `sw s0, 0(sp)`
- C) `addi sp, sp, 4` → `sw s0, 0(sp)` D) `addi sp, sp, -8` → `sw s0, 4(sp)`

b) (Câu 49): Phát biểu nào đúng về hoạt động của con trỏ ngăn xếp SP (Stack Pointer)?

- A. Khi cất thêm vào ngăn xếp, nội dung của SP được tăng.
- B. SP là thanh ghi chứa địa chỉ của ngăn nhớ đáy ngăn xếp.
- C. SP nằm ở bộ nhớ chính.
- D. Không có phát biểu nào đúng.

Hướng dẫn giải:

- **Giải ý a):** Để lưu 1 word (4 byte) vào ngăn xếp, ta phải giảm con trỏ `sp` đi 4 byte (`addi sp, sp, -4`) rồi lưu vào đỉnh ngăn xếp mới (`sw s0, 0(sp)`) → Chọn **A**.
- **Giải ý b):**
  - A sai vì khi cất thêm vào ngăn xếp, nội dung của SP phải **giảm**.
  - B sai vì SP chứa địa chỉ của **đỉnh** ngăn xếp (không phải đáy).
  - C sai vì SP là thanh ghi nằm bên trong **CPU** (không nằm ở bộ nhớ chính).
  - Vậy **Đáp án đúng là D (Không có phát biểu nào đúng)**.

Bài tập 4.48 (Đề Cuối kỳ 2022.2 - #a3c257, 2024.1 - #e79a4d & Ôn tập Cuối kỳ P2 - Câu 48)

Cho thân chương trình con hợp ngữ RISC-V gồm có 3 đoạn lệnh như dưới đây:

```
# Đoạn lệnh cất nội dung các thanh ghi s2 (x18) và s3 (x19) ra ngăn xếp:
    addi sp, sp, -8
    sw   s3, 4(sp)
    sw   s2, 0(sp)
# Đoạn lệnh không liên quan đến sp và ngăn xếp:
    ...
# Đoạn lệnh khôi phục lại nội dung s2, s3 và sp:
```

Hãy chỉ ra đoạn lệnh phù hợp để khôi phục lại nội dung `s2 (x18)`, `s3 (x19)` và `sp`?

A. `lw s2, 0(sp)` → `lw s3, 4(sp)` → `addi sp, sp, 8`

B. `addi sp, sp, 8` → `lw s3, 4(sp)` → `lw s2, 0(sp)`

C. `lw s2, 4(sp)` → `lw s3, 0(sp)` → `addi sp, sp, 8`

D. `addi sp, sp, 8` → `lw s3, 0(sp)` → `lw s2, 4(sp)`

Hướng dẫn giải:

- **Nguyên tắc khôi phục ngăn xếp:** Lúc cất vào ở độ lệch nào so với `sp` thì lúc đọc ra (`lw`) phải lấy ở đúng độ lệch đó **trước khi** cộng trả lại `sp`.
- Ở đầu hàm, ta cất `s3` vào `4(sp)` và cất `s2` vào `0(sp)`.
- Vậy khi khôi phục, ta phải đọc `lw s2, 0(sp)` và `lw s3, 4(sp)` trước, rồi cuối cùng mới giải phóng ngăn xếp bằng lệnh `addi sp, sp, 8`. (Phương án B và D cộng `sp` trước khi `lw` nên đọc sai địa chỉ; Phương án C đọc nhầm chéo giá trị của `s2` và `s3`).
- **Đáp án đúng: A (Tùy chọn 1).**

Bài tập 4.49 (Đề Cuối kỳ 2022.2 - #9aec2f, 2024.1 - #f023de & Ôn tập Cuối kỳ P2 - Câu 40)

Cho đoạn lệnh RISC-V như dưới đây, bên trái là địa chỉ của lệnh tương ứng:

```
0x00402034          jal  x1, XYZ
0x00402038          ...
...
0x00402060  XYZ:    addi sp, sp, -8
...
0x00402090          jalr x0, 0(x1)
```

Hãy cho biết các giá trị sau (viết theo số Hexa 8 chữ số):

a) Nội dung thanh ghi `PC` và thanh ghi `x1 (ra)` ngay sau khi thực hiện lệnh `jal x1, XYZ`?

b) Nội dung thanh ghi `PC` sau khi thực hiện lệnh `jalr x0, 0(x1)`?

Hướng dẫn giải:

- **Giải ý a) Ngay sau khi thực hiện lệnh `jal x1, XYZ` (tại địa chỉ `0x00402034`):**
  - Lệnh `jal` nhảy tới nhãn `XYZ`, nên thanh ghi `PC` nhận địa chỉ của nhãn `XYZ`:

    $$\texttt{PC} = \texttt{0x00402060}$$

  - Đồng thời, lệnh `jal` cất địa chỉ của câu lệnh kế tiếp ($\text{PC}_{\text{jal}} + 4 = \texttt{0x00402034} + 4$) vào thanh ghi `x1 (ra)`:

    $$\texttt{x1 (ra)} = \texttt{0x00402038}$$

- **Giải ý b) Ngay sau khi thực hiện lệnh trở về `jalr x0, 0(x1)`:**

  - Lệnh `jalr x0, 0(x1)` nạp giá trị trong thanh ghi `x1` (`0x00402038`) trả lại cho thanh ghi `PC` để quay về chương trình chính:

$$\texttt{PC} = \texttt{0x00402038}$$

- **Đáp số tổng hợp (như trong Câu 40 - Ôn tập P2):** `0x00402060/0x00402038/0x00402038`.

4.2.8 Nhóm 8: Dịch Mã máy ↔ Hợp ngữ, Tính địa chỉ đích và Lập trình Vào-Ra

Bài tập 4.50 (Đề Cuối kỳ 2023.1 - #9f7fd7 & Ôn tập Cuối kỳ P2 - Câu 60)

Lệnh nào sau đây trong RISC-V yêu cầu trường `opcode`?

A. `ori` B. `sw` C. `j` (`jal`) D. `bge` (`beq`)

Hướng dẫn giải:

- **Mọi câu lệnh mã máy** của RISC-V (cả 6 khuôn dạng R, I, S, B, U, J) đều bắt buộc phải có trường `opcode` (7 bit từ bit 6 đến bit 0) để bộ điều khiển biết lệnh đó thuộc nhóm thao tác nào.
- **Đáp án đúng: Chọn tất cả các lệnh (A, B, C, D).**

Bài tập 4.51 (Đề Giữa kỳ 2024.1 - #b744e1)

Những lệnh nào sau đây trong RISC-V có trường hằng số (immediate field) dài **20-bit**?

1. `lui` 2. `auipc` 3. `jal` 4. `sw` 5. `lw` 6. `addi` 7. `beq`

Hướng dẫn giải:

- Nhìn vào bảng 6 khuôn dạng lệnh của RISC-V:
  - Các lệnh kiểu I (`lw, addi`), kiểu S (`sw`), và kiểu B (`beq`) đều chỉ mã hóa **12 bit** của trường `imm` trong lệnh máy.
  - Chỉ có các lệnh **kiểu U** (`lui`, `auipc`) và **kiểu J** (`jal`) mới có trường `imm` dài **20 bit** (từ bit 31 đến bit 12).
- **Đáp án đúng: Chọn `lui`, `auipc` và `jal`.**

Bài tập 4.52 (Đề Giữa kỳ 2024.1 - #1c2f2d, Ôn tập Giữa kỳ - Câu 50 & Ôn tập Cuối kỳ P2 - Câu 37)

a) (Câu #1c2f2d & Câu 50): Cho lệnh máy kiểu R của RISC-V ở dạng Hexa: `0x007302b3`. Toán hạng đích (destination operand `rd`) của lệnh này là gì?

- A. `x10` B. `x7` C. `x5` D. `x6`

b) (Ôn tập P2 - Câu 37): Cho mã máy của một lệnh trong RISC-V: `0000000 00111 00000 000 11111 0110011`. Toán hạng đích của nó là gì?

- A. Thanh ghi `zero` B. Thanh ghi `t6` C. Thanh ghi `PC` D. Đáp án khác

Hướng dẫn giải:

- **Mẹo tìm nhanh thanh ghi đích `rd` (bit 11:7):** Trong mọi lệnh có thanh ghi đích `rd` (kiểu R, I, U, J), trường `rd` luôn nằm ở **bit 11 đến bit 7** (ngay bên trái 7 bit opcode).
- **Giải ý a) Với `0x007302b3`:**
  - Đổi 3 chữ số Hexa cuối cùng (`2b3`) sang nhị phân 12-bit:

$$\texttt{2b3}_{(16)} = \texttt{0010 1011 0011}_{(2)} = \underbrace{\texttt{00101}}_{\text{rd (5 bit)}} \quad \underbrace{\texttt{0110011}}_{\text{opcode (7 bit)}}$$

  - Ta thấy $\texttt{rd} = \texttt{00101}_{(2)} = 4 + 1 = 5 \implies$ Thanh ghi đích là `x5` (tức `t0`) $\rightarrow$ Chọn **C** (`x5`).
- **Giải ý b):**
  - Đề đã tách sẵn các trường theo đúng thứ tự kiểu R: `funct7 | rs2 | rs1 | funct3 | rd | opcode`.
  - Trường `rd` đứng thứ hai từ phải sang là $\texttt{11111}_{(2)} = 31 \implies$ Thanh ghi `x31`, tên gợi nhớ là **Thanh ghi `t6`** $\rightarrow$ Chọn **B**.

Bài tập 4.53 (Đề Giữa kỳ 2024.1 - #65dce1 & Ôn tập Giữa kỳ - Câu 59)

Cho một lệnh rẽ nhánh `beq` có mã máy là `0x01ee8463`. Các thanh ghi nguồn (`rs1, rs2`) của câu lệnh này là gì?

A. `x30, x31` B. `x29, x29` C. `x5, x6` D. `x29, x28` E. `x29, x30`

Hướng dẫn giải:

- Cú pháp lệnh `beq` là: `beq rs1, rs2, label`. Nhưng trong mã máy 32-bit (từ trái sang phải), vị trí của chúng là:

$\underbrace{\texttt{imm[12,10:5]}}_{\text{7 bit (31:25)}}$	$\underbrace{\texttt{rs2}}_{\text{5 bit (24:20)}}$	$\underbrace{\texttt{rs1}}_{\text{5 bit (19:15)}}$	$\underbrace{\texttt{funct3}}_{\text{3 bit (14:12)}}$	$\underbrace{\texttt{imm[4:1,11]}}_{\text{5 bit (11:7)}}$	$\underbrace{\texttt{opcode}}_{\text{7 bit (6:0)}}$

– Đổi mã máy `0x01ee8463` sang nhị phân:

$$\texttt{0x01ee8463} = \texttt{0000 0001 1110 1110 1000 0100 0110 0011}_{(2)}$$

– Tách theo đúng số bit từ trái sang phải (7 - 5 - 5 - 3 - 5 - 7):

$$\underbrace{0000000}_{\text{7 bit}} \mid \underbrace{11110}_{\texttt{rs2 = 30}} \mid \underbrace{11101}_{\texttt{rs1 = 29}} \mid \underbrace{000}_{\text{funct3}} \mid \underbrace{01000}_{\text{5 bit}} \mid \underbrace{1100011}_{\text{opcode}}$$

– Vậy $\texttt{rs1} = 11101_{(2)} = 29$ (`x29`) và $\texttt{rs2} = 11110_{(2)} = 30$ (`x30`).

– **Đáp án đúng: E (`x29, x30`).**

Bài tập 4.54 (Đề Cuối kỳ 2024.1 - #b9e098 & Ôn tập Giữa kỳ - Câu 7)

Cho lệnh máy RISC-V được biểu diễn theo số Hexa: `0x012B2423`. Hãy chỉ ra lệnh hợp ngữ tương ứng?

A. `add t0, s4, s5`

B. `lw t2, 4(s6)`

C. `sw s2, 8(s6)`

D. `addi t0, s3, -4`

Hướng dẫn giải:

– Bước 1 – Xác định `opcode` từ 2 chữ số Hexa cuối (`23`):

$$\texttt{0x23} = \texttt{0010 0011}_{(2)} \implies \text{7 bit cuối là } \texttt{0100011}_{(2)} = 35_{(10)}$$

Tra bảng opcode, `0100011` là mã của nhóm lệnh **Store (Kiểu S)**.

– Bước 2 – Đổi toàn bộ `0x012B2423` sang nhị phân và tách theo khuôn dạng S-type (7 - 5 - 5 - 3 - 5 - 7):

$$\texttt{0x012B2423} = \underbrace{0000000}_{\texttt{imm[11:5]=0}} \mid \underbrace{10010}_{\texttt{rs2=18 (s2)}} \mid \underbrace{10110}_{\texttt{rs1=22 (s6)}} \mid \underbrace{010}_{\texttt{funct3=2 (sw)}} \mid \underbrace{01000}_{\texttt{imm[4:0]=8}} \mid \underbrace{0100011}_{\texttt{opcode}}$$

– Vậy lệnh hợp ngữ tương ứng là: `sw s2, 8(s6)` (hoặc nếu câu hỏi chỉ hỏi tên lệnh như ở Câu 7 Ôn tập GK thì chọn `sw`).

– **Đáp án đúng: C (`sw s2, 8(s6)`).**

Bài tập 4.55 (Tổng hợp 6 câu dịch Hợp ngữ sang Mã máy Hexa trong Ôn tập Giữa kỳ - Câu 12, 26, 29, 55, 57, 66, 71)

Hãy dịch các câu lệnh hợp ngữ RISC-V sau sang mã máy biểu diễn ở hệ Hexa (8 chữ số, có `0x` ở đầu):

a) **(Câu 55 – Kiểu R):** `sub t0, t2, t1`

b) **(Câu 71 – Kiểu I):** `addi s2, s1, -14`

c) **(Câu 12 – Kiểu I Load):** `lh t1, -13(s2)`

d) **(Câu 66 – Kiểu S Store):** `sw s11, -16(s4)`

e) **(Câu 29 – Kiểu B Branch xuôi):** `beq s0, t5, L1` (biết địa chỉ lệnh là `0x00401050`, địa chỉ `L1` là `0x00401060`).

f) **(Câu 57 – Kiểu B Branch ngược):** `bne t0, s0, L2` (biết địa chỉ lệnh là `0x0040008C`, địa chỉ `L2` là `0x00400070`).

g) **(Câu 26 – Kiểu J Jump):** `jal x0, L1` (biết địa chỉ lệnh là `0x004030AC`, địa chỉ `L1` là `0x00403074`).

Hướng dẫn giải:

Giải ý a) `sub t0, t2, t1` **(Kiểu R):**

- Tra số hiệu thanh ghi: `rd` = `t0` = `x5` (`00101`); `rs1` = `t2` = `x7` (`00111`); `rs2` = `t1` = `x6` (`00110`).
- Lệnh `sub` có `opcode` = `0110011`, `funct3` = `000`, `funct7` = `0100000`.
- Ghép 32 bit (`funct7 | rs2 | rs1 | funct3 | rd | op`):

  $$0100000\ 00110\ 00111\ 000\ 00101\ 0110011 = 0100\ 0000\ 0110\ 0011\ 1000\ 0010\ 1011\ 0011_{(2)} = 0$$

Giải ý b) `addi s2, s1, -14` **(Kiểu I):**

- Số hiệu: `rd` = `s2` = `x18` (`10010`); `rs1` = `s1` = `x9` (`01001`); `funct3` = `000`; `op` = `0010011`.
- Đổi `imm` = -14 sang bù 2 12-bit: $+14 = 0000\ 0000\ 1110_2 \implies -14 = 1111\ 1111\ 0010_2 = 0xFF2$.
- Ghép 32 bit (`imm[11:0] | rs1 | funct3 | rd | op`):

  $$1111\ 1111\ 0010\ 01001\ 000\ 10010\ 0010011 = 1111\ 1111\ 0010\ 0100\ 1000\ 1001\ 0001\ 0011_{(2)} =$$

Giải ý c) `lh t1, -13(s2)` **(Kiểu I Load):**

- Số hiệu: `rd` = `t1` = `x6` (`00110`); `rs1` = `s2` = `x18` (`10010`); `funct3` = `001` (`lh`); `op` = `0000011`.
- Đổi `imm` = -13 sang bù 2 12-bit: $+13 = 0000\ 0000\ 1101_2 \implies -13 = 1111\ 1111\ 0011_2 = 0xFF3$.

– Ghép 32 bit:

$$\texttt{1111 1111 0011 10010 001 00110 0000011} = \texttt{1111 1111 0011 1001 0001 0011 0000 0011}_{(2)} =$$

Giải ý d) `sw s11, -16(s4)` **(Kiểu S Store):**

– Số hiệu: `rs2` = `s11` = `x27` (11011); `rs1` = `s4` = `x20` (10100); `funct3` = `010`; `op` = `0100011`.

– Đổi `imm` = −16 sang bù 2 12-bit: $\texttt{1111 1111 0000}_2 \implies$ `imm[11:5]` = `1111111`, `imm[4:0]` = `10000`.

– Ghép 32 bit (`imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | op`):

$$\texttt{1111111 11011 10100 010 10000 0100011} = \texttt{1111 1111 1011 1010 0010 1000 0010 0011}_{(2)} = 0$$

Giải ý e) `beq s0, t5, L1` **(Slide trang 114):**

– Tính độ lệch: `imm` = `0x00401060` − `0x00401050` = `0x10` = +16 = $\texttt{0 0 000000 1000 0}_{(2)}$ (bit 12 đến bit 0).

– Số hiệu: `rs1` = `s0` = `x8` (01000); `rs2` = `t5` = `x30` (11110); `funct3` = `000`; `op` = `1100011`.

– Ghép theo B-type (`imm[12,10:5] | rs2 | rs1 | funct3 | imm[4:1,11] | op`):

$$\texttt{0 000000 11110 01000 000 1000 0 1100011} = \texttt{0x01E40863}$$

Giải ý f) `bne t0, s0, L2` **(Slide trang 115):**

– Tính độ lệch: `imm` = `0x00400070` − `0x0040008C` = −`0x1C` = −28 = $\texttt{1 1 111111 0010 0}_{(2)}$ (bit 12 đến bit 0).

– Số hiệu: `rs1` = `t0` = `x5` (00101); `rs2` = `s0` = `x8` (01000); `funct3` = `001`; `op` = `1100011`.

– Ghép theo B-type:

$$\texttt{1 111111 01000 00101 001 0010 1 1100011} = \texttt{0xFE8292E3}$$

Giải ý g) `jal x0, L1` **(Slide trang 119):**

– Tính độ lệch: `imm` = `0x00403074` − `0x004030AC` = −`0x38` = −56 = $\texttt{1 11111111 1 1111100100 0}_{(2)}$ (bit 20 đến bit 0).

– Ghép theo J-type (`imm[20,10:1,11,19:12] | rd | op`):

$$\texttt{1 1111100100 1 11111111 00000 1101111} = \texttt{0xFC9FF06F}$$

Bài tập 4.56 (Đề Cuối kỳ 2024.1 - #3ad10f, Giữa kỳ 2024.1 - #a2a36c & Ôn tập Giữa kỳ - Câu 28)

a) **(Đề CK 2024.1 - #3ad10f):** Giả thiết lệnh `bge s0, s1, L1` của RISC-V có mã máy là `0x02945063`, được đặt tại word nhớ có địa chỉ `0x00404014`. Hãy chỉ ra địa chỉ đích của lệnh đó (nếu điều kiện thỏa mãn). Biết BTA = PC + SignExt($\{\text{imm}_{12:1}, 1'b0\}$).

- A. `0x0040401F` B. `0x00404034` C. `0x00404030` D. `0x00404018`

b) **(Ôn tập GK - Câu 28):** Giả sử `PC` hiện tại (địa chỉ của lệnh `beq`) là `0x2000`. Với câu lệnh `(0x2000) beq x1, x2, -12`, nếu `x1` == `x2` thì địa chỉ lệnh tiếp theo (Target PC) sẽ là bao nhiêu?

- A. `0x1FF4` B. `0x1FF8` C. `0x200C` D. `0x1FE8`

c) **(Đề GK 2024.1 - #a2a36c):** Cho chương trình RISC-V bắt đầu từ địa chỉ `0x00400000`:

```
        addi s0, zero, 100    # 0x00400000
        slti t0, s0, -100     # 0x00400004
        beq  t0, zero, else   # 0x00400008
        addi t2, t2, 1        # 0x0040000C
        j    endif            # 0x00400010
else:   addi t2, t2, -1       # 0x00400014
endif:  nop                   # 0x00400018
```

Hỏi giá trị thập phân của trường hằng số 12-bit (`imm[12:1]`) hoặc độ lệch byte trong lệnh `beq t0, zero, else` là bao nhiêu? (Các đáp án: 8, 3, 2, 6).

Hướng dẫn giải:

- **Giải ý a):**
  - Đổi mã máy `0x02945063` sang nhị phân:

$$\underbrace{0\ 000001}_{\text{imm[12,10:5]}}\ \Big|\ \underbrace{01001}_{\text{rs2}}\ \Big|\ \underbrace{01000}_{\text{rs1}}\ \Big|\ \underbrace{101}_{\text{funct3}}\ \Big|\ \underbrace{0000\ 0}_{\text{imm[4:1,11]}}\ \Big|\ \underbrace{1100011}_{\text{opcode}}$$

  - Ghép lại các bit của $\text{imm}_{12:0} = \{\text{imm}_{12}, \text{imm}_{11}, \text{imm}_{10:5}, \text{imm}_{4:1}, 0\}$:

$$\text{imm} = 0\ \ 0\ \ 000001\ \ 0000\ \ 0_{(2)} = \texttt{0x20} = 32_{(10)}$$

  - Cộng vào địa chỉ lệnh `PC = 0x00404014`:

$$\text{BTA} = \texttt{0x00404014} + \texttt{0x20} = \texttt{0x00404034} \implies \text{Chọn } \mathbf{B}.$$

- **Giải ý b):**

* Trong RISC-V, độ lệch cộng trực tiếp vào PC = `0x2000`. Với độ lệch $-12 = -$`0x0C`:

$$\text{Target PC} = \texttt{0x2000} - \texttt{0x0C} = \texttt{0x1FF4} \implies \text{Chọn } \mathbf{A}.$$

– **Giải ý c):**

* Lệnh `beq` nằm ở địa chỉ `0x00400008`, nhãn `else` nằm ở địa chỉ `0x00400014`.
* Khoảng cách tính theo byte từ `beq` đến `else` là: `0x14` − `0x08` = 12 byte (tương ứng 3 lệnh).
* Vì 12 bit lưu trong mã máy của lệnh `beq` là các bit imm[12 : 1] (đã bỏ đi bit 0 bằng 0, tức chia 2), nên giá trị thập phân của 12 bit đó là: $12 \div 2 = \mathbf{6} \implies$ Chọn **6**.

Bài tập 4.57 (Đề Cuối kỳ 2022.2 - #a1e199, 2024.1 - Trang 3 & Ôn tập Giữa kỳ - Câu 8)

Cho máy tính dùng bộ xử lý theo kiến trúc RISC-V. Giả sử hệ thống có 04 cổng vào-ra 32-bit được gán các địa chỉ tương ứng là:

– `P1` ở địa chỉ `0xFFFF000C`

– `P2` ở địa chỉ `0xFFFF0010`

– `P3` ở địa chỉ `0xFFFF0014`

– `P4` ở địa chỉ `0xFFFF0018`

Hãy cho biết đoạn chương trình hợp ngữ RISC-V dưới đây làm gì?

```
lui s0, 0xFFFF0
lw  t0, 0x0C(s0)
sw  t0, 0x18(s0)
lw  t1, 0x10(s0)
sw  t1, 0x14(s0)
```

A. Nhận dữ liệu từ cổng P1 đưa ra cổng P2, nhận dữ liệu từ P3 đưa ra cổng P4

B. Nhận dữ liệu từ cổng P1 đưa ra cổng P4, nhận dữ liệu từ P2 đưa ra cổng P3

C. Nhận dữ liệu từ cổng P1 đưa ra cổng P3, nhận dữ liệu từ P2 đưa ra cổng P4

D. Nhận dữ liệu từ cổng P4 đưa ra cổng P1, nhận dữ liệu từ P3 đưa ra cổng P2

Hướng dẫn giải:

– Lệnh 1 (`lui s0, 0xFFFF0`): Khởi tạo địa chỉ cơ sở `s0` = `0xFFFF0000`.

– Lệnh 2 (`lw t0, 0x0C(s0)`): Đọc (nhận) dữ liệu từ địa chỉ `0xFFFF000C` (cổng **P1**) vào `t0`.

- Lệnh 3 (`sw t0, 0x18(s0)`): Ghi (đưa) dữ liệu trong `t0` ra địa chỉ `0xFFFF0018` (cổng **P4**).
- Lệnh 4 (`lw t1, 0x10(s0)`): Đọc (nhận) dữ liệu từ địa chỉ `0xFFFF0010` (cổng **P2**) vào `t1`.
- Lệnh 5 (`sw t1, 0x14(s0)`): Ghi (đưa) dữ liệu trong `t1` ra địa chỉ `0xFFFF0014` (cổng **P3**).
- **Đáp án đúng: B (Nhận dữ liệu từ cổng P1 đưa ra cổng P4, nhận dữ liệu từ P2 đưa ra cổng P3).**
- *Lưu ý:* Trong đề 2022.2 cũ, thứ tự 2 cặp lệnh được đảo lại là đọc `0x10` (P2) ghi ra `0x18` (P4), đọc `0x0C` (P1) ghi ra `0x14` (P3); bạn chỉ cần đối chiếu đúng số đuôi `0C=P1, 10=P2, 14=P3, 18=P4` là luôn chọn đúng 100%.

Bài tập 4.58 (Đề Cuối kỳ 2023.1 - #90c4ff & Ôn tập Cuối kỳ P2 - Câu 55)

Trong đoạn chương trình dưới đây, CPU đang cố gắng đọc một byte từ thiết bị I/O. Phương pháp lập trình I/O nào đang được sử dụng?

```
.text
        li    s0, 0xFFFF0004      # Data register
        li    s1, 0xFFFF0000      # Status register
Check:  lw    t1, 0(s1)
        andi  t1, t1, 0x1
        beq   t1, zero, Check
        lb    t0, 0(s0)           # read data from device
```

A. Polling (Thăm dò / Hỏi vòng)

B. Overlapping I/O (I/O chồng lấn)

C. Interrupt (Ngắt)

D. DMA (Truy cập bộ nhớ trực tiếp)

Hướng dẫn giải:

- Nhãn `Check` thực hiện đọc thanh ghi trạng thái (`0xFFFF0000`), kiểm tra bit sẵn sàng số 0 (`andi t1, t1, 0x1`). Nếu bit này bằng 0 (`beq t1, zero, Check`) thì CPU quay lại nhãn `Check` để hỏi tiếp liên tục cho đến khi thiết bị sẵn sàng mới đọc dữ liệu (`lb t0, 0(s0)`).
- Đây chính là phương pháp **Polling (Thăm dò / Vào-ra bằng chương trình)**.
- **Đáp án đúng: A (Polling).**

Bài tập 4.59 (Đề Giữa kỳ 2022.2 - Đề 0009 & 0008, Câu 1 – 7)

Cho bản đồ bộ nhớ (Memory map) lưu trữ theo thứ tự đầu nhỏ (Little-Endian) từ địa chỉ `0x400000` đến `0x400017` gồm 24 byte (tương ứng 6 lệnh máy 32-bit, vì $24 \div 4 = 6$).

- Tại địa chỉ `0x400010` có 4 byte liên tiếp là: `0x20` (`0x400010`), `0x48` (`0x400011`), `0x4B` (`0x400012`), `0x01` (`0x400013`).
- Tại địa chỉ `0x40000C` (Đề 0008) có 4 byte liên tiếp là: `0x01` (`0x40000C`), `0x00` (`0x40000D`), `0xA7` (`0x40000E`), `0x12` (`0x40000F`).

Hãy viết mã máy (ở hệ 16) tại địa chỉ `0x400010` và tại địa chỉ `0x40000C`, đồng thời cho biết bản đồ bộ nhớ 24 byte mô tả bao nhiêu lệnh mã máy?

Hướng dẫn giải:

- **Số lệnh mã máy trong bản đồ 24 byte:** Mỗi lệnh dài 4 byte, nên 24 byte mô tả $24 \div 4 = \mathbf{6}$ lệnh mã máy.
- **Ghép mã máy 32-bit từ 4 byte theo Little-Endian:** Byte ở địa chỉ lớn nhất đứng bên trái, byte ở địa chỉ nhỏ nhất đứng bên phải:
  - Mã máy ở địa chỉ `0x400010`: Ghép ngược từ `0x400013` về `0x400010` ta được: `0x014B4820`.
  - Mã máy ở địa chỉ `0x40000C` (Đề 0008): Ghép ngược từ `0x40000F` về `0x40000C` ta được: `0x12A70001`.

CHƯƠNG 5

SỐ HỌC MÁY TÍNH

5.1 Phần 1: Lý thuyết trọng tâm

5.1.1 Hệ đếm và Chuyển đổi giữa các hệ đếm

1. Ba hệ đếm cơ bản trong máy tính

- **Hệ thập phân (Decimal – Cơ số 10):** Gồm 10 chữ số từ `0` đến `9`. Dùng n chữ số thập phân biểu diễn được 10^n giá trị (từ 0 đến $10^n - 1$).
- **Hệ nhị phân (Binary – Cơ số 2):** Gồm 2 chữ số là `0` và `1` (mỗi chữ số gọi là 1 **bit**). Dùng n bit biểu diễn được 2^n giá trị khác nhau.
- **Hệ mười sáu (Hexadecimal – Cơ số 16):** Gồm 16 chữ số: `0-9` và `A` (10), `B` (11), `C` (12), `D` (13), `E` (14), `F` (15). Hệ Hexa dùng để viết gọn cho số nhị phân: **cứ 1 nhóm 4-bit nhị phân ứng với đúng 1 chữ số Hexa**.

2. Bảng quy đổi nhanh 4-bit Nhị phân – Hexa – Thập phân

Nhị phân	Hex	Hệ 10	Nhị phân	Hex	Hệ 10	Nhị phân	Hex	Hệ 10	Nhị phân	Hex
0000	0	0	0100	4	4	1000	8	8	1100	C
0001	1	1	0101	5	5	1001	9	9	1101	D
0010	2	2	0110	6	6	1010	A	10	1110	E
0011	3	3	0111	7	7	1011	B	11	1111	F

3. Cách đổi số thực thập phân (có phần nguyên và phần lẻ) sang nhị phân

Ví dụ cần đổi số $13.75_{(10)}$ sang hệ nhị phân (bước bắt buộc khi làm bài tập IEEE 754):

- **Phần nguyên (13):** Chia liên tiếp cho 2 lấy dư, hoặc phân tích thành tổng các lũy thừa của 2 $(128, 64, 32, 16, 8, 4, 2, 1)$:

$$13 = 8 + 4 + 1 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = \texttt{1101}_{(\mathbf{2})}$$

- **Phần lẻ (0.75):** Nhân liên tiếp phần lẻ với 2 và lấy phần nguyên theo chiều từ trên xuống (hoặc nhớ nhanh: $0.5 = \texttt{0.1}_2$; $0.25 = \texttt{0.01}_2$; $0.75 = 0.5 + 0.25 = \texttt{0.11}_2$; $0.125 = \texttt{0.001}_2$; $0.375 = \texttt{0.011}_2$; $0.625 = \texttt{0.101}_2$):

$$0.75 \times 2 = \mathbf{1}.5 \implies 0.5 \times 2 = \mathbf{1}.0 \implies 0.75_{(10)} = \texttt{0.11}_{(\mathbf{2})}$$

– Ghép lại: $13.75_{(10)} = \texttt{1101.11}_{(\mathbf{2})}$.

5.1.2 Đại số Boolean, Cổng logic và Mạch số cơ bản

1. Sáu cổng logic cơ bản cần nhận diện hình vẽ

– **Cổng AND ($F = A \cdot B$):** Hình chữ D (đầu phẳng, mũi tròn). Bằng 1 khi tất cả đầu vào bằng 1.

– **Cổng NAND ($F = \overline{A \cdot B}$):** Hình cổng AND có thêm **vòng tròn nhỏ (nút đảo)** ở đầu ra. Bằng 0 khi tất cả đầu vào bằng 1.

– **Cổng OR ($F = A + B$):** Hình đầu nhọn, đuôi cong lõm vào. Bằng 1 khi có ít nhất 1 đầu vào bằng 1.

– **Cổng NOR ($F = \overline{A + B}$):** Hình cổng OR có thêm **vòng tròn nhỏ** ở đầu ra.

– **Cổng XOR ($F = A \oplus B = A\overline{B} + \overline{A}B$):** Giống cổng OR nhưng có thêm **một vạch cong song song ở phần đuôi**. Bằng 1 khi hai đầu vào khác nhau.

– **Cổng NOT ($F = \overline{A}$ hay A'):** Hình tam giác có vòng tròn nhỏ ở mũi, dùng để đảo bit ($0 \leftrightarrow 1$).

2. Định lý De Morgan: $\overline{A \cdot B} = \overline{A} + \overline{B}$ và $\overline{A + B} = \overline{A} \cdot \overline{B}$.

3. Các mạch tổ hợp và mạch dãy cơ bản trong CPU

– **Mạch tổ hợp (Combinational Circuits – không có nhớ):** Đầu ra chỉ phụ thuộc vào đầu vào tại thời điểm hiện tại. Điển hình gồm:

  * **Bộ ghép kênh (MUX – Multiplexer):** Có 2^n đầu vào dữ liệu, n đầu vào chọn (S), và 1 đầu ra. Với MUX 2-ra-1: $F = D_0 \cdot \overline{S} + D_1 \cdot S$ (nếu $S = 0$ chọn D_0, nếu $S = 1$ chọn D_1).
  * **Bộ giải mã (Decoder):** Có n đầu vào và 2^n đầu ra, tại mỗi thời điểm chỉ có duy nhất 1 đầu ra tích cực.
  * **Bộ cộng toàn phần 1-bit (Full-Adder):** Cộng 3 bit (A, B, C_{in}) tạo ra bit tổng S và bit nhớ ra $C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}$. Ghép nối tiếp n bộ cộng 1-bit ta được bộ cộng n-bit.

– **Mạch dãy (Sequential Circuits – có nhớ):** Đầu ra phụ thuộc cả đầu vào hiện tại lẫn trạng thái quá khứ. Được xây dựng từ các phần tử nhớ **Flip-Flop** (SR, JK, D) để tạo thành **Thanh ghi (Register)** và Bộ đếm.

5.1.3 Biểu diễn Số nguyên không dấu và Số nguyên có dấu (Mã bù 2)

1. Số nguyên không dấu (Unsigned Integer) n-bit

- Mọi bit đều dùng để biểu diễn độ lớn: $A = \sum_{i=0}^{n-1} a_i 2^i$.
- **Dải giá trị:** Từ 0 đến $2^n - 1$. (Với 8 bit: từ 0 đến 255; với 16 bit: từ 0 đến 65535).

2. Số nguyên có dấu (Signed Integer) theo Mã bù hai (Two's Complement)

Trong máy tính hiện đại và RISC-V, số nguyên có dấu luôn được biểu diễn bằng **Mã bù hai**. Bit ngoài cùng bên trái (a_{n-1}) là **bit dấu (Sign bit)**:

- Nếu **bit dấu** = 0: Đó là **số dương** (hoặc bằng 0). Giá trị tính bình thường như số không dấu.
- Nếu **bit dấu** = 1: Đó là **số âm**.

Quy tắc 2 bước tìm Mã bù hai của một số âm (hoặc đổi từ số âm bù 2 về độ lớn):

1. **Bước 1 (Tìm bù 1):** Đảo tất cả các bit của số dương tương ứng (bit `0` đổi thành `1`, bit `1` đổi thành `0`).
2. **Bước 2 (Tìm bù 2):** Cộng thêm **1** vào bit cuối cùng bên phải của số bù 1.

Công thức tính nhanh giá trị của số nguyên có dấu 8-bit ($a_7a_6a_5a_4a_3a_2a_1a_0$):

Thay vì đảo bit cộng 1, với số 8-bit bạn chỉ cần nhớ trọng số của bit đầu tiên (a_7) là $-$**128**, còn các bit sau vẫn mang trọng số dương ($+64, +32, +16, +8, +4, +2, +1$):

$$A = -a_7 \times 128 + a_6 \times 64 + a_5 \times 32 + a_4 \times 16 + a_3 \times 8 + a_2 \times 4 + a_1 \times 2 + a_0 \times 1$$

Ví dụ: Số `0x8E` = `1000 1110`$_{(2)}$ có giá trị là: $-128 + 8 + 4 + 2 = -$**114**.

Bảng tổng hợp dải giá trị của số nguyên có dấu n-bit:

Số bit (n)	**Dải giá trị thập phân**	**Số âm nhỏ nhất (Hex)**	**Số dương lớn nhất (Hex)**
8-bit (`char`)	$[-128,\ +127]$	`0x80` (-128)	`0x7F` ($+127$)
12-bit (`imm`)	$[-2048,\ +2047]$	`0x800` (-2048)	`0x7FF` ($+2047$)
16-bit (`short`)	$[-32768,\ +32767]$	`0x8000` (-32768)	`0x7FFF` ($+32767$)
32-bit (`int`)	$[-2^{31},\ +2^{31}-1]$	`0x80000000`	`0x7FFFFFFF`

(Lưu ý: Số $-$**1** ở mọi độ dài bit đều gồm toàn chữ `F`: 8-bit là `0xFF`, 16-bit là `0xFFFF`, 32-bit là `0xFFFFFFFF`).

5.1.4 Cộng/Trừ số nguyên và Hiện tượng Tràn số học (Overflow)

1. Tràn nhớ (Carry Out) với số không dấu vs. Tràn số học (Overflow) với số có dấu

- **Với số không dấu:** Tràn xảy ra khi tổng vượt quá $2^n - 1$, làm bit nhớ ra ngoài cùng $C_{\text{out}} = 1$.

- **Với số có dấu (Overflow):** Không phụ thuộc vào C_{out}. Khi cộng hai số **khác dấu**, kết quả **không bao giờ bị tràn**. Hiện tượng tràn số học (Overflow) chỉ xảy ra khi **cộng hai số CÙNG DẤU mà kết quả thu được lại NGƯỢC DẤU** (do vượt ra ngoài dải $[-2^{n-1}, +2^{n-1}-1]$):
  - **Tràn dương:** (Số dương) + (Số dương) $\rightarrow$ Kết quả vượt quá $+2^{n-1}-1$, biến thành **Số âm**.
  - **Tràn âm:** (Số âm) + (Số âm) $\rightarrow$ Kết quả nhỏ hơn -2^{n-1}, biến thành **Số dương**.
  - Đối với phép trừ $X - Y = X + (-Y)$: Tràn xảy ra khi lấy (Số dương) − (Số âm) ra kết quả âm, hoặc lấy (Số âm) − (Số dương) ra kết quả dương.

2. Công thức giải bài toán "Biến kiểu `char` 8-bit bị tràn trong ngôn ngữ C"

Khi cộng hai biến kiểu `char` (8-bit có dấu, dải $[-128, +127]$) trong C:

- Nếu tổng toán học $S = a + b > +127$ (tràn dương): Giá trị thực tế hiển thị ra màn hình là $\mathbf{S - 256}$.

  Ví dụ: $a = 65, b = 65 \implies S = 130 > 127 \implies$ Kết quả $= 130 - 256 = \mathbf{-126}$.

- Nếu tổng toán học $S = a + b < -128$ (tràn âm): Giá trị thực tế hiển thị ra màn hình là $\mathbf{S + 256}$.

  Ví dụ:
  $$a = -91, b = -67 \implies S = -158 < -128 \implies \text{Kết quả} = -158 + 256 = \mathbf{+98}.$$

3. Công thức giải bài toán "Tìm dải giá trị của thanh ghi thứ hai gây tràn khi `add`"

Giả sử thanh ghi thứ nhất chứa số dương $K > 0$ (ví dụ $K = 512 =$ `0x200` hoặc $K = 1024 =$ `0x400`). Khi thực hiện lệnh cộng 32-bit `add rd, rs1, rs2`, tràn xảy ra khi tổng vượt quá số dương lớn nhất 32-bit là MAX = `0x7FFFFFFF`:

$$K + \text{rs2} > \texttt{0x7FFFFFFF} \iff \text{rs2} \geq \texttt{0x7FFFFFFF} - K + 1 = \texttt{0x80000000} - \mathbf{K}$$

Vậy dải giá trị của thanh ghi thứ hai gây tràn số học luôn bắt đầu từ (`0x80000000` $- K$) đến `0x7FFFFFFF` (Lưu ý: điểm kết thúc luôn là `0x7FFFFFFF`, tuyệt đối không chọn đáp án kết thúc bằng `0x80000000` vì `0x80000000` là số âm, cộng với số dương K không bao giờ tràn!).

5.1.5 Nhân/Chia số nguyên và Mở rộng RVM trong RISC-V

1. Các lệnh Nhân và Chia số nguyên (RV32M)

- **Nhân hai thanh ghi 32-bit tạo tích 64-bit:**
  - `mul rd, rs1, rs2`: Lưu **32 bit thấp** của tích vào `rd`.
  - `mulh rd, rs1, rs2`: Lưu **32 bit cao** của tích (nhân 2 số có dấu). (`mulhu` dùng cho 2 số không dấu; `mulhsu` dùng cho `rs1` có dấu, `rs2` không dấu).

- **Chia hai thanh ghi 32-bit:**
  - `div rd, rs1, rs2` (có dấu) và `divu` (không dấu): Lấy **phần thương nguyên**.
  - `rem rd, rs1, rs2` (có dấu) và `remu` (không dấu): Lấy **phần số dư**.

2. Quy tắc dấu của Thương (`div`) và Số dư (`rem`) trong phép chia có dấu

- **Dấu của Thương (Q):** Hai số cùng dấu thì thương dương (+), hai số trái dấu thì thương âm (−).
- **Dấu của Số dư (R):** Số dư **luôn luôn cùng dấu với SỐ BỊ CHIA** (thừa số đứng trước), bất kể số chia âm hay dương! Công thức kiểm tra: Số bị chia = (Số chia × Thương) + Số dư.

Ví dụ: Chia 67 cho −13: Thương $Q = 67 \div (-13) = -\mathbf{5}$, Số dư $R = 67 - (-13) \times (-5) = +\mathbf{2}$.

5.1.6 Số thực dấu phẩy động chuẩn IEEE 754

1. Cấu trúc chuẩn IEEE 754 độ chính xác đơn 32-bit (Single Precision)

Một số thực 32-bit được chia làm 3 trường từ trái sang phải (1 bit - 8 bit - 23 bit):

Bit 31 (1 bit)	**Bit 30:23 (8 bit)**	**Bit 22:0 (23 bit)**
Bit dấu **S**	Số mũ lệch **e** (Biased Exponent)	Phần lẻ định trị **m** (Fraction)

Công thức xác định giá trị số thực X:

$$X = (-1)^S \times (1.m)_{(2)} \times 2^{e-127}$$

Trong đó:

- **Bit dấu S:** Nếu $S = 0$ thì X là số dương (+); nếu $S = 1$ thì X là số âm (−).
- **Số mũ thực E:** $E = e - 127$, suy ra $e = E + 127$ (số 127 gọi là độ lệch Bias của chuẩn 32-bit; với chuẩn 64-bit thì e dài 11 bit, Bias là 1023, m dài 52 bit).
- **Phần định trị $(1.m)_{(2)}$:** Luôn có số **1**. ẩn ở trước dấu phẩy nhị phân. Các bit sau dấu phẩy có giá trị lần lượt là $2^{-1} = 0.5$, $2^{-2} = 0.25$, $2^{-3} = 0.125$, $2^{-4} = 0.0625$, $2^{-5} = 0.03125$...

2. Các quy ước đặc biệt của chuẩn IEEE 754 32-bit (rất hay hỏi trắc nghiệm)

- **Số ±0:** Khi tất cả 8 bit mũ $e = 0$ (`00000000`) và tất cả 23 bit $m = 0$.
- **Số Vô cùng (±∞):** Khi tất cả 8 bit mũ $e = 255$ (`11111111`) và tất cả 23 bit $m = 0$ (tức là `0x7F800000` cho $+\infty$ và `0xFF800000` cho $-\infty$).
- **Không phải là số (NaN – Not a Number):** Khi tất cả 8 bit mũ $e = 255$ (`11111111`) và phần lẻ $m \neq 0$ (có ít nhất 1 bit bằng 1 trong 23 bit của m).

– **Đếm số lượng biểu diễn NaN:** Với $e = \texttt{11111111}$, có 2 cách chọn bit dấu S (0 hoặc 1). Với mỗi dấu S, 23 bit của m có 2^{23} tổ hợp, trừ đi 1 tổ hợp toàn bit 0 (vì toàn 0 là $\pm\infty$) thì còn $(2^{23} - 1)$ tổ hợp. Tính cho cả 2 dấu S, tổng số cách biểu diễn NaN là:

$$2 \times (2^{23} - 1) = \mathbf{2^{24} - 2}$$

5.2 Phần 2: Toàn bộ bài tập Chương 4 và Hướng dẫn giải

5.2.1 Nhóm 1: Nhận diện Cổng logic và Biểu thức Đại số Boolean

Bài tập 5.1 (Ôn tập Giữa kỳ - Câu 1 & Câu 53)

a) (Câu 1): Cho hình vẽ cổng logic có 2 đầu vào A, B, thân cổng hình mũi nhọn giống cổng OR nhưng có thêm **một đường cong song song ở phần đuôi**. Hình đó biểu diễn phép toán logic nào?

- A. XOR B. NAND C. AND D. NOR

b) (Câu 53): Cho hình vẽ cổng logic có 2 đầu vào A, B, thân cổng hình chữ D (giống cổng AND) và có thêm **một vòng tròn nhỏ ở đầu ra** F. Hình đó biểu diễn phép toán logic nào?

- A. NAND B. NOR C. XOR D. AND

Hướng dẫn giải:

- **Ý a):** Cổng có đầu nhọn kèm 1 vạch cong tách rời ở đuôi là ký hiệu chuẩn của cổng **XOR** ($F = A \oplus B$) $\rightarrow$ Chọn **A (XOR)**.
- **Ý b):** Cổng hình chữ D là AND, thêm vòng tròn đảo ở đầu ra thành NOT-AND tức là **NAND** ($F = \overline{AB}$) $\rightarrow$ Chọn **A (NAND)**.

Bài tập 5.2 (Slide Chương 4 - Trang 30 & Ôn tập Giữa kỳ - Câu 70)

Cho sơ đồ mạch logic tổ hợp có 3 đường tín hiệu vào A, B, C (mỗi đường đều có một nhánh đi thẳng và một nhánh đi qua cổng đảo NOT tạo ra A', B', C'). Mạch gồm 3 cổng AND 3-đầu-vào ở tầng trước nối vào 1 cổng OR ở đầu ra F:

- Cổng AND thứ nhất (trên cùng) nối với: nhánh sau cổng NOT của A (A'), nhánh thẳng của B (B), và nhánh sau cổng NOT của C (C').
- Cổng AND thứ hai (ở giữa) nối với: nhánh sau cổng NOT của A (A'), nhánh thẳng của B (B), và nhánh thẳng của C (C).
- Cổng AND thứ ba (dưới cùng) nối với: nhánh thẳng của A (A), nhánh thẳng của B (B), và nhánh sau cổng NOT của C (C').

Mạch trên có biểu thức đại số Boolean là gì?

A. $F = A'B'C' + A'BC + AB'C$

B. $F = ABC' + A'BC + AB'C$

C. $F = A'BC + A'BC' + ABC'$

D. $F = (A + B + C')(A + B' + C)(A' + B + C)$

Hướng dẫn giải:

- Dọc đầu ra của từng cổng AND:
  - Cổng AND trên cùng tạo ra tích: $A'BC'$.
  - Cổng AND ở giữa tạo ra tích: $A'BC$.
  - Cổng AND dưới cùng tạo ra tích: ABC'.
- Cổng OR ở cuối cộng 3 tích trên lại: $F = A'BC' + A'BC + ABC'$ (viết lại theo thứ tự giao hoán phép cộng là $F = A'BC + A'BC' + ABC'$).
- **Đáp án đúng: C ($F = A'BC + A'BC' + ABC'$).**

5.2.2 Nhóm 2: Chuyển đổi và So sánh Số nguyên có dấu (Mã bù 2)

Bài tập 5.3 (Đề Cuối kỳ 2022.2 - #f9c617 & Ôn tập Giữa kỳ - Câu 32)

Giả thiết k là biến số nguyên có dấu 16-bit, cho $k = -68$. Hãy cho biết nội dung k biểu diễn theo số Hexa (có `0x` ở đầu)?

A. `0xFFBC` B. `0x00BC` C. `0x8044` D. `0xFF44`

Hướng dẫn giải:

- **Bước 1 – Biểu diễn số dương $+68$ ở dạng 16-bit:** Ta phân tích $68 = 64 + 4 = 2^6 + 2^2 = \texttt{0100 0100}_{(2)} = \texttt{0x44}$. Biểu diễn đủ 16-bit là:

$$+68 = \texttt{0000 0000 0100 0100}_{(2)}$$

- **Bước 2 – Lấy bù 1 (đảo tất cả các bit):**

$$\text{Bù 1} = \texttt{1111 1111 1011 1011}_{(2)}$$

- **Bước 3 – Lấy bù 2 (cộng thêm 1 vào bit cuối):**

$$-68 = \texttt{1111 1111 1011 1100}_{(2)} = \texttt{0xFFBC}$$

- **Đáp án đúng: A (`0xFFBC`).**

Bài tập 5.4 (Đề Cuối kỳ 2022.2 - #7151, 2024.1 - #0182da, 2020.2 & Ôn tập Giữa kỳ - Câu 48, Câu 58)

Hãy xác định giá trị thập phân của các biến số nguyên **có dấu** sau đây:

a) **(Đề CK 2022.2 & Ôn tập GK Câu 58):** Biến 32-bit có dấu $i =$ `0xFFFFFFF3`.

b) **(Đề CK 2024.1 & Ôn tập GK Câu 48):** Biến 32-bit có dấu $i =$ `0xFFFFFFF4`.

c) **(Đề CK 2020.2):** Biến 8-bit có dấu $i =$ `0x8E`.

Hướng dẫn giải:

- **Mẹo tính nhẩm siêu tốc cho các số âm dạng `0xFFFFFFF...`:** Vì `0xFFFFFFFF` $= -1$, nên khi các chữ số đầu đều là `F`, bạn chỉ cần lấy chữ số Hexa cuối cùng trừ đi 16:
  - **Ý a) Với $i =$ `0xFFFFFFF3`:** Chữ số cuối là $3 \implies i = 3 - 16 = \mathbf{-13}$. (Hoặc đổi $\texttt{3} = \texttt{0011}_2 \xrightarrow{\text{đảo bit}} \texttt{1100}_2 \xrightarrow{+1} \texttt{1101}_2 = 13 \implies \mathbf{-13}$).
  - **Ý b) Với $i =$ `0xFFFFFFF4`:** Chữ số cuối là $4 \implies i = 4 - 16 = \mathbf{-12}$.
- **Giải ý c) Với biến 8-bit có dấu $i =$ `0x8E`:**
  - Đổi sang nhị phân 8-bit: $\texttt{0x8E} = \texttt{1000 1110}_{(2)}$.
  - Vì bit đầu tiên là 1 (trọng số -128), các bit 1 còn lại ở vị trí $8, 4, 2$:

$$i = -128 + 8 + 4 + 2 = -128 + 14 = \mathbf{-114}$$

Bài tập 5.5 (Đề Cuối kỳ 2023.1 - #581f03, Giữa kỳ 2024.1 - #30a33e & Ôn tập Giữa kỳ - Câu 33)

a) (Đề GK 2024.1 & Ôn tập GK Câu 33): Giá trị thập phân của toán hạng hằng số (immediate operand) trong câu lệnh `addi t0, s0, 0x1FF` là bao nhiêu?

- A. -2047 B. -511 C. 2047 D. 511

b) (Đề CK 2023.1 - #581f03): Nếu hằng số 12-bit có dấu trong lệnh `addi` là `0xFFA` (hoặc 16-bit `0xFFFA`), thì giá trị thập phân của toán hạng hằng số đó là bao nhiêu?

- A. 65530 B. -65530 C. 6 D. -6

Hướng dẫn giải:

- **Giải ý a):** Hằng số 12-bit có dấu `0x1FF` bắt đầu bằng chữ số $\texttt{1} = \texttt{0001}_2$ (bit dấu số 11 bằng 0), nên là **số dương**:

$$\texttt{0x1FF} = 1 \times 16^2 + 15 \times 16^1 + 15 \times 16^0 = 256 + 240 + 15 = \mathbf{511} \implies \text{Chọn } \mathbf{D\ (511)}.$$

– **Giải ý b):** Hằng số `0xFFA` (12-bit) hoặc `0xFFFA` (16-bit) có các chữ đầu đều là `F` (bit dấu bằng 1, số âm) và chữ số cuối là `A` = 10. Áp dụng mẹo lấy chữ cuối trừ 16:

$$10 - 16 = \mathbf{-6} \implies \text{Chọn } \mathbf{D}\ (-6).$$

Bài tập 5.6 (Đề Giữa kỳ 2023.2 - Đề 1 Câu 2 & Giữa kỳ 2022.2 - Đề 0008 Câu 4, Đề 0006 Câu 2)

a) (Đề GK 2023.2): Biểu diễn các số nguyên có dấu +105 và −42 theo mã bù hai 8-bit và 16-bit dưới dạng số Hexa.

b) (Đề GK 2022.2 - Đề 0008 & 0006): Hoàn thành bảng chuyển đổi số nguyên 8-bit có dấu giữa Hệ 10, Hệ 2 bù 2 và Hệ 16 bù 2 cho các giá trị: 77; −86; −19; và chuỗi bit `0011 1101` (`3D`).

Hướng dẫn giải:

Giải ý a):

– **Với số** +105**:** Phân tích $105 = 64 + 32 + 8 + 1 = \texttt{0110 1001}_{(2)} = \texttt{0x69}$.

∗ Biểu diễn 8-bit (Hexa): `0x69`.

∗ Biểu diễn 16-bit (mở rộng dấu dương bằng cách thêm `00` vào bên trái): `0x0069`.

– **Với số** −42**:** Ta có $+42 = 32 + 8 + 2 = \texttt{0010 1010}_{(2)}$. Đảo bit được `1101 0101`, cộng 1 được $\texttt{1101 0110}_{(2)} = \texttt{0xD6}$.

∗ Biểu diễn 8-bit (Hexa): `0xD6`.

∗ Biểu diễn 16-bit (mở rộng dấu âm bằng cách thêm `FF` vào bên trái): `0xFFD6`.

Giải ý b) Bảng chuyển đổi 8-bit có dấu:

Hệ 10	**Hệ 2 (Bù 2)**	**Hệ 16**	**Giải thích cách tính**
+77	`0100 1101`	`4D`	$77 = 64 + 8 + 4 + 1 = \texttt{0100 1101}_2$
−86	`1010 1010`	`AA`	$+86 = \texttt{0101 0110}_2 \xrightarrow{\text{bù 2}} \texttt{1010 1010}_2$
−19	`1110 1101`	`ED`	$+19 = \texttt{0001 0011}_2 \xrightarrow{\text{bù 2}} \texttt{1110 1101}_2$
+61	`0011 1101`	`3D`	$32 + 16 + 8 + 4 + 1 = +61$

Bài tập 5.7 (Đề Giữa kỳ 2022.2 - Đề 0009, 0008 Câu 2 & Đề 0006 Câu 1)

a) (Đề 0006 - Câu 1): Hãy sắp xếp các số nguyên 8-bit có dấu sau theo thứ tự **giảm dần**: −108 (hệ 10), `0001 0001` (hệ 2), `91` (hệ 16).

b) (Đề 0009 - Câu 2): Mảng A gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở `0x40000E` lần lượt là: `0xCE`, `0x12`, `0x20`. Hãy sắp xếp chúng theo giá trị **tăng dần**.

c) (Đề 0008 - Câu 2): Mảng A gồm 3 phần tử 8-bit có dấu ở địa chỉ cơ sở `0x400008` lần lượt là: `0x38`, `0x00`, `0xD6`. Hãy sắp xếp chúng theo giá trị **tăng dần**.

Hướng dẫn giải:

- **Nguyên tắc so sánh nhanh số 8-bit có dấu ở dạng Hexa:**
  - Các số bắt đầu bằng chữ số từ `0` đến `7` (`0x00` → `0x7F`) là **số dương** (hoặc 0).
  - Các số bắt đầu bằng chữ số từ `8` đến `F` (`0x80` → `0xFF`) là **số âm** (luôn nhỏ hơn số dương!).
- **Giải ý a):** Đổi cả 3 số về hệ 10:
  - Số thứ nhất: -108.
  - Số thứ hai: $\texttt{0001 0001}_{(2)} = 16 + 1 = +17$.
  - Số thứ ba: $\texttt{0x91} = \texttt{1001 0001}_{(2)} = -128 + 16 + 1 = -111$.
  - Vì $+17 > -108 > -111$, thứ tự giảm dần là: `0001 0001` > -108 > `91`.
- **Giải ý b):** Trong 3 số `0xCE, 0x12, 0x20`, có `0xCE` là số âm (-50), còn $\texttt{0x12} = 18 < \texttt{0x20} = 32$. Thứ tự tăng dần: `0xCE` < `0x12` < `0x20`.
- **Giải ý c):** Trong 3 số `0x38, 0x00, 0xD6`, có `0xD6` là số âm (-42), tiếp theo là `0x00` (0) và `0x38` ($+56$). Thứ tự tăng dần: `0xD6` < `0x00` < `0x38`.

5.2.3 Nhóm 3: Phép cộng/trừ số nguyên và Hiện tượng Tràn số học (Overflow)

Bài tập 5.8 (Đề Cuối kỳ 2022.2 - #6d82ac & Ôn tập Giữa kỳ - Câu 31)

Trong ngôn ngữ lập trình C, biến kiểu `char` là biến dữ liệu số nguyên có dấu 8-bit. Cho chương trình viết bằng C dưới đây, hãy cho biết kết quả của biến `sum` hiển thị trên màn hình khi chạy chương trình đó?

```
#include <stdio.h>
int main() {
    char a = -91;
    char b = -67;
    char sum = a + b;
    printf("sum = %d", sum); // hiển thị sum dạng thập phân
    return 0;
}
```

A. -98 B. -158 C. 158 D. 98

Hướng dẫn giải:

- Tính tổng toán học thông thường: $S = (-91) + (-67) = -158$.
- Vì biến `sum` thuộc kiểu `char` (8-bit có dấu, chỉ biểu diễn được trong đoạn $[-128, +127]$), nên giá trị $-158 < -128$ gây ra **tràn âm (Overflow)**.
- Áp dụng công thức tràn âm 8-bit (cộng thêm $2^8 = 256$):

$$\text{sum} = -158 + 256 = \mathbf{98}$$

- **Đáp án đúng: D (98).**

Bài tập 5.9 (Đề Giữa kỳ 2024.1 - #f2d536, Ôn tập Giữa kỳ - Câu 41 & Giữa kỳ 2022.2 - Đề 0007 Câu 1)

Giả thiết tất cả các số đều được biểu diễn dưới dạng số nguyên 8-bit có dấu. Hãy cho biết kết quả ở hệ thập phân khi máy tính thực hiện các phép toán sau:

a) **(Đề GK 2024.1 & Ôn tập GK Câu 41):** $+65 + 65 =?$

b) **(Đề GK 2022.2 - Đề 0007, Câu 1a):** $5 + 125 =?$

c) **(Đề GK 2022.2 - Đề 0007, Câu 1b):** $(-108) + (-42) =?$

Hướng dẫn giải:

Dải biểu diễn của số nguyên 8-bit có dấu là $[-128, +127]$. Nếu vượt quá $+127$ thì trừ đi 256; nếu nhỏ hơn -128 thì cộng thêm 256:

- **Giải ý a):** $S = 65+65 = 130 > 127$ (tràn dương) $\implies$ Kết quả $= 130-256 = \mathbf{-126}$.
- **Giải ý b):** $S = 5+125 = 130 > 127$ (tràn dương) $\implies$ Kết quả $= 130-256 = \mathbf{-126}$.
- **Giải ý c):** $S = (-108) + (-42) = -150 < -128$ (tràn âm) $\implies$ Kết quả $= -150 + 256 = \mathbf{+106}$.

Bài tập 5.10 (Đề Cuối kỳ 2023.1 - #cd9132 & Ôn tập Giữa kỳ - Câu 34)

Giả sử thanh ghi `s0` đang giữ giá trị thập phân là 1024. Dải giá trị nào của thanh ghi `s1` sẽ gây ra tràn số học (overflow) khi thực hiện lệnh `add s2, s0, s1`?

A. `0x7ffffc00 - 0x7fffffff`

B. `0x7ffffc01 - 0x80000000`

C. `0x7ffffbff - 0x7fffffff`

D. `0x7ffffc00 - 0x80000000`

Hướng dẫn giải:

- Thanh ghi `s0` đang giữ số dương $K = +1024 = 2^{10} = \texttt{0x00000400}$.
- Khi cộng với số dương `s0` = 1024, tràn số học chỉ xảy ra khi `s1` cũng là một số dương đủ lớn sao cho tổng vượt quá số dương lớn nhất 32-bit là `0x7fffffff`.
- Giá trị nhỏ nhất của `s1` bắt đầu gây tràn là:

$$\texttt{s1}_{\min} = \texttt{0x80000000} - \texttt{0x400} = \texttt{0x7ffffc00} \quad (\text{vì } \texttt{0x1000} - \texttt{0x400} = \texttt{0xc00})$$

- Giá trị lớn nhất của số dương 32-bit là `0x7fffffff`. (Không bao gồm `0x80000000` vì `0x80000000` là số âm -2^{31}).
- **Đáp án đúng: A (`0x7ffffc00 - 0x7fffffff`).**

Bài tập 5.11 (Đề Giữa kỳ 2024.1 - #e83dac)

Giả sử thanh ghi `t0` đang giữ giá trị thập phân là 512. Dải giá trị nào của thanh ghi `t1` sẽ gây ra tràn số học (overflow) khi thực hiện lệnh `add t2, t0, t1`?

A. `0x7FFFFD00 - 0x7FFFFFFF`

B. `0x7FFFFDFF - 0x7FFFFFFF`

C. `0x7FFFFE00 - 0x80000000`

D. `0x7FFFFE00 - 0x7FFFFFFF`

Hướng dẫn giải:

- Đổi $K = 512$ sang hệ Hexa: $512 = 2 \times 256 = \texttt{0x00000200}$.
- Giá trị nhỏ nhất của `t1` bắt đầu gây tràn là:

$$\texttt{t1}_{\min} = \texttt{0x80000000} - \texttt{0x200} = \texttt{0x7FFFFE00} \quad (\text{vì } \texttt{0x1000} - \texttt{0x200} = \texttt{0xE00})$$

- Điểm kết thúc của dải số dương là `0x7FFFFFFF`.
- **Đáp án đúng: D (`0x7FFFFE00 - 0x7FFFFFFF`).**

Bài tập 5.12 (Ôn tập Cuối kỳ P2 - Câu 65)

Đoạn lệnh nào sau đây có xảy ra hiện tượng tràn số học (OVERFLOW)?

A.
```
li x8, 0xFFFFFFFF  li x9, 0x80000000  sub x10, x8, x9
```

B.
```
li x8, 0xFFFFFFFF  li x9, 0xFFFFFFFF  sub x10, x8, x9
```

C.
```
li x8, 0xFFFFFFFF  li x9, 0x80000000  add x10, x8, x9
```

D.
```
li x8, 0xFFFFFFFF  li x9, 0xFFFFFFFF  add x10, x8, x9
```

Hướng dẫn giải:

- Ta biết `0xFFFFFFFF` $= -1$, còn `0x80000000` $= -2^{31}$ (là số âm nhỏ nhất có thể biểu diễn bằng 32-bit).
- Kiểm tra từng phương án:
  - Phương án A: $(-1) - (-2^{31}) = 2^{31} - 1 =$ `0x7FFFFFFF` (vừa khít số dương lớn nhất, chưa tràn).
  - Phương án B: $(-1) - (-1) = 0$ (không tràn).
  - **Phương án C:** Cộng hai số âm $(-1) + (-2^{31}) = -(2^{31} + 1) < -2^{31}$. Vì `x9` đã ở tận cùng biên âm, cộng thêm -1 sẽ vượt ra ngoài dải biểu diễn và lật thành số dương `0x7FFFFFFF` $\implies$ **Xảy ra tràn âm (Overflow)!**
  - Phương án D: $(-1) + (-1) = -2$ (không tràn).
- **Đáp án đúng: C.**

5.2.4 Nhóm 4: Lệnh Chia và Lấy phần dư số nguyên (`div, rem`)

Bài tập 5.13 (Đề Giữa kỳ 2024.1 - #054a4a)

Giá trị của các thanh ghi `t0, t1` sau khi thực hiện đoạn mã RISC-V dưới đây là gì?

```
li  s0, 67
li  s1, -13
div t0, s0, s1
rem t1, s0, s1
```

A. `t0` = `0xfffffffb`, `t1` = `0x00000010`

B. `t0` = `0x00000005`, `t1` = `0xfffffffd`

C. `t0` = `0xfffffffb`, `t1` = `0x00000005`

D. `t0` = `0xfffffffb`, `t1` = `0x00000002`

Hướng dẫn giải:

- Ta có số bị chia `s0` $= +67$ và số chia `s1` $= -13$.
- **Tính thương `t0` (`div t0, s0, s1`):**
  - Chia độ lớn: $67 \div 13 = 5$ (vì $13 \times 5 = 65$).
  - Vì hai số trái dấu nên thương mang dấu âm: `t0` $= -5$.
  - Đổi -5 sang Hexa 32-bit (lấy $16 - 5 = 11 =$ `b`): `t0` = `0xfffffffb`.
- **Tính số dư `t1` (`rem t1, s0, s1`):**

* Độ lớn phần dư: $67 - 65 = 2$.
* Theo quy tắc: **Số dư luôn cùng dấu với số bị chia** (`s0` $= +67 > 0$), nên số dư mang dấu dương: `t1` $= +2 =$ `0x00000002`.

– **Đáp án đúng: D (`t0` $=$ `0xfffffffb`, `t1` $=$ `0x00000002`).**

5.2.5 Nhóm 5: Số thực dấu phẩy động chuẩn IEEE 754 32-bit

Phương pháp làm nhanh 2 dạng bài tập IEEE 754 32-bit:

– **Dạng 1 (Cho mã Hexa 32-bit, tìm số thập phân X):**

1. Đổi 3 chữ số Hexa đầu tiên (12 bit đầu) sang nhị phân.
2. Bit đầu tiên là dấu S (0 là +, 1 là −).
3. 8 bit tiếp theo là $e \implies$ Tính số mũ thực $E = e - 127$.
4. Các bit còn lại là phần lẻ $m \implies$ Ghép thêm số **1.** vào trước m rồi nhân với 2^E (dịch dấu phẩy sang phải E vị trí nếu $E > 0$).

– **Dạng 2 (Cho số thập phân X, tìm mã Hexa 32-bit):**

1. Xác định bit dấu S ($X > 0 \Rightarrow S = 0$; $X < 0 \Rightarrow S = 1$).
2. Đổi $|X|$ sang nhị phân, rồi dịch dấu phẩy về ngay sau bit `1` đầu tiên để đưa về dạng chuẩn $1.m \times 2^E$.
3. Tính $e = E + 127$ và đổi e ra 8 bit nhị phân.
4. Ghép [S (1 bit) | e (8 bit) | m (23 bit)] rồi nhóm 4 bit một từ trái sang phải để viết thành 8 chữ số Hexa.

Bài tập 5.14 (Đề Cuối kỳ 2022.2 - #bd4c95, 2024.1 - #e73be8 & Ôn tập Giữa kỳ - Câu 24, Câu 61)

a) **(Đề CK 2022.2 & Ôn tập GK Câu 61):** Giả thiết X là biến số dấu phẩy động theo chuẩn IEEE 754 32-bit, cho nội dung $X =$ `0x40400000`. Hãy xác định giá trị của X dưới dạng thập phân.

b) **(Đề CK 2024.1 & Ôn tập GK Câu 24):** Cho nội dung $X =$ `0x40A00000`. Hãy xác định giá trị của X dưới dạng thập phân.

Hướng dẫn giải:

– **Giải ý a) Với $X =$ `0x40400000`:**

* Đổi 3 chữ số Hexa đầu (`404`) sang 12 bit nhị phân: `0100 0000 0100`$_{(2)}$.
* Tách các trường:
  · Bit dấu $S =$ `0` (số dương).

· 8 bit mũ $e = \texttt{1000 0000}_{(2)} = 128 \implies E = 128 - 127 = 1$.

· Phần lẻ $m = \texttt{100000...} \implies 1.m = 1.1_{(2)} = 1 + 0.5 = 1.5$.

∗ Tính giá trị: $X = +1.5 \times 2^1 = \mathbf{3}$.

– **Giải ý b) Với $X = \texttt{0x40A00000}$:**

∗ Đổi 3 chữ số Hexa đầu (`40A`) sang 12 bit nhị phân: $\texttt{0100 0000 1010}_{(2)}$.

∗ Tách các trường:

· Bit dấu $S = \texttt{0}$ (số dương).

· 8 bit mũ $e = \texttt{1000 0001}_{(2)} = 129 \implies E = 129 - 127 = 2$.

· Phần lẻ $m = \texttt{010000...} \implies 1.m = 1.01_{(2)} = 1 + 0.25 = 1.25$.

∗ Tính giá trị: $X = +1.25 \times 2^2 = 1.25 \times 4 = \mathbf{5}$.

Bài tập 5.15 (Đề Giữa kỳ 2024.1 - #94ec4c & Ôn tập Giữa kỳ - Câu 5, Câu 63)

a) **(Đề GK 2024.1 & Ôn tập GK Câu 5):** Cho số thực dấu phẩy động IEEE 754 32-bit được biểu diễn bởi word `0xC11A0000`. Giá trị của số này ở hệ thập phân là gì?

b) **(Ôn tập GK - Câu 63):** Cho số thực dấu phẩy động IEEE 754 32-bit được biểu diễn bởi `0xC1E00000`. Giá trị của X ở hệ thập phân là gì?

Hướng dẫn giải:

– **Giải ý a) Với $X = \texttt{0xC11A0000}$:**

∗ Đổi 4 chữ số Hexa đầu (`C11A`) sang 16 bit nhị phân:

$$\texttt{C11A}_{(16)} = \texttt{1100 0001 0001 1010}_{(2)} = \underbrace{\texttt{1}}_{S} \;\Big|\; \underbrace{\texttt{1000 0010}}_{e=130} \;\Big|\; \underbrace{\texttt{0011010...}}_{m}$$

∗ Tách các trường:

· Bit dấu $S = 1 \implies$ Số âm $(-)$.

· Số mũ: $e = 128 + 2 = 130 \implies E = 130 - 127 = 3$.

· Phần định trị: $1.m = 1.001101_{(2)}$.

∗ Dịch dấu phẩy sang phải $E = 3$ vị trí:

$$X = -1.001101_{(2)} \times 2^3 = -1001.101_{(2)} = -(8 + 1 + 0.5 + 0.125) = \mathbf{-9.625}$$

– **Giải ý b) Với $X = \texttt{0xC1E00000}$:**

∗ Đổi 3 chữ số Hexa đầu (`C1E`) sang 12 bit nhị phân:

$$\texttt{C1E}_{(16)} = \texttt{1100 0001 1110}_{(2)} = \underbrace{\texttt{1}}_{S} \;\Big|\; \underbrace{\texttt{1000 0011}}_{e=131} \;\Big|\; \underbrace{\texttt{110...}}_{m}$$

∗ Ta có: $S = 1$ (số âm); $e = 128 + 3 = 131 \implies E = 131 - 127 = 4$; $1.m = 1.11_{(2)}$.

∗ Dịch dấu phẩy sang phải 4 vị trí:

$$X = -1.1100_{(2)} \times 2^4 = -11100_{(2)} = -(16 + 8 + 4) = \mathbf{-28}$$

Bài tập 5.16 (Đề Cuối kỳ 2021.2 - Mã đề B5, Câu 1.1)

Chuyển đổi các số biểu diễn dưới dạng Hexa sau về số thực dấu phẩy động theo chuẩn IEEE 754 32-bit:

a) `0x42A78000`

b) `0xC1560000`

Hướng dẫn giải:

– **Giải ý a) Với `0x42A78000`:**

∗ Đổi 5 chữ số Hexa đầu (`42A78`) sang nhị phân:

$$\texttt{0100 0010 1010 0111 1000}_{(2)} = \underbrace{\texttt{0}}_{S=0} \;\Big|\; \underbrace{\texttt{1000 0101}}_{e=133} \;\Big|\; \underbrace{\texttt{01001111000...}}_{m}$$

∗ Ta có $S = 0$ (dương); $E = 133 - 127 = 6$; $1.m = 1.01001111_{(2)}$.

∗ Nhân với 2^6 (dịch dấu phẩy sang phải 6 vị trí):

$$X = +1010011.11_{(2)} = (64 + 16 + 2 + 1) + (0.5 + 0.25) = +\mathbf{83.75}$$

– **Giải ý b) Với `0xC1560000`:**

∗ Đổi 4 chữ số Hexa đầu (`C156`) sang nhị phân:

$$\texttt{1100 0001 0101 0110}_{(2)} = \underbrace{\texttt{1}}_{S=1} \;\Big|\; \underbrace{\texttt{1000 0010}}_{e=130} \;\Big|\; \underbrace{\texttt{1010110...}}_{m}$$

∗ Ta có $S = 1$ (âm); $E = 130 - 127 = 3$; $1.m = 1.101011_{(2)}$.

∗ Nhân với 2^3 (dịch dấu phẩy sang phải 3 vị trí):

$$X = -1101.011_{(2)} = -(8 + 4 + 1 + 0.25 + 0.125) = \mathbf{-13.375}$$

Bài tập 5.17 (Đề Giữa kỳ 2024.1 - #386374 & Ôn tập Giữa kỳ - Câu 45, Câu 65)

Biểu diễn các số thực thập phân sau đây theo chuẩn IEEE 754 32-bit dưới dạng số Hexa:

a) **(Đề GK 2024.1 & Ôn tập GK Câu 45):** -13.75

b) **(Ôn tập GK - Câu 65):** -46.375

Hướng dẫn giải:

Giải ý a) Biểu diễn số -13.75**:**

- Bước 1: Vì $-13.75 < 0$ nên bit dấu $\mathbf{S = 1}$.
- Bước 2: Đổi 13.75 sang nhị phân và chuẩn hóa:

$$13 = \texttt{1101}_2; \quad 0.75 = \texttt{0.11}_2 \implies 13.75 = \texttt{1101.11}_{(2)} = 1.\mathbf{10111} \times 2^{\mathbf{3}}$$

- Bước 3: Tính số mũ lệch $e = E + 127 = 3 + 127 = 130 = 128 + 2 = \texttt{1000 0010}_{(\mathbf{2})}$.
- Bước 4: Ghép 3 trường $[S \mid e \mid m]$ và nhóm 4 bit thành chữ số Hexa:

$$\underbrace{\texttt{1}}_{S}\ \underbrace{\texttt{1000 0010}}_{e}\ \underbrace{\texttt{101 1100 0000...}}_{m} = \texttt{1100 0001 0101 1100 0000...}_{(2)} = \texttt{0xC15C0000}$$

Giải ý b) Biểu diễn số -46.375**:**

- Bước 1: Số âm nên bit dấu $\mathbf{S = 1}$.
- Bước 2: Đổi 46.375 sang nhị phân và chuẩn hóa:
  - Phần nguyên: $46 = 32 + 8 + 4 + 2 = \texttt{101110}_{(2)}$.
  - Phần lẻ: $0.375 = 0.25 + 0.125 = \texttt{0.011}_{(2)}$.
  - Ghép và dịch dấu phẩy sang trái 5 vị trí: $46.375 = \texttt{101110.011}_{(2)} = 1.\mathbf{01110011} \times 2^{\mathbf{5}}$.
- Bước 3: Tính số mũ lệch $e = 5 + 127 = 132 = 128 + 4 = \texttt{1000 0100}_{(\mathbf{2})}$.
- Bước 4: Ghép 3 trường $[S \mid e \mid m]$ và đổi ra Hexa:

$$\underbrace{\texttt{1}}_{S}\ \underbrace{\texttt{1000 0100}}_{e}\ \underbrace{\texttt{011 1001 1000...}}_{m} = \texttt{1100 0010 0011 1001 1000 0000...}_{(2)} = \texttt{0xC2398}$$

Bài tập 5.18 (Ôn tập Giữa kỳ - Câu 19 & Câu 35)

a) **(Câu 19):** Cho số thực dấu phẩy động 32 bit theo chuẩn IEEE 754 viết theo hệ Hexa có 3 chữ số có ý nghĩa lớn nhất (bên trái) là `7F8`, phần còn lại chưa được biết. Khẳng định nào dưới đây có khả năng đúng? (Chọn các khả năng đúng)

- 1. Số thực này biểu diễn giá trị dương xác định
- 2. Số thực này bằng 0
- 3. Số thực này biểu diễn NaN (Not a Number)
- 4. Số thực này biểu diễn vô cùng

b) **(Câu 35):** Có bao nhiêu cách biểu diễn **NaN (Not a Number)** khi dùng số thực dấu phẩy động 32 bit theo chuẩn IEEE 754?

– A. $2^{23}-1$ B. 128 C. $2^{24}-2$ D. 2

Hướng dẫn giải:

– **Giải ý a):** Đổi 3 chữ số Hexa đầu `7F8` sang 12 bit nhị phân:

$$\texttt{7F8}_{(16)} = \texttt{0111 1111 1000}_{(2)} = \underbrace{\texttt{0}}_{S=0} \;\Big|\; \underbrace{\texttt{1111 1111}}_{e=255} \;\Big|\; \underbrace{\texttt{000...}}_{\text{3 bit đầu của } m}$$

Ta thấy cả 8 bit của trường mũ e đều bằng 1 ($e = 255$). Theo quy ước đặc biệt của IEEE 754 (Slide trang 100):

* Nếu 20 bit còn lại đều bằng 0 (tức $m = 0$), thì số này biểu diễn $+\infty$ **(vô cùng)**.
* Nếu trong 20 bit còn lại có ít nhất 1 bit bằng 1 (tức $m \neq 0$), thì số này biểu diễn **NaN (Not a Number)**.
* Vậy cả hai khẳng định **3 (biểu diễn NaN)** và **4 (biểu diễn vô cùng)** đều có khả năng đúng.

– **Giải ý b):**

* Một số là NaN khi trường mũ $e = \texttt{11111111}$ (cố định 1 cách) và trường định trị m (23 bit) khác 0.
* Số cách chọn 23 bit của m sao cho không đồng thời bằng 0 là: $2^{23}-1$.
* Vì bit dấu S ở đầu có thể bằng 0 hoặc bằng 1 (2 cách chọn), nên tổng số cách biểu diễn NaN là:

$$2 \times (2^{23}-1) = 2^{24}-2$$

* **Đáp án đúng: C ($2^{24}-2$).**